Otto-von-Guericke-Universität Magdeburg

Faculty of Mechanical Engineering

Institute of Logistics and Material Flow Technology

Master Thesis

from

Akshay Dineshbhai Chandrola

A Comparative Study of Python Libraries for Discrete Event Simulation

Advisors:

Supervisor

Jun.-Prof. Sebastian Lang
Otto von Guericke University,
Magdeburg, Germany

Co-supervisor

M. Sc. Malte Kania
Otto von Guericke University,
Magdeburg, Germany

OTTO VON GUERICKE UNIVERSITÄT MAGDEBURG | MB | ILM | FAKULTÄT FÜR MASCHINENBAU INSTITUT FÜR LOGISTIK UND MATERIALFLUSSTECHNIK

Aufgabenstellung	Masterarbeit		MA-ILM-0275
für	Akshay Dineshbhai Chandrola	Studiengang: Andere	Matr.-Nr.: 234905

A Comparative Study of Python Libraries for Discrete Event Simulation

Eine vergleichende Studie von Python-Bibliotheken für ereignisdiskrete Simulation

Discrete Event Simulation (DES) is a technique employed to model the behavior of real-world processes, systems, or networks by tracking changes that occur at specific points in time. Unlike continuous simulations, which progress gradually over time, state changes in DES models are subject to discrete events. In recent years, there has been a noticeable shift from using commercial tools, such as Plant Simulation, to leveraging Python as plain programmig language for DES modeling. This trend is driven by Python's flexibility, accessibility, and its extensive range of Open Source libraries.

This thesis aims to provide a comprehensive comparison of various Python libraries for DES by implementing a case study. The case study should also incorporate processes that are complicated to model with DES, for instance freely moving objects such as Automated Guided Vehicles (AGVs) and humans. The comparison should take into account several evaluation criterias, for instance modeling effort and computational performance, in order to offer valuable insights into the strengths and weaknesses of each library.

Schwerpunkte:

- Theoretical Foundation: Discrete Event Simulation (DES), Use cases for DES
- State of the Art: Conduction of a literature review of existing Python libraries for DES, examining their main features, specific application domains, and key differences
- Case Study Planning: Conceptualization of a case study for comparing the researched Python DES libraries; identification of KPIs and planning of a procedure for a structured experimental comparison
- Case Study Implementation: Implementation of the case study with the investigated Python libraries
- Evaluation: Comparison of libraries based on defined KPIs and the procedure for a structured experimental comparison
- Critical appraisal of the own work, recommendations and outlook on future research

Die Arbeit ist nach den geltenden Normen für die Anfertigung von Abschlussarbeiten an der Universität Magdeburg zu erstellen. Einzureichen sind zwei gedruckte und gebundene Prüfungsexemplare mit je einer digitalen Fassung auf CD oder USB-Stick (Texte in Word und pdf, Grafiken als Originaldatei, digital vorhandene Quellen). Zusätzlich ist eine PowerPoint-Summaryfolie entsprechend der Vorlage im Studentenportal abzugeben.

Erstgutachter: Jun.-Prof. Dr.-Ing. Sebastian Lang (ILM)
Zweitgutachter: M. Sc. Malte Kania (ILM)

Ausgabedatum: 30. JULI 2024
Abgabedatum:

S. Lang

Jun.-Prof. Dr.-Ing. Sebastian Lang

OTTO VON GUERICKE UNIVERSITÄT MAGDEBURG | MB | ILM

FAKULTÄT FÜR MASCHINENBAU
INSTITUT FÜR
LOGISTIK UND MATERIALFLUSSTECHNIK

Aufgabenstellung	Masterarbeit		MA-ILM-0275
für	Akshay Dineshbhai Chandrola	Studiengang: Andere	Matr.-Nr.: 234905

A Comparative Study of Python Libraries for Discrete Event Simulation

Eine vergleichende Studie von Python-Bibliotheken für ereignisdiskrete Simulation

Die Arbeit ist nach der Gestaltungsrichtlinie und den Hinweisen zur Durchführung von Abschlussarbeiten der FMB auszuführen und in zweifacher Ausfertigung fristgerecht im Prüfungsamt der FMB, hilfsweise im Campus Service Center, einzureichen. Das Abgabedatum muss amtlich dokumentiert werden!

Unterschrift: _____

Akshay Dineshbhai Chandrola

Akshay Dineshbhai, Chandrola:
A Comparative Study of Python Libraries for Discrete Event Simulation
Master Thesis, Otto-von-Guericke-Universität Magdeburg, 2025

Acknowledgement

I would like to express my deepest gratitude to my supervisors, Jun.-Prof. Sebastian Lang and M. Sc. Malte Kania, for their continuous support, guidance, and encouragement throughout the development of this work. Their expertise and insightful feedback have been invaluable in shaping the direction and content of this thesis.

I would also like to thank Mareike Hergenröther for her support and encouragement during this process. Furthermore, I extend my heartfelt thanks to my family and friends for their unwavering support and understanding during this challenging time. Their encouragement, patience, and belief in me have been instrumental in keeping me motivated and focused throughout this journey.

Finally, I would like to thank everyone who contributed to the success of this work through their input, feedback, and support. Your contributions have been essential to the completion of this work.

Abstract

This master's thesis compares Python libraries for discrete-event simulation (DES). DES is a technique used to model the behavior of real-world processes, systems or networks by tracking changes that occur at discrete points in time. In recent years, the focus has shifted from commercial tools such as Plant Simulation to open-source libraries in Python. This trend is due to the flexibility, accessibility and extensive open-source community of Python.

In the context of this work, the Python libraries SimPy, Salabim, PyDSOL-core, Akatosh and the HADES-framework are analyzed and compared using a case study approach. The case study involves complex processes such as the modelling of free moving objects, including automated guided vehicle (AGVs) and humans. The aim is to provide a thorough analysis of the strengths and weaknesses of the libraries by implementing and evaluating them based on defined criteria, like real-time simulation and resource management capabilities.

The work begins with a theoretical introduction to DES and an overview of current applications. Followed by a literature review of existing Python libraries, highlighting their main features and differences. The case study is planned, implemented and libraries are systematically compared using pre-defined criteria. Finally, the work provides a critical evaluation of the results, recommendations for the use of libraries and an outlook on future research.

Keywords: Discrete event simulation, Python, SimPy, Salabim, PyDSOL-core, Akatosh, HADES-Framework

Abstract (in German)

Diese Masterarbeit vergleicht Python-Bibliotheken für ereignisdiskrete Simulationen (DES). DES ist eine Technik, die verwendet wird, um das Verhalten von realen Prozessen, Systemen oder Netzwerken zu modellieren, indem Veränderungen verfolgt werden, die zu diskreten Zeitpunkten auftreten. In den letzten Jahren hat sich der Fokus von kommerziellen Tools wie Plant Simulation auf Open-Source-Bibliotheken in Python verlagert. Dieser Trend ist auf die Flexibilität, Zugänglichkeit und die umfangreiche Open-Source-Community von Python zurückzuführen.

Im Rahmen dieser Arbeit werden die Python-Bibliotheken SimPy, Salabim, PyDSOL-core, Akatosh und das HADES-framework anhand eines Fallstudienansatzes analysiert und verglichen. Die Fallstudie umfasst komplexe Prozesse wie die Modellierung frei beweglicher Objekte, einschließlich automatisierter Transportfahrzeuge (AGVs) und Menschen. Ziel ist es, durch Implementierung und Evaluierung der Bibliotheken anhand definierter Kriterien, wie Echtzeitsimulation und Ressourcenmanagementfähigkeiten, eine umfassende Analyse der Stärken und Schwächen der Bibliotheken bereitzustellen.

Die Arbeit beginnt mit einer theoretischen Einführung in DES und einem Überblick über aktuelle Anwendungen. Darauf folgt eine Literaturrecherche zu bestehenden Python-Bibliotheken, die deren Hauptmerkmale und Unterschiede hervorhebt. Die Fallstudie wird geplant, implementiert und die Bibliotheken systematisch anhand vordefinierter Kriterien verglichen. Abschließend liefert die Arbeit eine kritische Bewertung der Ergebnisse, Empfehlungen für den Einsatz der Bibliotheken und einen Ausblick auf zukünftige Forschungsarbeiten.

Declaration of Authorship

I, Akshay Dineshbhai Chandrola,

- declare that I have written the submitted work independently and without unauthorized help,
- that I have not used any resources beyond those explicitly permitted in advance and stated by me,
- that I marked parts of work that are taken from other documentations (including internet sources and AI-based tools such as OpenAI (2025a) indicating the source, and
- that I have not submitted the present work in the same or a similar form to another examination office.

I confirm that I have used AI-based tools such as ChatGPT (OpenAI, 2025a; OpenAI, 2025b; OpenAI, 2025c) for some research purposes, but that I have carefully checked all content and processed it independently.

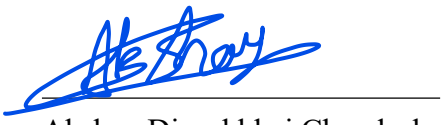
I consent

- that my work can be checked for plagiarism using software. I am aware that submitting a plagiarized paper is a serious academic offence and that deception will be punished in accordance with the study and examination regulations that apply to me.

I am aware

- that in the event of an attempt at deception, the work will not be recognized,
- that if I use AI-based tools in my work, I bear full responsibility for any incorrect or distorted content, incorrect references, violations of data protection and copyright law or plagiarism generated by AI.

My own intellectual achievement was always at the forefront, and I was always in control of the process.



Akshay Dineshbhai Chandrola
Magdeburg, 14.01.2025

Contents

Acknowledgement V
Abstract VI
Abstract (in German) VII
Declaration of Authorship VIII
List of Figures XII
List of Tables XIII
List of Abbreviation XIV
1. Introduction 1
1.1 When simulation is the right approach 2
1.2 When simulation should not be used 2
1.3 Advantages and disadvantage of simulation 3
1.4 Type of Simulation 4
1.4.1 Discrete event simulation (DES) 4
1.4.2 Continuous simulation 4
1.4.3 Agent-based simulation (ABS) 4
1.4.4 Monte Carlo simulation (MCS) 4
Comparison of the types of simulation 5
1.5 Objective and Problem statement 5
1.6 Structure of the work 6
2. Theoretical Foundation 7
2.1 Historical Evolution of DES 7
2.2 Main components DES 8
2.3 Performance metrics, analysis methods and verification/validation theory 10
2.3.1 Performance metrics 10
2.3.2 Analysis Methods 11
2.3.3 Verification and validation Theory 11
2.4 Mathematical fundamentals 12
2.4.1 Basics of queuing theory 12
2.4.2 Stochastic Processes 14
2.4.3 Monte Carlo Methods 14
2.4.4 Statistical Evaluation 14
2.5 Algorithms for DES 15
2.5.1 Event-Driven Algorithm 15
2.5.2 Time-driven algorithm 16

2.5.3 Event calendar management ... 17
2.5.4 Resource allocation algorithms ... 18
2.6 Advantages and Disadvantages of DES ... 19
2.7 Applications of DES ... 20
2.8 DES in Present-day Research and Development ... 21
3. State of the Art ... 23
3.1 Methodology for the literature review ... 23
3.1.1 Research objective ... 23
3.1.2 Search Strategy ... 23
3.2 Overview of selected Python libraries for DES ... 27
3.2.1 Detailed overview of SimPy ... 27
3.2.2 Detailed overview of Salabim ... 28
3.2.3 Overview of PyDSOL-core ... 29
3.2.4 Overview of Akatosh ... 30
3.2.5 Overview of HADES-framework ... 31
Comparison of selected libraries ... 33
4. Evaluation criteria ... 34
4.1 Features for comparison ... 34
4.2 Methodology for evaluation ... 36
4.2.1 System configuration ... 36
4.2.2 Development environment ... 36
4.2.3 Steps for evaluation ... 36
4.2.4 Visualization of results ... 37
5. Design and implementation of the case study ... 38
5.1 Case study design ... 38
5.1.1 Case study overview ... 38
5.1.2 Theoretical calculations ... 41
5.2 Case-Study implementation ... 45
5.2.1 Implementation strategy ... 45
6. Evaluation of selected libraries ... 48
6.1 Evaluation of SimPy ... 48
6.2 Evaluation of Salabim ... 50
6.3 Evaluation of PyDSOL-core ... 53
6.4 Evaluation of Akatosh ... 55
6.5 Evaluation of HADES-framework ... 58

Challenges faced during the implementation 60
7. Conclusion 63
7.1 Strengths of this work 63
7.2 Limitations of this work 63
7.3 Comparative analysis of selected libraries 63
7.4 Practical applications of DES libraries in industry 66
7.5 Recommendations for future research 66
Bibliography 67
Appendix 70
SimPy Implementation code: 70
Salabim implementation code 74
PyDSOL-core implementation code 79
Akatosh implementation code 84
HADES-framework implementation code 89

List of Figures

Figure 1. Simulation as experimentation 1
Figure 2. Event-driven algorithm 15
Figure 3. Time-driven algorithm 16
Figure 4. Event calendar management 17
Figure 5. Resource allocation algorithm 18
Figure 6. Selection process of the chosen libraries. 25
Figure 7. Flow Diagram of the case study process. 40
Figure 8. Implementation of all steps for the case study. 46
Figure 9. Function of the arrival process in SimPy. 48
Figure 10. Code snippet for the resource management in SimPy. 49
Figure 11. Code that describes the simulation of mobile resources in SimPy. 49
Figure 12. Visual diagrams for SimPy created with matplotlib. 50
Figure 13. Code snippet for the customization of the arrival processes in Salabim. 51
Figure 14. Code snippet for the resource management with Salabim. 51
Figure 15. Code snippet for the simulation of moving resources in Salabim. 52
Figure 16. 2D animation of the case study with Salabim. 52
Figure 17. Code snippet example for logging AGV failures in Salabim. 53
Figure 18. Code snippet of the customization of the arrival processes in PyDSOL-core. 54
Figure 19. Code snippet of the resource management with PyDSOL-core. 54
Figure 20. Code snippet for the customization of the arrival process with Akatosh. 56
Figure 21. Code snippet for the customization of the arrival process with HADES Framework. 58
Figure 22. Code snippet for the resource management with HADES Framework. 59
Figure 23. Code snippet for the statistical analysis with HADES Framework. 59

List of Tables

Table 1. Comparison of simulation types. 5
Table 2. Initial comparison of selected libraries. 33
Table 3. Features for comparison of the selected libraries. 35
Table 4. Constants and assumptions for the case study. 41
Table 5. Comparative analysis of the selected libraries for this case study. 64

List of Abbreviation

ABS	Agent-Based Simulation
AGV	Automated Guided Vehicle
ANOVA	Analysis of Variance
DEVS	Discrete Event System Specification
DES	Discrete-Event Simulation
DSOL	Distributed Simulation Object Library
FEL	Future Event List
FIFO	First In, First Out
HADES	Hierarchical Asynchronous Discrete-Event Simulation
KPI	Key Performance Indicator
LIFO	Last In, First Out
MCS	Monte Carlo Simulation
NOK	Not Okay
PyDSOL	Python Distributed Simulation Object Library
PyPI	Python Package Index
SEAD	Suppression of Enemy Air Defenses
SPT	Shortest Processing Time
V&V	Verification and Validation

1. Introduction

Simulation is a powerful tool to model and analyze complex systems, enabling the study of real or artificial systems behavior under different conditions without implementation of physical systems. It can be performed by hand or with the help of computer, using actual data available or artificially generated data to simulate actual scenarios of the real system. Processes that take place in this system, are modelled using mathematical models, algorithms or computer-aided tools (Banks et al., 2014). Simulation is widely used in numerous fields including engineering, economics, medicine, environmental science and social science (Banks et al., 2014; Law and Kelton, 2000). Simulation serves as a decision-support tool that minimizes risk, reduces costs and optimizes system performance.

A basic example is the simulation of a queue in a supermarket. A model can be created that simulates the arrival of customers and the service time of the cashiers. By varying the input parameters – such as the number of open checkouts or the average arrival rate of customers – it is possible to examine how queue and waiting times change. This simulation helps to develop efficient strategies, such as the optimal number of cashiers or the best distribution of customer flows, to minimize waiting times and improve customer service (Robinson, 2005).

Simulation differs from traditional methods in allowing deeper understanding of the changes and processes of a system overtime. While classical approaches often only provide snapshots, simulation shows us how system develops, how different components of the system interacts with each other and how certain decision impacts the behavior of the system. This makes it particularly valuable in situations involving dynamics, such as optimizing production processes, traffic control or managing hospital staff.

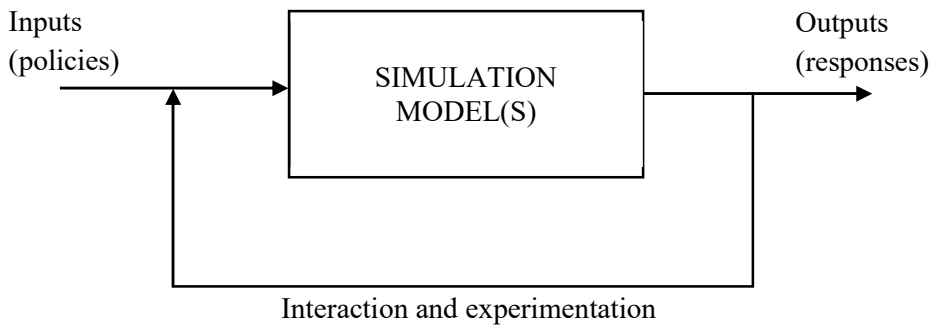


Figure 1. Simulation as experimentation

The diagram in Figure 1, based on Pidd (2004), illustrates the basic approach of simulation as experimentation. Here, simulation serves as a test environment for various input policies in order to analyze their effects. The inputs (policies) are entered into the simulation model, which then produces various responses or reactions. This iterative process of interaction and experimentation makes it possible to understand the effects of different strategies under controlled conditions before applying them in real scenarios. Unlike traditional models, which provide static results, simulation offers the possibility to test dynamic processes and find optimized solutions for complex systems.

A notable instance is the use of simulation throughout the COVID-19 pandemic. These simulations enabled a more accurate prediction of the virus's transmission and identified which interventions – like lockdowns, mask requirements, or vaccination efforts – are most effective. These models enabled policymakers to make informed decisions before the measures were implemented in real life. Simulations helped to save lives because they provided a safe testing ground to explore the effects of different strategies without taking real risks (Sun et al., 2023). What is particularly impressive about this, is that simulations enable users to make complex relationships understandable and supports to make informed decisions, even when the uncertainty is great.

This chapter initially discusses when to use simulation, its advantages and disadvantages, concepts of system and model and comparing different methodologies used for simulation. Lastly, the problem statement and an outline of this work is provided.

1.1 When simulation is the right approach

Simulation is an extremely powerful tool that can be used in a variety of scenarios. However, there are certain circumstances in which simulation is particularly useful such as:

- **High system complexity:** Simulation is particularly useful when a system involves many interacting variables or elements which are difficult to solve analytically. A good example is a manufacturing process involving multiple machines, inventories and production steps. Here, simulation provides a way to take all relevant factors into account without getting lost in complex mathematical formulas (Robinson, 2005).
- **Uncertain or variable data:** When the input data is uncertain or variable, as in stochastic processes or scenarios with unpredictable influences, simulation offers a way to model these uncertainties and analyze their effects. It can therefore help to understand the probabilities of different scenarios and make informed decisions (Law & Kelton, 2000).
- **Expensive or impossible prototypes:** In many sectors, such as aviation or the automotive industry, physical tests or prototypes can be extremely expensive or even dangerous. Simulations make it possible to carry out such tests virtually, avoiding the need for physical prototypes and saving time and money (Banks et al., 2014).
- **Analyzing future scenarios:** Simulations are particularly useful for analyzing long-term developments, such as urban planning, climate change projections or economic planning. They offer the possibility to model different future scenarios and analyze how various factors could affect the system (Sargent, 2013).

1.2 When simulation should not be used

Although simulations offer many advantages, there are also situations in which their use is not justified. Few of these situations are listed below:

- **A simple analytical model is sufficient:** If the system can be described with mathematical formulas and an analytical solution is available, then the use of a simulation

is often not necessary. In cases where simple models already exist, a detailed simulation is unnecessary and represents an effort that would be disproportionate to the benefits.

- **Missing or insufficient data:** A simulation is based on input data, and this data must be accurate and reliable. If insufficient or untrustworthy data is only available, the simulation will produce erroneous or inaccurate results. In such cases, it is better to resort to other form of analysis.
- **High modelling effort:** Creating, validating and executing simulations can be expensive and time-consuming in many cases. If the modelling effort is disproportionally high, For example, for simple or already known processes, a simulation is not justified and more efficient methods should be considered (Law & Kelton, 2000).
- **Rough estimates are sufficient:** If a rough estimate or qualitative analysis is sufficient to make a decision, a complex simulation is often redundant. In such cases, it may be more effective to use simpler models or empirical values (Pidd, 2004).

1.3 Advantages and disadvantage of simulation

Simulation offers numerous advantages, but it also has some limitations that need to be considered. In the following, few major advantages and disadvantages are discussed.

Advantages:

- **Flexibility:** One of the greatest strengths of simulation is its adaptability. It can be applied to almost any system, whether it is a production process, a financial model or the simulation of social interactions. This makes it a versatile tool that can be used in a variety of industries (Banks et al., 2014).
- **Insight into complex systems:** Simulations enable an in-depth analysis of systems that involve many variables and complex interactions. By modelling real-time processes, simulations can help to understand and optimize a system's dynamics (Robinson, 2005).
- **No risks to real systems:** Another advantage of simulation is that it allows tests and experiments to be carried out without endangering the real systems. This is particularly important in safety-critical areas such as aviation or medical research, where failed attempts could have serious consequences.

Disadvantages:

- **High effort:** Building and validating a model can be time-consuming and expensive. Accurate modelling often requires extensive data collection and detailed planning, which can significantly increase the effort (Law & Kelton, 2000).
- **Dependency on data:** The accuracy of the simulation results is directly dependent on the quality of the input data used. If this data is missing or inaccurate, simulation results may also be flawed. This is a major challenge, especially in areas where data for a situation is uncertain (Pidd, 2004).
- **Complexity of interpretation:** Simulations often produce a large amount of data, which can be complex and difficult to understand. Expert knowledge is often required

to interpret this data correctly. This can make it difficult for non-specialists to use the simulation.

1.4 Type of Simulation

Simulation is a versatile tool with different types and has its specific application area and requirements. Each type has its own strengths. Here is an overview of the four main types of simulation:

1.4.1 Discrete event simulation (DES)

Discrete event simulation (DES) is particularly suitable for systems in which the state changes as result of individual, discrete events. An example would be a production line where machine orders are processed one after the other. DES models the sequence of events such as 'machine starts', 'order is processed' or 'machine stops'. This type of simulation is often used in logistics, manufacturing and healthcare to identify bottlenecks and optimize processes.

A major advantage of DES is that it is ideal for simulating clearly defined events. It is often used in scenarios where resources are limited and their optimal use needs to be ensured (Banks et al., 2014).

1.4.2 Continuous simulation

In contrast to DES, continuous simulation deals with systems which change continuously over time. This type is often used to model physical phenomena such as the flow of water in a river or behavior of electrical circuits.

For example, to understand how a dam works, continuous simulation can calculate how the water level in the dam changes when the outflow or inflow fluctuates. This type of simulation is particularly useful when dynamic processes need to be modelled and analyze, that cannot be described by individual events (Law & Kelton, 2000).

1.4.3 Agent-based simulation (ABS)

Agent-based simulation (ABS) models systems through the interaction between agents that have individual rules and behaviors. A typical example would be traffic simulation in a city, where vehicles, pedestrians and traffic lights are modelled as individual agents.

Each agent acts based on set rules, and the interactions between them can model complex systems. For example, a simulation could show how interaction between an autonomous vehicle affects the traffic flow (Macal and North, 2014). The main advantage of this method is its flexibility and ability to analyze emergent patterns of behavior.

1.4.4 Monte Carlo simulation (MCS)

Monte Carlo simulation uses random processes to analyze uncertainties in the systems. This method is frequently used to simulate system related to finances, for example to assess the risk of an investment. By generating many random scenarios, MCS can calculate the probability distribution of possible outcomes.

A typical example would be the evaluation of a portfolio: what happens if the stock market falls by 10% or interest rates rise? MCS makes it possible to answer such questions and make informed decisions under uncertainty (Metropolis & Ulam, 1949). The biggest advantage of this method is its ability to map stochastic variability.

Comparison of the types of simulation

The following table (Table 1) provides an overview of the characteristics of the different types of simulation. The choice of the appropriate type of simulation depends on the specific requirements of the system and the objectives of the analysis.

Table 1. Comparison of simulation types.

Type of Simulation	DES	Continuous	ABS	MCS
Model type	Discrete events	Continuous processes	Individual agents	Random samples
Area of application	Manufacturing, logistics	Physical systems	Social systems, Markets	Risk analysis, Finance
Modelling/ Calculation style	Event-driven	Time-dependent	Rule-based interactions	Statistical methods
Advantages	Efficiency for event systems	Precision for dynamic systems	Flexibility for complex systems	Handling of uncertainties
Disadvantages	High modelling complexity	High computational efforts	High modelling complexity	Requires large samples

1.5 Objective and Problem statement

The main objective of this master's work is to conduct a comprehensive analysis of Python libraries for DES. After a detailed review of Python libraries released or last updated from February 2023 to March 2024, this work conducts comparative analysis of five python libraries: Salabim, SimPy, PyDSOL-core, HADES-framework and Akatosh. The aim of the work is to evaluate the strengths and weaknesses of these libraries by implementing a case study that includes complex scenarios such as the interaction between automated guided vehicles (AGVs) and human workers.

The case study is used to analyze key performance indicators (KPIs) and features, including resource management, real-time simulation and modelling flexibility. By examining these factors, the best tools for specific applications in DES will be determined and practical recommendations for researchers and practitioners in simulation science will be developed.

Problem statement

Python, as a programming language, has grown into a preferred platform for implementing DES due to its flexibility, accessibility, rich ecosystem of libraries, capability to support other modern systems and cost-effectiveness. The shift from proprietary tools like Arena or Plan Simulation to Python reflects an immense trend in embracing open-source solutions and enabling greater customization and collaboration. Libraries like SimPy, PyDSOL-core, and others have brought sophisticated DES capabilities to Python developers, allowing for diverse applications ranging from logistics, healthcare to disaster mitigation ((Ullrich & Lückerath, 2017).

There is a wide range of libraries available, however, there is often a lack of a systematic and practice-oriented comparative study that shows which library is best suited for which specific case with mobile resources. This work aims to fill this gap by not only providing a theoretical analysis but also implementing libraries and evaluating them using a concrete case study.

1.6 Structure of the work

This work is structured to comprehensively address the research objectives, covering both theoretical foundations and practical applications. The introduction establishes the fundamentals of simulation, including its appropriate use cases, advantages, and limitations, while also outlining the research objectives and the underlying problem statement. Building on this, the theoretical foundation delves into DES, discussing its historical development, core components, and mathematical underpinnings such as queuing theory and stochastic processes. It also highlights DES algorithms and their modern applications in various industries.

The state of the art presents a detailed analysis of selected Python libraries for DES, including SimPy, Salabim, PyDSOL-core, Akatosh, and the HADES framework. This section compares the libraries, emphasizing their unique features and use cases. The evaluation criteria follow, detailing the framework and metrics used to assess these libraries. The design and implementation of the case study then illustrate a practical application based on a multiphase manufacturing process, supported by theoretical calculations, process flow diagrams, and a step-by-step implementation strategy.

The evaluation section offers a performance analysis of the selected libraries, examining their strengths, challenges, and comparative effectiveness based on the case study. Finally, the conclusion summarizes the findings, discussing the libraries' strengths and limitations while providing recommendations for future research and practical applications of DES in industrial scenarios. The work concludes with a bibliography and an appendix, including implementation code to ensure transparency and reproducibility.

2. Theoretical Foundation

DES is computational methodology designed to model systems that evolve over time through discrete changes in state. These changes, referred to as events, occur at specific points in time and are the primary drivers of the simulation. DES is particularly suited to systems with stochastic or dynamic elements, such as manufacturing lines, call centers, healthcare, facilities, and military operations. DES's mathematical foundations, algorithms and theoretical concepts form the basis for its successful application. The following chapter examines the fundamental aspects of DES in detail.

2.1 Historical Evolution of DES

The history of DES dates back to the mid-20th century with the rise of computer technology. Initially developed to model manufacturing systems, DES has since expanded into diverse fields. The following milestones mark the most significant developments in the history of DES.

Integration with computers played a significant role in the evolution of DES. Early DES models relied heavily on specialized mainframe computers, which were costly and had limited accessibility. During the 1950s and 1960s, DES models were predominantly used in industries to simulate production lines and optimize manufacturing processes (Fishman, 2001). The transition from mainframe computers to personal computers in the 1980s marked a pivotal moment for DES. Affordable and more powerful personal computers made DES not only more accessible but also practical for a broader range of users. This advancement significantly increased the popularity of DES, encouraging companies, institutions, and universities to integrate it into their workflows (Kelton et al., 2004).

Another important development in DES history was the development of specialized software solutions. In the early years, DES was heavily dependent on handcrafted models and algorithms developed for specific applications. With the introduction of commercial and open-source solutions, the creation and execution of simulation model became simple. Some of the important software solutions that emerged include:

- **Arena:** One of the earliest commercial software solutions for DES, with user-friendly interfaces. It promoted the use of DES in the industry (Kelton et al., 2004).
- **SimPy:** An open-source library specifically developed for Python which quickly gained popularity due to its simplicity and flexibility. It enabled the modelling of complex systems and promoted the application of DES in educational institutes (Matloff, 2008).
- **Salabim:** Another python based open-source solution that focuses on modelling resource based systems and stands out for its efficiency and ease of use (Ruud van der Ham, 2024).

These tools significantly reduced the required effort for modelling and simulation and furthermore allowed users with minimum programming language to create DES models.

As the technology and software developed, DES became an essential tool in strategic decision-making, particularly due to its versatility. It found widespread application across industries such as logistics, manufacturing, healthcare, and defense. The ability of DES to model complex systems and predict the effects of various decisions made it indispensable for organizations aiming to optimize operations and plan for future challenges. Strategic applications of DES encompass areas like:

- **Logistics:** DES is used to optimize supply chains and warehousing processes by simulating scenarios such as inventory management and transport routes (Robinson, 2005).
- **Healthcare:** DES has been used in hospitals to analyze patient flows, optimize staff schedules and identify bottlenecks in emergency departments (Doudareva & Carter, 2022).
- **Defense:** In military applications, DES has been used to model complex scenarios such as troop movements and resource management to improve the efficiency of operations (Bagdatli & Mavris, 2012).

The ability to realistically simulate complex systems enabled decision-makers to make informed and risk-free decisions and therefore significantly increased the influence of DES on strategic planning.

In recent years, DES has evolved through the integration of modern technologies. Noteworthy trends include digital twins, which connect DES with real-time data, enabling dynamic decision-making, particularly in manufacturing and logistics (Karanjkar & Joshi, 2023). Another significant trend is the development of hybrid models, which combine DES with continuous and agent-based modeling to address complex multi-scale problems, expanding the potential for simulation in diverse applications (Atalan et al., 2022). Additionally, machine learning has been integrated into DES, improving the accuracy and adaptability of simulation models by incorporating predictive algorithms (Agalianos et al., 2020). These advancements are pushing the boundaries of DES, allowing for more efficient, flexible, and accurate simulations in a variety of industries.

The development of discrete event simulation reflects the advancement of computer technology and the increasing importance of data-driven decision making. From the first mainframe-based models to modern, hybrid approaches, DES has undergone an impressive transformation. The continuous improvement of software tools and the integration of new technologies ensure their relevance as an indispensable tool for analyzing and optimizing complex systems.

2.2 Main components DES

DES is based on modelling systems in which changes are triggered by discrete events. To accurately model such systems, it is necessary to define the main components included in DES. These main components include entities, resources, events, attributes, states, queues and event calendars. Each of these components plays an important role in modelling and analyzing a system.

Entities represent the movable objects or units within a system that are tracked by the model**.** They are the central elements which flow through the system and are processed by the resources. Entities often have specific properties or attributes that distinguish them from other entities. For instance, patients in a hospital could have different levels of urgency or treatment requirements. Some examples of entities include:

- In a production line, entities can be products or workpieces.
- In a hospital, patients could be modelled as entities.
- In the transport system, vehicles or pedestrians could be the entities.

Resources are the elements used by entities to perform a process or change a state. These can be limited, which in many cases lead to bottlenecks or queues. A key challenge in modelling resources is their optimal allocation. In a call center scenario, this could mean ensuring that enough employees are available to handle calls to minimize waiting times. Resources can include:

- **Human resources:** Doctors, nurses, call center employees.
- **Machines and devices:** Production machines, computer equipment, vehicles.
- **Infrastructure:** Hospital beds, production stations, storage units.

Events are triggers that cause changes in the system's state. They define when and how these changes occur in the model. Events are discrete, occurring at specific points in time, and typically managed through a future event list (FEL), where the next event for each entity is listed. These events determine the process of the simulation and control when state changes occur in the system (Fishman, 2001). Some examples of events are:

- **Arrival events:** A customer enters a queue or a patient arrives at the hospital.
- **Processing results:** A machine finishes processing a workpiece or a call center conversation is completed.
- **Failure events:** A machine failure or a delay in the delivery process.

Attributes describe the properties or characteristics of the entities or resources in the system. These properties can be static or dynamic and influence how entities are treated within the system. The attributes make it possible to represent the diversity of the entities in the model and to model specific behaviors. Few examples of attributes are:

- A patient can be assigned attributes such as age, disease type or treatment time.
- A product on a production line can be assigned properties such as weight, color, or production type.

State describes the current status of the system at a specific point in time. The state can change as a result of events and are crucial for analyzing system behavior. Tracking the state is essential to evaluate the performance of the system and identify opportunities for improvements (Robinson, 2005)**.** Example of states that can be tracked include:

- Number of people in a queue.
- Availability of a resource, for example, whether a machine is in operation or has failed.

- The status of a patient, for example, ‘waiting’, ‘under treatment’, or ‘discharged’.

Queues occur when demand for resources exceeds availability. They are a common feature of many real-world systems and play a central role in DES. Analyzing queues can help identify bottlenecks in the system and to develop strategies to minimize or avoid them, such as increasing resource capacity or prioritizing certain entities (Bangsow, 2012). Examples of queues include:

- Customers waiting at a supermarket checkout.
- Patients waiting for a doctor.
- Products waiting to be processed at processing station.

Event calendar is a crucial element of DES. It organizes and store all the future events and their scheduled times, controlling the timing of the simulation by ensuring that events are processed in the correct order. The event calendar makes it possible to efficiently control the simulation process and ensure that all events processed correctly (Kelton et al., 2004). Functions of the event calendar include:

- Contains all scheduled events, such as arrivals, processing or completions.
- Determines the next event to be executed.
- Updates the system state based on executed events.

The main components of DES – entities, resources, events, attributes, states, queues, and event calendar – provides the basis for modelling and analyzing complex systems. Each of these components plays a specific role and helps to realistically map the behavior of a system. Understanding these components is crucial to successfully developing DES models and gaining valuable insights into system behavior.

2.3 Performance metrics, analysis methods and verification/validation theory

DES is used in many areas to analyze and optimize complex systems. The success of a simulation depends crucially on the definition of appropriate performance metrics, application of sound analytical methods and the verification and validation of the model. This section examines these key topics in detail.

2.3.1 Performance metrics

KPIs are essential for evaluating the efficiency and effectiveness of a simulated system. They help to identify weaknesses, highlight opportunities for improvement and make decisions based on quantitative data. The selection of the right KPIs depends on the use case and the goal of the simulation. Some of the most common KPIs are:

- **Average waiting time:** This metric indicates how long entities spend in queues they are processed. It is particularly important in call centers, hospitals or logistics systems (Robinson, 2005).
- **Resource utilization:** The utilization of a resource, such as a machine or an employee, is measured as a percentage of the time the resource is actively used. Optimal

resource utilization prevents overload and unnecessary idle time (Law & Kelton, 2000).

- **Throughput:** It indicates how many entities are successfully processed by the system per unit time. This metric is crucial for evaluating system efficiency in production and manufacturing lines.
- **Queue length:** The average or maximum length of a queue helps to identify bottlenecks and delay that can affect the performance of the system (Fishman, 2001).
- **System time:** This metric measures the total amount of time an entity spends in the system, including waiting and processing time. It provides a detailed view of the system efficiency.

2.3.2 Analysis Methods

Analyzing simulation results is a crucial part of the DES process. The chosen analysis methods must ensure that the results are reliable and meaningful. Here most important analysis methods are briefly described.

Descriptive statistics are used to analyze central tendencies (like mean values) and scatter (e.g. standard deviations) of the results. For example, the average waiting time in a queue can be calculated and compared with predefined values (Law & Kelton, 2000).

Scenario analysis examines how different input parameters affect system's performance. For example, it could analyze how increase or decrease in resources will affects a throughput (Robinson, 2005).

Sensitivity analysis evaluates how sensitive the system is to changes in input parameters. This method helps to identify critical factors which affect the system performance the most (Fishman, 2001).

Analysis of variance (ANOVA) is used to evaluate the effects of different factors on the results. This method is particularly useful when several factors are analyzed simultaneously.

Monte Carlo analysis uses random sampling to evaluate the probability distribution of possible outcomes. It is particularly useful to analyze a system with high uncertainties (Kelton et al., 2004).

2.3.3 Verification and validation Theory

Verification and validation (V&V) are essential to ensure that the simulation model has been implemented correctly and provides realistic results.

Verification checks that the model has been implemented as intended and is free from logical or programming errors. Typical steps include:

- **Debugging:** Identifies and fixes errors in the model.
- **Event logging:** Checks whether events are executed correctly or not.
- **Limit value feedback:** Aanalyzes extreme situations to test the robustness of the model (Sargent, 2013).

Validation assesses whether the model correctly depicts the real processes. There are various approaches to validation:

- **Data comparison:** The results of simulation are compared with historical data to check for consistency.
- **Technical review:** Experts from the relevant specialist area assess the plausibility of the model assumptions and results.
- **Comparison with other models:** The results of the model are compared with similar models to ensure consistency (Robinson, 2005).

Verification and validation should be iterative to ensure that the model is continuously improved.

Performance metrics, analysis methods and V&V techniques form the base for the successful application of DES. Performance metrics help to evaluate system efficiency, while analysis methods provide deeper insights into the results. V&V ensure that the model is both accurate and realistic. A careful application of these concepts is critical to gaining sound and reliable insights from a DES.

2.4 Mathematical fundamentals

The mathematical foundations of DES are based on queueing theory, stochastic processes and other fundamental concepts of probability theory. These concepts enable the modelling and analysis of complex systems which undergo discrete state changes. This section provides a detailed overview of mathematical models and their application in DES.

2.4.1 Basics of queuing theory

Queuing theory is the heart of DES, especially for modelling systems in which entities wait for resources. The most important models of queueing theory include the M/M/1 and the M/M/c and its extensions (Fishman, 2001).

The **M/M/1 Model** is the simplest queueing model and describes a system with the following features:

- **One queue:** All incoming entities are queued in a single queue.
- **One server:** Only one resource is available for service.
- **Poisson distributed arrivals:** The arrival times of the entities follow a Poisson process with an arrival rate λ (Fishman, 2001).
- **Exponentially distributed service times:** The service time of each server is exponentially distributed with a service rate μ (Law & Kelton, 2000).

Important formulas in the **M/M/1 model** are:

- Average number of entities in the system (L):

$$L = \frac{\lambda}{\mu - \lambda}, \text{ if } \lambda < \mu \tag{1}$$

This represents the average number of entities in the queue or being served. A higher arrival rate λ relative to service rate μ leads to more entities in the system (Fishman, 2001).

- Average waiting time in the system (W):

$$W = \frac{1}{\mu - \lambda} \tag{2}$$

The average time an entity spends in the system, including both waiting and service time (Kelton et al., 2004).

- Server utilization (ρ):

$$\rho = \frac{\lambda}{\mu} \tag{3}$$

The utilization factor shows how busy the server is. Values of $\rho > 0.8$ indicate a high load, which may cause longer waiting times (Law & Kelton, 2000).

The **M/M/c-Model** extends the M/M/1-model by introducing multiple servers (c). This model is used in scenarios where multiple resources work in parallel, such as in call centers or hospitals (Fishman, 2001).

Formulas in the **M/M/c-model** are:

- Probability that an entity must wait (P_w): The calculation of P_w is based on the Erlang-C formula:

$$P_W = \frac{\frac{(\frac{\lambda}{\mu})^c}{c!}}{\sum_{k=0}^{c-1} \frac{(\frac{\lambda}{\mu})^k}{k!} + \frac{(\frac{\lambda}{\mu})^c}{c!(1-\rho)}} \tag{4}$$

Here, $\rho = \frac{\lambda}{c\mu}$ is the utilization per server (Fishman, 2001).

- Average waiting time in the queue (W_q):

$$W_q = \frac{P_W}{c\mu - \lambda} \tag{5}$$

This formula measures the average time an entity spends waiting in the queue before service (Fishman, 2001).

Advanced queueing models offer greater flexibility in modeling real-world systems, where the assumptions of the simpler models, such as M/M/1 and M/M/c, may not hold true. These models allow for more complex distributions of both arrival and service times, enabling them to better represent scenarios where exponential distributions do not apply.

- **M/G/1-Model:** This model allows for more general distributions of service times (G for "General"). It is used when the service time does not follow an exponential distribution (Law & Kelton, 2000).
- **G/G/1-Model:** Here, both the arrival and service times are arbitrarily distributed. This model is more flexible but analytically more difficult to solve (Kelton et al., 2004).

2.4.2 Stochastic Processes

Stochastic processes are a central foundation of DES as they model the randomness in arrival and service processes (Robinson, 2005).

A **Poisson process** describes the number of events occurring within a fixed time interval, with the following probability distribution:

$$P(N(t) = k) = \frac{(\lambda t)^k e^{-\lambda t}}{k!}, \text{ k= 0, 1, 2,…} \quad (6)$$

Here, λ is the average rate of events per time unit and t is the considered time period (Fishman, 2001).

The Poisson process is often used in discrete-event simulations to model the arrival of entities in systems. A closely related concept is **Markov processes**, which are stochastic processes where the next state depends only on the current state (Markov property). In DES, Markov processes are used to model state transitions in queueing systems where the future behavior is determined solely by the present state, and not by the sequence of events that preceded it (Law & Kelton, 2000).

2.4.3 Monte Carlo Methods

Monte Carlo methods play an essential role in DES, especially for analyzing uncertainties. These methods use random sampling to estimate probabilities and distributions of system outcomes (Kelton et al., 2004). Examples of its applications in DES are simulation of arrival processes where random samples generate arrival times based on Poisson distribution (Robinson, 2005), and assessment of uncertainties by analyzing the impact of variations in arrival or service times on system performance (Fishman, 2001).

2.4.4 Statistical Evaluation

Statistical analysis is crucial to assess the results of a simulation. Important metrics include:

- **Confidence intervals:** They indicate the range in which the true mean of the simulation results lies with a certain probability:

  $$\bar{X} \pm z \cdot \frac{s}{\sqrt{n}} \quad (7)$$

  Here, $\bar{X}$ is the mean, s is the standard deviation, n is the sample size, and z is the critical value (Robinson, 2005).
- **Hypothesis testing:** Hypothesis tests evaluate whether the differences between var ious scenarios are significant (Kelton et al., 2004).

The mathematical foundations of DES, including queueing theory, stochastic processes, and Monte Carlo methods, allow for precise modelling and analysis of complex systems. These concepts are indispensable for understanding the dynamics of discrete events and making informed decisions based on simulation results.

2.5 Algorithms for DES

In DES, algorithms play a central role in controlling the timing of events, managing resources efficiently and modelling stochastic processes realistically. This section highlights the central algorithms that enable DES and shows their possible applications in different contexts.

2.5.1 Event-Driven Algorithm

The event-driven algorithm is the fundamental mechanism that organizes the sequence of events in a DES. It enables events to be processed in order in which they occur and ensures that model remains dynamic and resource efficient (Fishman, 2001). Figure 2 shows basic pseudocode for event-driven algorithm.

Event-Driven algorithm

```
Initialize event list (sorted by event time)
Initialize system state variables

While event list is not empty:
        event = Remove the event with the smallest time from the event list
         Process the event (update system state)
         Schedule new events (if any) based on the state change
End While
```

Figure 2. Event-driven algorithm

The basic concept of the event-driven algorithm involves the following steps:

- **Initialization**: First, the event list is initialized with the scheduled events, which are sorted by event time. Additionally, system state variables are set up to capture the current state of the system.
- **Event Selection:** The algorithm selects the next event based on the earliest scheduled time from the event list. The system then processes this event.
- **Processing and State Update:** Once an event is processed, the system's state is updated to reflect the changes caused by that event.
- **Event Scheduling**: If the current event generates new events, they are scheduled and inserted into the event list based on their respective times.
- **Repetition:** This cycle repeats until a termination condition is met, such as when the event list is empty or the maximum simulation time is reached.

The advantages of the event-driven algorithm are significant. It allows for the efficient use of resources, as the simulation time only advances when events occur, and it is ideal for systems with clearly defined and sporadic events, such as production lines or queueing systems.

For example, in a call center simulation, incoming calls can trigger events when they arrive. When a call ends, a new event may be generated (such as the availability of an employee), which allows the model to efficiently simulate the system’s dynamics (Robinson, 2005).

2.5.2 Time-driven algorithm

In contrast to the event-driven approach, the time-driven algorithm advances the simulation in fixed time steps (Δt), which is useful when continuous changes in the system need to be tracked. This algorithm is ideal for systems where state changes occur continuously, and the model needs to account for changes that happen at regular intervals (Law & Kelton, 2000). Figure 3 gives an example of a time-driven algorithm.

Time-Driven algorithm

```
# Initialize system state variables and simulation parameters
current_time = 0
time_step = 1
end_time = 100
system_state = initialize_system_state()

While current_time <= end_time:
# Update system state for the current time step
update_system_state(system_state, current_time)
        # Process events at the current time step
process_events(current_time)
        # Advance to the next time step
        current_time += time_step
End While
```

Figure 3. Time-driven algorithm

The time-driven algorithm works as follows:

- **Initialization:** The simulation begins with an initial system state and predefined parameters, such as the current time and the time step (Δt). A maximum simulation time is also set.
- **State Update:** For each time step, the system state is updated based on the elapsed time.
- **Event Processing:** Any events that occur during the current time step are processed, and their effects on the system are accounted for.
- **Time Advancement:** After processing the events, the simulation time is advanced by Δt, and the cycle repeats until the maximum time is reached.

The advantages of the time-driven algorithm lie in its ability to model systems with continuous changes, such as ecological systems or physical processes like fluid flow. However, it comes with a disadvantage in terms of computational cost, as it may perform unnecessary calculations when no events occur in a given time step.

A practical example of the time-driven algorithm could be the simulation of a dam's water level, where the water level is continuously monitored, and updates are made at fixed intervals (Kelton et al., 2004).

2.5.3 Event calendar management

Event calendar management is a key aspect of the event-driven approach, using a priority-based data structure to maintain the correct temporal order of events. This ensures that the simulation processes events in the order in which they occur, even when events are added or modified during the simulation (Fishman, 2001). For instance, Figure 4 gives an example of an event calendar management.

Event calendar management

```
# Initialize event calendar (priority queue)
event_calendar = PriorityQueue()
# Schedule initial event
schedule_event(initial_event, initial_event_time)

While event_calendar is not empty:
    # Get the next event to process
    event_time, event = event_calendar.pop()
    # Update current time
    current_time = event_time
    # Process the event
    process_event(event, event_time)
    # Schedule new events
    schedule_new_events(event_time)
End While
```

Figure 4. Event calendar management

In the event calendar management approach:

- **Event Scheduling:** Events are scheduled based on their time of occurrence and added to the event calendar, typically using a priority queue.
- **Event Processing:** The algorithm retrieves the event with the earliest scheduled time from the calendar and processes it. As events are processed, new events may be generated and added back into the calendar.
- **Efficient Management:** By using a priority queue, the calendar can efficiently insert and remove events, ensuring that they are processed in the correct order.

This approach allows for a clear organization of events, making it easy to manage and process events in a timely manner. It ensures that events are handled in the proper sequence, leading to reliable and accurate simulation outcomes.

In the context of a network traffic simulation, event calendar management can be used to model the arrival and processing of data packets in a network. Each packet arrival event is scheduled based on the time it is expected to arrive at the network node. Once the event is processed, such as when the packet is transmitted or queued, new events may be generated, such as the arrival of additional packets or a change in network conditions (e.g., congestion or link failure). The event calendar ensures that these events are processed in the correct order, maintaining the simulation's integrity and accuracy.

2.5.4 Resource allocation algorithms

In many DES scenarios, resource allocation algorithms are essential for managing limited resources and ensuring that entities are processed efficiently. These algorithms control the allocation of resources based on certain criteria, such as priority or availability, and help manage queues effectively (Robinson, 2005). Figure 5 illustrate a basic example of this algorithm.

Resource allocation algorithm

```
Initialize machine states (busy or idle)
Initialize job queue (FIFO)

While simulation runs:
    If a job arrives:
        Add job to queue
    If machine is idle and job queue is not empty:
        Process the first job in the queue
        Update machine state to busy
        After processing, update machine state to idle
    End If
End While
```

Figure 5. Resource allocation algorithm

For example, in a simple resource allocation algorithm:

- **Initialization:** Resources (such as machines) are initialized, and the job queue is set up (often using a First-In-First-Out (FIFO) system).
- **Job Arrival:** When a job arrives, it is added to the job queue.
- **Resource Availability:** If a resource is idle and there are jobs in the queue, the first job in the queue is selected for processing. Once the job is completed, the resource is marked as idle again.
- **Repetition:** This cycle continues as long as jobs are arriving and resources are available.

Different variations of resource allocation algorithms exist, including:

- **FIFO:** Jobs are processed in the order they arrive.
- **Shortest Processing Time (SPT):** Jobs with shorter processing times are prioritized.
- **Priority-based Allocation:** Resources are allocated based on the urgency or importance of the job, such as in hospital emergency rooms.

These algorithms help simulate the behavior of systems where resources are limited and need to be allocated efficiently based on various criteria

DES algorithms form the backbone of complex system modeling and analysis. The event-driven algorithm enables efficient processing of discrete state changes, whereas the time-driven algorithm offers flexibility for continuous systems. Additionally, event calendar management ensures the correct sequencing of events, while resource allocation algorithms help manage limited resources efficiently.

The selection of the appropriate algorithm depends on the specific system being modeled and the nature of the events or processes within that system. Whether simulating a manufacturing process, managing resources in a call center, or modeling continuous systems like ecological processes, these algorithms provide the necessary tools to create accurate and efficient simulations.

2.6 Advantages and Disadvantages of DES

DES offers numerous benefits, making it a preferred approach for modeling complex systems across various domains. The key advantages include:

- **Event-Driven Approach:** DES processes state changes only when specific events occur. This event-driven nature allows the simulation to focus computational resources solely on moments of change, making it highly efficient in terms of resource usage. Rather than simulating every moment in continuous time, DES advances the simulation only when an event is scheduled to occur, reducing the need for unnecessary calculations during periods of system inactivity. This approach is particularly useful in environments where changes happen occasionally, such as in manufacturing processes or call centers (Law & Kelton, 2000).
- **Granularity:** DES provides a high level of granularity in capturing system behavior. It enables detailed tracking of individual components, such as resource utilization, queue dynamics, or the state of specific entities within the system. This level of detail allows for a more accurate representation of complex systems, making it possible to analyze and optimize specific parts of a system (e.g., wait times, bottlenecks, or resource allocation). The granularity offered by DES is valuable for understanding how individual actions or events impact the overall system performance.
- **Flexibility:** DES is adaptable to a wide range of applications. From structured systems, such as assembly lines with predictable processes, to more complex and stochastic systems, such as those found in healthcare, logistics, and manufacturing operations, DES can model diverse environments effectively. It is well-suited for systems where randomness plays a critical role in behavior, such as service times, arrival rates, and other stochastic variables. The versatility of DES makes it a powerful tool for modeling both well-defined and dynamic systems, offering flexibility in its use across industries (Lang et al., 2021).

While DES offers numerous advantages, it also comes with several limitations that can hinder its applicability in certain scenarios:

- **Scalability:** One of the primary challenges of DES is scalability. As the complexity of the system increases—whether in terms of the number of entities, events, or the level of detail required—the computational resources needed for running simulations also increase. Large-scale models with many interrelated components can become computationally expensive, particularly if they involve complex event interactions or require frequent updates. In such cases, running simulations may require significant processing power, which could lead to longer simulation times and increased costs.

Therefore, scalability remains an important consideration when designing a DES model for large and intricate systems.

- **Integration Challenges:** Another limitation of DES is its potential difficulty in integrating with other modeling methodologies, such as system dynamics or agent-based modeling. While DES is highly effective for modeling discrete, event-driven processes, it may not capture continuous feedback loops or the interactions between multiple agents as effectively as other approaches. For complex, multi-scale problems that involve continuous processes, system-level feedback, and agent-based behavior, combining DES with other modeling techniques can be a complex task. Integration between these methods requires careful design to ensure compatibility and meaningful results, but it is often necessary to obtain a comprehensive understanding of the system's behavior. The challenge lies in creating hybrid models that effectively merge the strengths of different approaches while minimizing potential conflicts in assumptions or methodologies.

DES provides significant advantages, such as efficient computational use through its event-driven approach, detailed granularity for capturing system behaviors, and the flexibility to model a wide range of systems. However, its limitations, particularly in scalability and integration, need to be addressed for effective application in large-scale or complex systems. Despite these challenges, DES remains a powerful tool for simulating and analyzing systems where discrete events drive state changes and can provide valuable insights into system performance and optimization.

2.7 Applications of DES

DES has found widespread application across various industries, demonstrating its adaptability and effectiveness in solving complex system challenges. Some key application areas of DES are discussed in this section.

In manufacturing and logistics, DES plays a critical role in optimizing workflows, scheduling tasks, and reducing downtime. Specific applications include assembly line optimization, where DES helps identify bottlenecks in production lines, evaluates different configurations, and suggests improvements to increase throughput. By simulating various scenarios, manufacturers can determine the most efficient process flow and resource allocation. In addition, DES supports inventory management by simulating supply chains and evaluating the impact of lead times, helping businesses optimize stock levels, reduce excess inventory, and ensure that products are available when needed (Bangsow, 2012). These applications demonstrate the value of DES in enhancing operational efficiency and supporting strategic decision-making in manufacturing and logistics sectors.

Within the healthcare sector, DES is applied to improve patient flows, staff scheduling, and resource allocation. For example, in emergency department operations, DES models patient arrivals, treatment times, and resource utilization. By simulating various scenarios, healthcare providers can reduce wait times and optimize staff allocation, ensuring timely patient care (Doudareva & Carter, 2022). Similarly, in surgical suite utilization, DES helps optimize the

scheduling of operating rooms and other surgical resources to maximize patient throughput while minimizing delays. By simulating surgery schedules, healthcare administrators can enhance the efficiency of surgical teams and reduce patient wait times. These examples highlight how DES contributes to more efficient and effective healthcare management.

DES is also employed in military applications to model complex systems, including command structures, logistics, and mission execution. A notable example is in system-of-systems design, where DES is used to model the interactions between various military systems to improve mission outcomes. For instance, Bagdatli and Mavris (2012) describe the application of DES in the Suppression of Enemy Air Defenses (SEAD) mission, where it was used to simulate and optimize the coordination of different defense systems, ultimately enhancing mission efficiency and success. This demonstrates how DES plays a vital role in improving the effectiveness and coordination of military operations.

Recent advancements have seen the integration of DES with technologies like digital twins, where real-time data is incorporated into simulations to support dynamic decision-making. This integration is particularly valuable in industries such as manufacturing and logistics, where real-time data can enhance system performance and assist managers in making informed decisions. For example, in manufacturing, DES-powered digital twins use real-time data from sensors and systems to optimize production processes and detect potential inefficiencies in real time. This dynamic feedback loop enables manufacturers to adjust to changing conditions, improve resource utilization, and minimize downtime (Karanjkar & Joshi, 2023; Agalianos et al., 2020).

These diverse applications demonstrate the wide-reaching potential of DES across various sectors, from improving operational efficiency in healthcare to enhancing decision-making in defense and manufacturing.

2.8 DES in Present-day Research and Development

Modern research in DES is focused on enhancing its capabilities through the integration of advanced technologies. Some of the key advancements in DES include:

- **Machine Learning:** By incorporating machine learning algorithms into DES, predictive analytics can be used to improve scenario evolution, enhance forecasting accuracy, and optimize decision-making processes (Atalan et al., 2022). This enables simulations to adapt more effectively to dynamic systems and provide better predictions of future states.
- **Hybrid Models:** Researchers are increasingly combining DES with other simulation approaches, such as continuous or agent-based models, to tackle complex, multi-scale problems. These hybrid models enable the simulation of systems where both discrete events and continuous processes need to be represented simultaneously, offering a more comprehensive view of the system.
- **Open-Source Tools:** The rise of Python-based simulation tools like SimPy and Salabim has significantly increased the accessibility of DES. These open-source libraries offer flexible and powerful platforms for researchers and industry professionals to

implement and experiment with DES models, fostering broader adoption and innovation in the field.

The theoretical foundations of DES provide a solid base for understanding and applying the method in various contexts. From the mathematical principles and algorithms to the core components and application areas, DES serves as a powerful tool for modeling and optimizing complex systems. The integration of modern technologies, such as Python and machine learning, highlights that DES is an innovative and adaptable tool, continuously evolving to meet the demands of modern applications.

While this chapter has explored the methodological and technical aspects of DES, the next chapter will focus on analyzing the current state of the art. This analysis will evaluate the available Python libraries for implementing DES, helping to identify the most suitable tools for further exploration. The goal is to establish a foundation for selecting the appropriate Python libraries that will be used throughout the remainder of this work.

3. State of the Art

This chapter discusses the current state of art for open-source DES python libraries. It describes the methodology of the literature search as well as the overview of selected libraries including an initial comparison.

3.1 Methodology for the literature review

This section outlines a methodological approach used to do a detailed analysis of existing DES libraries in Python and documents the process to ensure reproducible and informed selection.

3.1.1 Research objective

The purpose of this review is to identify and compare several Python-based libraries for DES in order to assess their features, usability, and suitability for specific application domains, particularly in executing workflow tasks like AGV systems. Through a systematic analysis and testing of the selected libraries, the aim is to address gaps in existing research and provide a comprehensive overview, especially in light of the limited number of studies offering a direct comparison of open-source Python libraries for DES.

Additionally, much of the existing literature primarily focuses on comparisons between commercial DES tools and open-source alternatives. For instance, Lang et al. (2021) conducted a comparison of Arena, Plant Simulation, Salabim, JaamSim, and CloudSim by implementing a case study, highlighting differences in performance, flexibility, and ease of use. This review intends to build upon such efforts, providing valuable insights into the potential advantages and limitations of open-source tools in comparison to more established commercial tools.

3.1.2 Search Strategy

The literature overview implemented a systematic approach for assessing DES libraries by targeting tools which have been released or substantially revised from February 2023 to March 2024.

Initial search process: The Python Package Index (PyPI) platform was used as primary source for identifying relevant Python libraries, as it lists almost all available Python packages in a centralized manner. The following steps were performed:

- **Keywords and search terms:** Terms such as ‘DES’, ‘discrete-event simulation’ and ‘event-based simulation’ were used in PyPI database to find related libraries.
- **Filtering by topicality:** To ensure that only current and actively maintained libraries are considered, the function ‘Order by: Date last updated’ was used which arranges all the results based on the date and time it was last updated on PyPI database. As PyPI does not offer any specialized filter to limit search to specific time, further filtration was carried out manually to identify Python libraries released or updated between February 2023 and March 2024.
- **Results:** This initial search process resulted in 40 libraries that were potentially relevant for this study.

Documentation and publication review: In the next step, these 40 libraries were further filtered based on availability of documentation, publications or community engagement. The following platforms were used:

- **PyPI:** For searching for Python packages and their specific features, updates and documentation.
- **GitHub:** For source code repositories, community engagement and open-source library documentation.
- **Google scholar:** For academic articles and research papers.
- **Semantic scholar:** For the analysis and collection of scientific literature.
- **ResearchGate:** For scientific papers, discussions and networks
- **IEEE Xplore:** For technical publications and conference papers.

It was aimed to find additional information on the libraries, such as implementation examples, academic articles or discussions in the community.

After this review, 11 libraries remained that were sufficiently documented or had an active community. The remaining libraries and their versions are:

- Salabim 24.0.1
- py-des-lib 0.1.1
- SimPN 1.0.0
- Akatosh 2.3.3
- pDesy 0.5.1
- Ciw 3.1.0
- SimPy 4.1.1
- HADES-framework 1.0.3
- O2despy 0.0.7
- Simprocessed 0.2.0
- PyDSOL-core 4.0.0

Final selection and exclusion: The remaining libraries were carefully reviewed to assess their suitability to specific requirements of this work. The following inclusion and exclusion criteria were considered:

The **inclusion criteria** for selecting the Python-based libraries for this review are as follows:

- Libraries that support DES functionality and are actively developed.
- Libraries that provide sufficient documentation or examples to facilitate implementation.
- Support for complex workflows such as resource allocation and AGV modelling.
- Evidence of community engagement, such as active forums or GitHub activity or enough documentation.
- The guidelines for open-source licensing, supported with documentation.

The **exclusion criteria** for selecting the Python-based libraries for this review are as follows:

- Libraries that do not focus on DES, are infrequently updated, or fall outside the scope of this work.
- Insufficient documentation, examples or limited community support.
- Tools with proprietary or highly restrictive licenses.
- Libraries which use other libraries such as SimPy as base libraries.
- Focused on one particular application or lack of flexibility to support AGVs or human simulation.

Figure 6 represents in detail selection steps starting from initial search with more than 10,000 results to narrowing to 5 Python libraries.

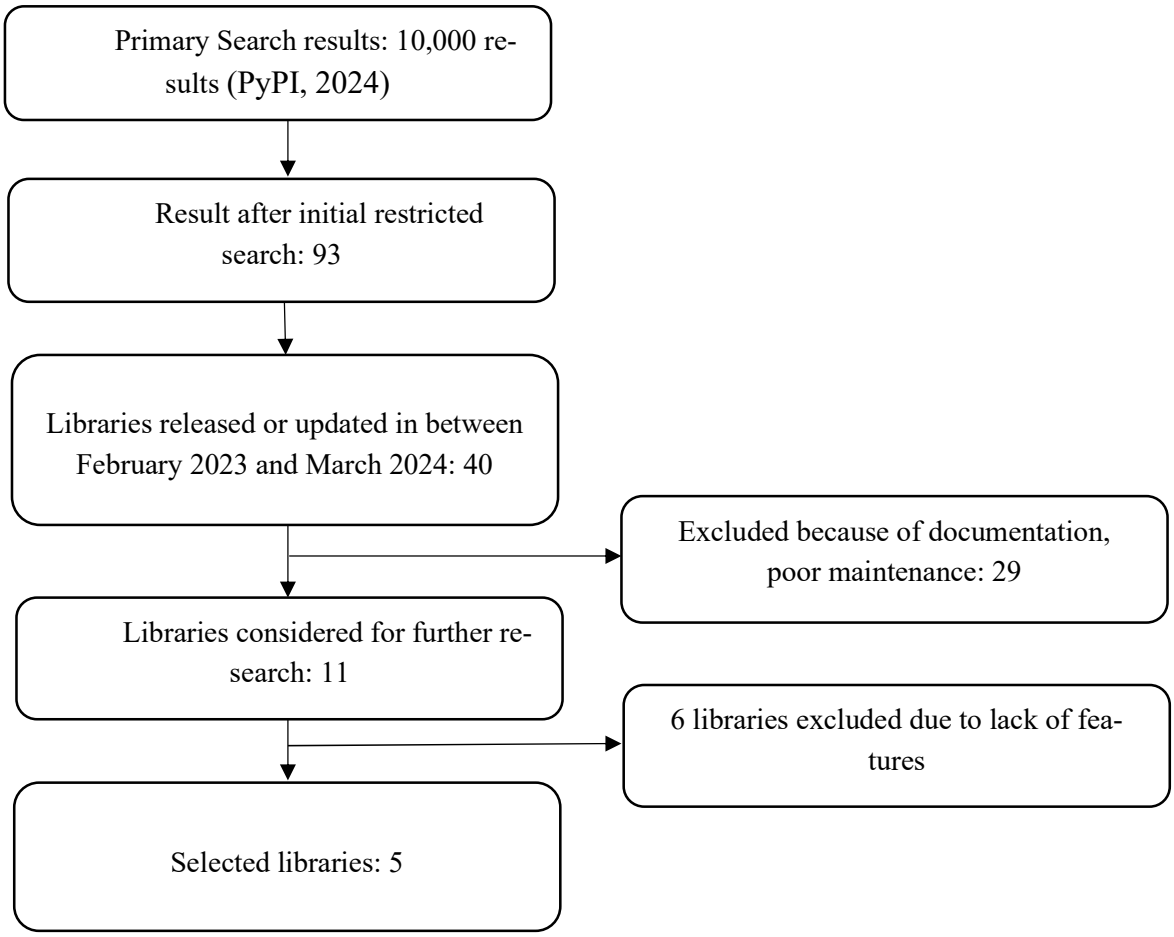


Figure 6. Selection process of the chosen libraries.

Several libraries were excluded from this review due to their limited functionality, focus on specific use cases, or lack of support for free-motion models, which are essential for simulating dynamic systems such as AGVs. The reasons for exclusion are as follows:

- **Ciw (3.1.0):** This library focuses on queueing systems and does not provide support for free-motion models, which are essential for simulating AGV systems. Additionally, its documentation is centered on queueing theory, limiting its applicability for more dynamic system modeling (Palmer et al., 2019).
- **SimPN (1.0.0):** Focus on Petri net modelling, which means that dynamic systems such as AGVs are not supported. Not suitable for free object movement (SimPN 2024).
- **pDesy (0.5.1):** While this library excels in workflow modeling, it does not support free-motion models, which are crucial for simulating the movement of AGVs. Its narrow focus limits its applicability in broader system simulation (Mitsuyuki and Okubo, 2024).
- **py-des-lib (0.1.1):** Built upon SimPy, this library does not offer any new, independent functionality beyond what SimPy already provides. Therefore, it does not add value for this work, which aims to explore more versatile libraries for AGV simulation (Py-DES, 2024).
- **O2despy (0.0.7):** In the very early stages of development, this library has minimal documentation and lacks sufficient support for complex workflows, limiting its utility in robust DES applications.
- **Simprocessed (0.2.0):** This library is poorly documented and does not support AGV modeling or other dynamic system simulations. Its lack of adequate resources and specialized features led to its exclusion from the review.

After careful evaluation, five libraries have been selected for this study: SimPy, Salabim, PyDSOL-core, Akatosh, and HADES-framework. These libraries were chosen for their unique features, robust capabilities, and well-established documentation. SimPy is widely recognized for its process-oriented discrete-event simulation capabilities, making it an excellent choice for modeling complex systems, including AGVs and free-moving objects, with its efficient process handling and queue management features (Matloff, 2008). Salabim, on the other hand, provides advanced features such as statistical reporting and 2D/3D animation capabilities, making it especially useful for visualizing complex systems (Ruud van der Ham, 2024).

The HADES-framework stands out with its hybrid capabilities, integrating both discrete and continuous event modeling, which is essential for systems that require mixed modeling approaches (Bagdatli & Mavris, 2012). Although its documentation is less comprehensive, its flexibility makes it well-suited for AGV modeling. Akatosh, with its object-oriented architecture, allows for custom event definitions and interactions, making it adaptable for AGV use cases, though it requires user-defined implementations for certain scenarios (Akatosh, 2024). Finally, PyDSOL-core offers advanced features such as event scheduling and resource management, which are essential for simulating large-scale and complex systems. However, it requires more customization to fully leverage its potential (PyDSOL-core documentation, 2024).

The following section provides an in-depth overview of the selected libraries, drawing insights from the literature review. By analyzing the key features, strengths, and potential ap-

plications of each library, this section aims to present a clear understanding of how these libraries cater to the specific needs of DES for AGV systems and similar complex workflows. The review highlights the practical capabilities of each library, along with any limitations or special features that differentiate them within the context of simulation modeling. This comprehensive analysis is essential for understanding their suitability in the current research and real-world applications.

3.2 Overview of selected Python libraries for DES

In the following, a detailed overview of the selected libraries is presented with a focus on their specific strengths and applicability to DES.

3.2.1 Detailed overview of SimPy

SimPy is a process-based DES framework implemented in Python. It facilitates modelling of systems which evolve over time based on discrete events. SimPy is widely recognized for its simplicity, flexibility and powerful modelling capabilities, making it one of the most popular tools for simulation in both academia and industry. It is based on Python and ensures ease of use, particularly for users familiar with the language, and allows integration with Python's extensive ecosystem of libraries for tasks like data analysis and visualization (Matloff, 2008; Oujezsky & Horvath, 2016). SimPy is suitable for a wide range of applications, including manufacturing, healthcare, logistics and network simulation. For example, it has been successfully used to simulate complex queueing systems, job scheduling in AGVs and dynamic resource allocation scenarios (Marszałek et al., 2024; Oujezsky & Horvath, 2016; Matloff, 2008).

Key Features:

SimPy provides a process-oriented modelling paradigm, where system components are represented as processes. These processes use Python generators, which enable the simulation of asynchronous events. This approach supports straightforward coding and debugging, as processes yield control and resume precisely where they left off. SimPy's key components include:

- **Environment:** Acts as the central simulation clock and scheduler.
- Process: They define the behavior of simulation (SimPy documentation, 2024).
- **Resources:** Represent the modelling of shared system components, such as servers or tools and handle process synchronization.
- **Events:** Manage state changes triggered by specific actions or conditions (Matloff, 2008).

Strengths:

- **Ease of use:** SimPy's syntax is intuitive and its process-based approach is user-friendly for those familiar with Python.
- **Flexibility:** Users can implement a wide variety of simulation models, from simple queueing systems to complex networks.
- **Integration with Python:** Allows seamless use of data science libraries like NumPy, Pandas and Matplotlib for simulation analysis and visualization.

- **Documentation and community support:** Comprehensive resources and an active community makes it accessible for beginners and experts alike (SimPy documentation, 2024; Matloff, 2008)**.**

Limitations:

While SimPy is a powerful tool it does have some limitations. SimPy does not offer parallel execution of simulations, which can impact performance for large-scale or computationally intensive models (SimPy documentation, 2024). Additionally, its focus on simplicity means it may lack specialized features available in domain-specific simulation tools (Oujezsky & Horvath, 2016).

SimPy's versatility, ease of use and integration capabilities makes it a strong candidate for DES in diverse fields. Its design philosophy prioritizes simplicity and flexibility, making it particularly suitable for educational purposes, exploratory research and real-world problem-solving scenarios. These strengths justify its selection for the present study on AGVs and free-moving object simulations.

3.2.2 Detailed overview of Salabim

Salabim is an open-source Python library designed for DES, offering a comprehensive suite of tools for modelling, animation and statistical analysis. Developed by Ruud van der Ham, it is built upon the principles of process-oriented modelling as established in earlier simulation tools like Simula and SimPy (Ruud van der Ham, 2024). Salabim has been successfully applied across various domains. A notable example includes its use in the ceramic tile industry to improve warehouse logistics through DES. By visualizing and analyzing different storage policies, the library demonstrated its ability to optimize complex workflows and enhance operational efficiency (Taccini et al., 2023). It has also been employed in logistics and production environments to simulate and optimize material flows (Ruud van der Ham, 2024).

Key Features:

- **Process interaction and modelling:** Salabim supports process interaction methods that are intuitive and allow for detailed representation of system dynamics. Along with traditional yield-based approaches, Salabim introduced a "yieldless" mode, simplifying process descriptions and reducing coding errors (Ruud van der Ham, 2024).
- **Animation capabilities:** A standout feature of Salabim is its real-time 2D and 3D animation capabilities. These animations enable users to visualize system behavior dynamically, making it particularly useful for applications like AGVs and free-moving object simulation. Additionally, it supports video production for documentation and presentations (Ruud van der Ham, 2024).
- **Comprehensive resources and queues:** Salabim includes built-in constructs for handling resources, queues and stores. This makes it well-suited for modelling complex systems involving shared resources or material handling, such as manufacturing and logistics systems (Taccini et al., 2023).

- **Statistical sampling and monitoring:** The library provides tools for statistical sampling, monitoring and data collection during simulation runs. Integration with Python's Pandas and Matplotlib libraries allows for detailed analysis and visualization of simulation results (Ruud van der Ham, 2024).

Advantages:

- **Ease of use:** Salabim's process descriptions are simpler than traditional yield-based models, making it accessible for both new and experienced users.
- **Integrated animation:** The ability to animate simulations in real-time provides a significant advantage for analyzing dynamic systems and presenting results effectively.
- **Flexibility:** Its object-oriented architecture and integration with Python libraries like NumPy and Matplotlib ensure adaptability for a wide range of simulation scenarios.

Limitations:

While Salabim is an effective tool, its dependency on Python environments and limited support for parallel processing might pose challenges for large-scale simulation requiring high computational efficiency (Ruud van der Ham, 2024).

Salabim's combination of robust modelling tools, advanced animation features and ease of use makes it an excellent choice for discrete event simulation, particularly in applications involving dynamic systems like AGVs and logistics operations. Its strengths in animation and resource handling further justify its selection for studies involving free-moving objects.

3.2.3 Overview of PyDSOL-core

PyDSOL-core is an open-source Python library designed for DES, serving as the core component of the Python Distributed Simulation Object Library (PyDSOL). It is a Python adaptation of the java-based DSOL framework, which was introduced at the Winter Simulation Conference in 2002 (Zeigler et al., 2000). The framework is based on modelling and simulation theory, focusing on a structures approach to simulation studies (PyPI, 2024).

Key Features:

- **Event scheduling formalism:** Currently, PyDSOL-core implements the event scheduling formalism, allowing users to define and manage discrete events within a simulation environment (PyPI, 2024)
- **Modular design:** The framework is structured around key components such as the DSOLModel class, which users can extend to create simulation models, various Simulator implementations like the DEVSSimulaor and the experiment class for defining simulation experiments (PyPI, 2024).
- **Extensibility:** While the current implementation focuses on event scheduling, the design allows for future extensions to support other formalisms, including process interaction, differential equations, flow modelling and various DEVS (Discrete Event System Specification) variants (PyPI, 2024).
- **Integration with PyDSOL-Model:** An additional layer, PyDSOL-Model, provides standard objects such as sources, servers, sinks, nodes, links, entities and vehicles,

facilitating the development of DES models (averbraeck/PyDSOL-core, 2024; PyDSOL-model documentation, 2023).

Advantages:

- **Object-Oriented approach:** PyDSOL-core's design complies to object-oriented principles, promoting modularity and reusability in simulation modelling (PyPI, 2024) .
- **Foundation for advanced modelling**: The framework's architecture provides a solid foundation for implementing advanced modelling techniques, including agent-based modelling and hierarchical DEVS, aligning with the capabilities of its Java predecessor (PyPI, 2024).
- **Open-Source licensing:** Released under the BSD 3-clause license, PyDSOL-core allows for incorporation into other products for educational, commercial, or other purposes and can be extended or adapted freely (PyPI, 2024).

Limitations:

PyDSOL-core currently supports only the event scheduling formalism, and other modeling approaches such as process interaction and differential equations are not yet implemented. This limitation could restrict its applicability for certain simulation scenarios (PyPI, 2024). The framework is still in active development, with plans to incorporate features from the original Java DSOL, including support for differential equations and various DEVS variants, in future releases. However, these features are not yet available (PyPI, 2024). Furthermore, PyDSOL-core lacks a large, active community, meaning users may encounter challenges in finding extensive resources, discussions, or shared models, especially when compared to more established simulation libraries like SimPy or Salabim.

PyDSOL-core offers a structured object-oriented platform for DES in Python, building upon the established DSOL framework. Its modular design and open-source licensing makes it a promising tool for simulation modelling. However, the current focus on event scheduling formalism and the ongoing development of additional features suggests that users who are seeking for more comprehensive modelling capabilities may need to await future updates or consider PyDSOL-core with other tools to meet their specific simulation requirements.

3.2.4 Overview of Akatosh

Akatosh is a lightweight, object-oriented DES library for Python. Unlike traditional process-oriented libraries such as SimPy, which requires generator functions for event interactions, Akatosh summarizes events as InstantEvent or ContinousEvent objects, each with defined states, priorities and life cycles. This design simplifies the modelling of complex systems by allowing events to be created independently or within other events (ulfaric/Akatosh, 2024).

Key Features:

- **Object-Oriented event modelling:** Events in Akatosh are summarized as objects, enabling a clear and structured approach to simulation modelling (ulfaric/Akatosh, 2024).

- **Asynchronous execution:** The library supports asynchronous execution, allowing events occurring at the same simulated time to execute concurrently, unless differentiated by priority (ulfaric/Akatosh, 2024).
- **Resource management:** Akatosh includes resource management features similar to those in SimPy, with additional utilities for data collection from remote sources and interaction with entities (ulfaric/Akatosh, 2024).
- **Entity representation:** The entity class represents abstract entities with life cycles, such as followers and provides utility functions for resource interaction (ulfaric/Akatosh, 2024).
- **Real-Time simulation support:** Akatosh supports real-time simulation with a default time step of 0.1 seconds, enabling synchronization with real-time events (ulfaric/Akatosh, 2024).

Advantages:

- **Comprehensive functionality:** Akatosh offers all the functionalities present in SimPy, including process interaction and resource management, with additional features like entity representation and real-time simulation support (Akatosh 2024).
- **Simplified event handling:** By summarizing events as objects, Akatosh reduces the complexity associated with generator functions and streamlining the modelling process (ulfaric/Akatosh, 2024).
- **Asynchronous and real-Time capabilities:** The library's support for asynchronous execution and real-time simulation makes it suitable for applications requiring parallel event handling and synchronization with real-world time (ulfaric/Akatosh, 2024).

Limitations:

As a relatively new library, Akatosh may have limited community support and resources compared to more established simulation libraries. Additionally, users familiar with DES libraries like SimPy might require some time to adapt to Akatosh's object-oriented approach and its unique features. This learning curve could be a challenge for those looking to quickly implement their simulations, though the potential benefits of its advanced capabilities might outweigh this initial hurdle.

Akatosh presents a modern, object-oriented approach to DES in Python, offering comprehensive functionalities that include and extend beyond those of SimPy. Its support for asynchronous execution and real-time simulation and makes it a versatile tool for modelling complex systems. However, potential users should consider the current development status and community support when choosing a simulation library for their projects.

3.2.5 Overview of HADES-framework

HADES-framework (Hierarchical Asynchronous Discrete-Event Simulation) is a Python library designed for building DES with a focus on asynchronous processes. Developed to handle both process-based and agent-based modelling, HADES provides flexible tools for simulating real-world systems, especially those requiring parallel execution, such as networked environments (ki-oss/HADES: HADES, 2024).

Key Features:

- **Asynchronous simulation:** HADES is optimized for simulations that involve asynchronous workflows within a time-step. It is particularly effective in handling inbound/outbound tasks like database queries or large language model interactions (ki-oss/HADES: HADES, 2024).
- **Event modelling:** HADES library uses Pydantic-style fixed events, which ensure type safety and consistency when handling simulation events. This approach also simplifies debugging and state management (ki-oss/HADES: HADES, 2024).
- **Visualization tools:** HADES supports real-time visualization of simulation through WebSockets, Matplotlib integration and Mermaid diagram outputs. These features enable better analysis and presentation of simulation results (ki-oss/HADES: HADES, 2024).
- **Agent-Based and Process-Based models:** It accommodates both agent -based simulations and traditional process-based models, making it versatile for diverse applications (ki-oss/HADES: HADES, 2024).
- **Extensible design:** HADES allows users to define custom process and event classes for highly specific simulation needs, allowing flexibility and adaptability in modelling.

Advantages:

- **Real-time execution:** Its asynchronous execution capabilities allow simulations to interact with real-time events, making it suitable for real-world integration scenarios.
- **Comprehensive visualization options:** The ability to generate WebSocket-based visualizations for Boids simulation and interactive outputs supports in-depth analysis and intuitive presentations.
- **Ease of customization:** HADES's modular design simplifies the process of extending the framework to accommodate unique requirements.

Limitations:

HADES, while effective for asynchronous DES, does not natively support continuous dynamics required for modeling systems such as free-moving objects or AGVs (ki-oss/HADES, 2024). Additionally, due to the framework's relatively recent development, it has limited community support, resulting in fewer tutorials, examples, and community-driven resources compared to more established libraries. This could make it challenging for new users to quickly adopt and effectively implement the library in their simulations.

HADES-framework is an appropriate choice for this work due to its extensibility and asynchronous capabilities. While it requires additional customization to handle continuous dynamics, it provides a well-established foundation for discrete-event modelling of AGVs and human interactions. The ability to integrate agent-based behaviors aligns with the need to simulate human dynamics alongside AGVs.

Comparison of selected libraries

The following Table 2 provides an initial overview of the key features of selected libraries, including their core concepts, documentation, visualization options and supported use-case.

Table 2. Initial comparison of selected libraries.

Library	Version	Core concept	Documentation	Layout Visualization	Support for AGVs	Community Support
Salabim	24.0.1	Advanced Animation & Analysis	Well documented	2D/3D	Well suited	Moderate
SimPy	4.1.1	Process oriented DES	Comprehensive and easily accessible	None	Well suited	Very active
PyDSOL-core	4.0.0	Event planning & Resource Management	Basic, fewer examples	None	Well suited	Low
Akatosh	2.3.3	Object oriented DES	Detailed, but technically sophisticated	None	Well suited	Low
HADES-framework	1.0.3	Hybrid Modelling	Limited documentation	None	Restricted	Low

While SimPy and Salabim stand out for their ease of use and visualization capabilities, PyDSOL-core and Akatosh stand out for their adaptability. The HADES-framework offers an interesting hybrid modelling option, but with limited documentation.

This chapter provides a detailed discussion on the methodology, including the inclusion and exclusion criteria used for selecting the libraries, followed by an in-depth overview of each library. The focus is set on the evaluation criteria, which will form the base for comparing the libraires through their implementation in the case study.

4. Evaluation criteria

This chapter defines the evaluation framework which was used for comparing the five selected Python libraries for DES: SimPy, Salabim, PyDSOL-core, Akatosh and HADES framework. The evaluation focuses on 16 key features which are critical for modelling complex systems involving dynamic resources such as AGVs and human interactions. These criteria ensure a rigorous analysis of each library's strengths, limitations and suitability for the specific simulation needs.

4.1 Features for comparison

The selection of features is based on an in-depth analysis of the requirements of modern DES applications and a comprehensive literature review. Key references include foundational studies such as Matloff (2008) and Bagdatli & Mavris (2012), which provided insights into the core functionalities essential for evaluating DES libraries. The feature selection was further guided by the requirements of the case study, emphasizing AGVs and human interactions.

Justification for feature selection:

- **Practical relevance:** The features are closely related to the requirements of DES scenarios, especially regarding AGVs and human interactions. These includes, among others, simulation of mobile resources, queuing behavior and event tracking.
- **Scientific basis:** The feature selection is supported by existing studies on DES applications, which highlight the importance of resource management, event-driven architectures and statistical validation.
- **Adaptation to the case study:** The features were adapted to meet the requirements of the case study and realistically represent real-world scenarios with dynamic workflows.

Classification of features:

The features were grouped into four categories based on their application area:

- **Core simulation functions:** Adaptation of arrival processes, resource management and event control.
- **Scenario management:** Scenario testing, real-time simulation and model validation.
- **Output and analysis:** Statistical analysis, visualization and data export formats.
- **Utilities:** Error handling, debugging and integration options.

Each feature is critical for testing the libraries' capabilities and has been grouped according to its relevance to DES and real-world applications. Table 3 lists and describes these features, their significance and their assessment criteria.

Table 3. Features for comparison of the selected libraries.

Feature	What to measure
Customization of arrival processes	Measures the flexibility of libraries in defining entity arrivals using various parameters like interarrival times, rates and batch arrivals.
Resource capabilities	Evaluates the ability to define, manage and allocate resources, including priorities, capacities and pre-emptive functionalities.
Simulation of moving resources	Focuses on the ability to model and simulate free-moving resources, such as AGVS, across dynamic locations.
Simulation of queue behavior	Examines mechanisms for managing queue dynamics, including FIFO, Last-In-First-Out (LIFO) and priority-based strategies.
Event handling logic	Assesses the flexibility in defining and managing events like breakdowns, repairs and custom behaviors.
Statistical analysis	Analyzes built-in tools for collecting and processing performance metrics, such as resource utilization and waiting times.
Scenario testing	Evaluates the ease of configuring diverse test cases to analyze library performance under varying conditions.
Output visualization	Reviews visualization capabilities, including model animations, plots, and real-time tracking.
Model validation	Assesses the ability to valiadate models using theoretical benchmarks or real-world data.
Real-time simulation	Examines the ability to pause, modify and resume simulations interactively.
Testing framework	Evaluates the support for unit testing and automated evaluation of simulation components.
Queue dynamics policies	Focuses on the flexibility of switching between different queue management policies.
Error handling and debugging	Analyzes the robustness of error-handling tools and ease of debugging.

Integration capabilities	Examines compatibility with Python libraries like NumPy, Pandas, and external systems.
Event traceability	Measures the ability to log and track events during simulation for detailed analysis.
Data output formats	Reviews supported formats for simulation outputs, such as CSV, JSON and databases.

4.2 Methodology for evaluation

The evaluation methodology involves the practical implementation of each feature within simulated scenarios. These scenarios are designed to reflect dynamic workflows, including AGV movements and human interactions, to ensure realistic and relevant testing conditions. System configuration and development environment followed by steps for evaluation is described for efficient reproducibility.

4.2.1 System configuration

The implementation was conducted on a system equipped with an 11th Gen Intel Core i7-1165G7 processor (4 cores, 8 threads) and 16 GB of RAM, ensuring robust computational capabilities for simulation tasks. The operating system was Windows 11 Home (Version 23H2, OS Build 22631.4602), installed on February 24, 2023. The 64-bit architecture provided compatibility with advanced software tools, ensuring a stable and efficient development environment.

4.2.2 Development environment

The implementation was carried out in a controlled development environment to ensure consistency and reproducibility. Python3.10 was used as the programming language, while PyCharm served as integrated development environment. The configuration PYTHONUNBUFFERED=1 was used to enable real-time output. The libraries used were installed via pip, and additional modules such as pandas, NumPy and matplotlib were integrated to support advanced analysis and visualization.

4.2.3 Steps for evaluation

The steps of the evaluation focused on qualitative features and the user experience.

Implementation of features: Each of the 16 selected features (see section 4.1) was implemented in a controlled simulation environment. Each scenario was designed to simulate typical DES requirements without the need to record detailed performance metrics. The tested features include flexibility of arrival processes, resource management, behavioral models for queues, event control and statistical analysis and visualization options.

Ease of use: A subjective evaluation of the usability of each library was performed. The following aspects were considered:

- **Documentation:** How well is the library documentation understandable? Are there tutorials that makes the use of the library easier?
- **API design:** Is the API easy to use and well structured? Is integration with other libraries (e.g. Pandas, NumPy) possible?
- **Visualization options:** How easy is it to visualize the outputs of the simulation and are the visualization options sufficient?

Integration and extensibility: Each tool was tested for integration with other libraries and frameworks. The ability to combine DES models with common Python libraries (such as Pandas, NumPy, and Matplotlib) was evaluated. These libraries are important for data processing and advanced analysis and allow seamless integration into existing Python environments.

Documentation and learning resources: Each library was assessed based on the availability and quality of its documentation. The availability of tutorials and examples was evaluated to determine if the library provides sufficient resources, such as sample code and introductory documentation, to help new users quickly understand and use the library. Additionally, the developer community support was considered, including the existence of online forums, discussion groups, or other platforms that assist users in solving problems and providing ongoing support.

Flexibility and customizability: The customizability of the libraries with respect to specific simulations was examined. Were custom models or extensions easy to implement? The ability to customize and extend the libraries was tested based on the possibilities to implement specific simulation requirements, such as modelling AGVs or custom events.

4.2.4 Visualization of results

The focus of the visualization is to present the qualitative evaluations of the tested features:

- **Comparative table:** A comparison table was created summarizing the most important features of the five libraries, in particular the support for the 16 features that had been described. This table provides a quick overview of which library supports which feature and furthermore provides information on strengths and weaknesses of each library.
- **Visual representation**: For the visualization of simulation result, graphs were used, which represents data such as throughput, queue length and resource utilization.

This chapter outlined the evaluation framework based on the 16 key features for comparing the selected libraries. The next chapter will document the design of the case to test these features in real-world simulation scenarios, followed by the implementation of the case study.

5. Design and implementation of the case study

In the following chapter the design of the case study is presented, which serves as the foundation for evaluating the selected Python libraries for DES. The case study simulates a multistage manufacturing process involving AGVs, manual inspection and dynamic buffer management. The aim of this case study is to assess the libraries' performance in handling key features such as resource allocation, event handling and real-time simulation.

5.1 Case study design

The primary objectives of the case study are to comprehensively evaluate the capabilities of each selected library in modeling complex workflows, particularly focusing on the integration of AGVs and human interactions. The case study will also assess how each library handles dynamic scenarios, such as station failures and product routing, to ensure that the libraries can model real-world uncertainties and operational challenges. Furthermore, the libraries will be benchmarked based on predefined performance metrics, including throughput, wait time, and AGV utilization, to provide a quantitative assessment of their efficiency and effectiveness. This case study aims to provide insights into the practical strengths and limitations of each library in the context of simulating sophisticated systems and workflows.

5.1.1 Case study overview

In real production things such as dynamic event control, resource optimization and the ability to deal with errors are key challenges. Manufacturing systems must not only organize production efficiently, furthermore they must be able to react flexible to unforeseen events. For example, in the event of a machine breakdown or an unforeseen delay in logistics, solutions must be found quickly to not jeopardize the production flow. This ability to detect and rectify errors in real time is crucial to minimize downtime and keep operations running continuously.

Additionally, the effective utilization of resources - whether through automated transport systems such as AGVs or manual inspection stations - requires precise planning and management. A resource-optimized production system ensures that bottlenecks are quickly identified and capacities are adjusted accordingly. In this context, simulating such systems is a valuable way of testing different scenarios under controlled conditions and evaluating how different strategies can affect the overall performance of the system.

With this in mind, the manufacturing process presented in this case study is an ideal candidate to test the capabilities of the selected simulation libraries. By modelling production sequences that include both stationary and dynamic elements such as AGV transports and manual inspections, the libraries are tested for their flexibility and performance. In particular, the ability to integrate complex error events and optimize the production flow in real time represents a major challenge which has to be addressed with the help of the selected libraries.

The production process consists of three clearly defined phases – pre-processing, main processing and post-processing. These phases are closely interlinked and form a dynamic system that can react flexibly to changes. The following description shows the production process

and takes into account the essential dependencies and interactions between the individual components.

Pre-processing (Phase 1):

Pre-processing forms the base of the entire production process by preparing raw materials in such a way that they can be integrated into the further stages of production. The main task of pre-processing is to inspect raw materials and classify them into types like Product_A or Product_B. This enables targeted forwarding to the main process. Tasks of pre-processing phase are as follows.

Determines the product type and production quantity:

- The raw materials are analyzed and digitally recorded in order to define the product type and the quantity required for the further process.
- Defective or insufficient materials are removed from the production flow to ensure smooth further processing.

Preparation of materials for Station_1:

- Raw materials go through basic process steps such as cleaning, testing and formatting.
- The prepared materials are forwarded directly to Station_1, which ensures synchronization with the main processing.

After pre-processing, all ready-to-use materials are automatically transferred to Station_1, while unusable materials are ejected. The efficiency of this step is essential for the further flow of materials.

Main processing (Phase 2):

The main machining is the central section of the process and consists of processing at Station_1 and downstream manual inspection stations. In this phase, the raw materials are further processed into intermediate products and comprehensively tested.

Station_1:

- **Execution of the main processes:** Process_1 is a basic process and necessary for both products, Process_2 is exclusive to Product_A, and Process_3 is specialized for Product_B.
- **Handles station failures:** A failure of Station_1 leads to the automatic marking of the products in process as defective, which are then redirected to the Not Okay (NOK) buffer. The resumption of operations takes place after a repair, which has a direct impact on the buffers and the transport process.

Manual inspection stations:

- **Quality inspection and forwarding:** Products go through detailed manual quality control, where defective products are identified and sorted out.

- **Forwarding of tested products:** Products that pass the inspection go into the appropriate buffers: Buffer_A for Product_A and Buffer_B for Product_B. And defective products are redirected to the NOK buffer for later analysis or post-processing.

The quality and functionality of Station_1 and inspection stations have a direct impact on buffer capacity. A failure or delay in any of these components can affect the entire production flow.

Post-processing (Phase 3):

Post-processing is the ultimate step of the production process, where the final products are assembled and prepared for delivery.

- **Buffer management:** Buffer_A and Buffer_B serve as temporary storage for tested products that are ready for assembly. The NOK buffer collects defective products for further analysis or post-processing.
- **AGV transportation:** AGVs transport the products from the buffers to the assembly unit. Delays or overcrowding in the buffers can affect the route planning of the AGVs and requires a dynamic adjustment of the transport system.
- **Assembly station:** The Buffer_A and Buffer_B products are combined in the assembly unit to produce final products according to randomly generated order requirements. Before shipping, each final product is subjected to a final automatic inspection to ensure that it meets the requirements. Products that do not pass the final inspection are sorted out accordingly or reprocessed.

Process flow diagram:

The following diagram (Figure 7) visually represents the production system's workflow. It represents the transition of products through all three stages.

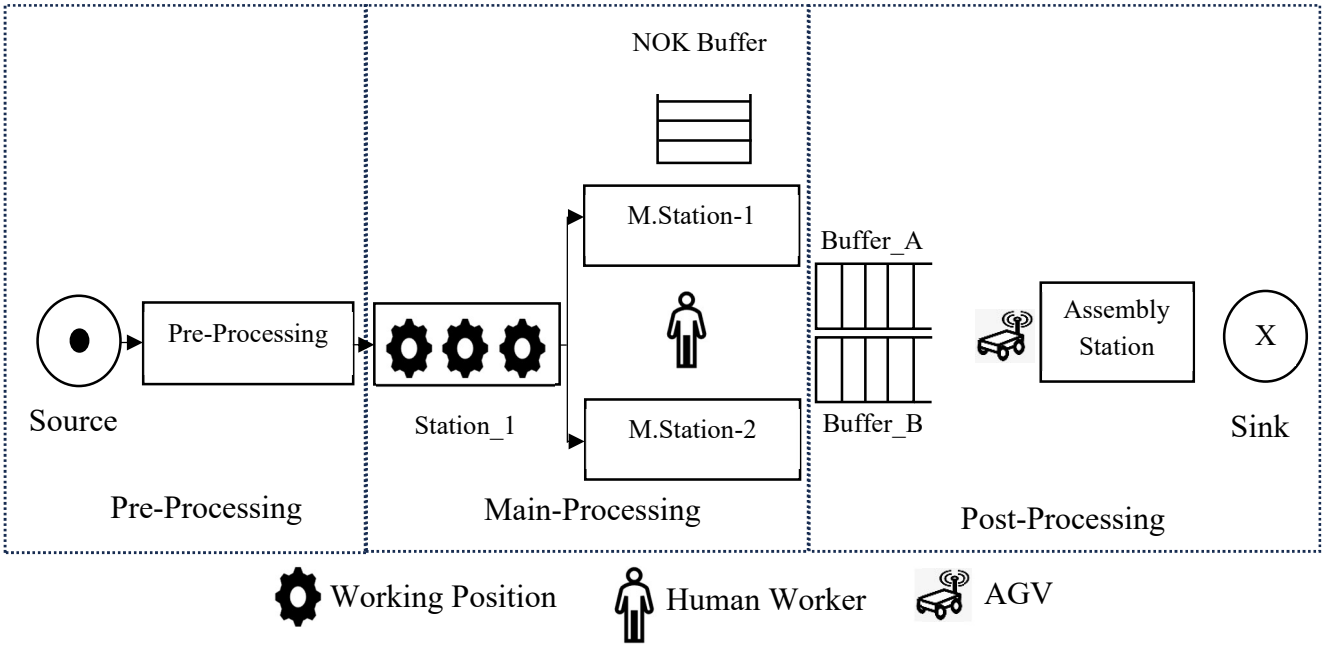


Figure 7. Flow Diagram of the case study process.

5.1.2 Theoretical calculations

This section provides a theoretical calculation for the case study, incorporating all relevant timings and distributions. Calculations are performed for 24-hour periods to evaluate system performance, including failure rates, AGV cycle times, inspection loads and assembly station utilization. These calculations help to estimate the operational efficiency and potential bottlenecks in the simulation.

Declared timing and assumptions:

The following constants and assumptions (see Table 4) are used for all calculations:

Table 4. Constants and assumptions for the case study.

Process	Parameter	Value	Justification
Pre-processing	Processing Time	10 min	Assumed based on typical industry times for initial material preparation tasks.
Main processing	Process_1 (Common)	8 min	Assumed from typical machining time for general processes in manufacturing.
	Process_2 (Product_A)	7 min	Based on typical customization tasks in discrete manufacturing
	Process_3 (Product_B)	7 min	Based on typical customization tasks in discrete manufacturing
	Mean time between failures (MTBF) (Station_1)	300 min	Typical MTBF for medium-complexity equipment
	Repair Time	30 min	Average repair duration as detailed in maintenance and reliability engineering studies
Manual inspection	Inspection Time	5 min	Standard manual inspection time for medium-sized products
AGV transportation	Distance	15 m	Assumed based on facility layout and AGV routing requirements.
	Speed	1.5 m/s	Average operating speed of AGVs from industrial benchmarks

	Pickup/Drop-off Time	20 sec	Estimated from AGV performance evaluations in typical production environments.
Assembly station	Processing Time	10 min	Based on typical assembly times for multi-component discrete products
Simulation period	Duration	24 hours	Standard simulation horizon for evaluating continuous production workflows

- **Pre-processing and main processing times:** These assumptions are based on industry benchmarks for material preparation and basic processing activities, which are typically in the range of 7 to 10 minutes for such operations. General machining and customization tasks are also consistent with typical process times in discrete manufacturing.
- **MTBF and repair time:** The MTBF value of 300 minutes is often observed for industrial equipment in typical manufacturing environments and the repair time is often around 30 minutes based on industry reports on machine downtime.
- **AGV parameters:** The assumed driving distance of 15 meters and the pick-up/drop-off time of 20 seconds are typical for industrial environments, where AGVs are used for material handling. The AGV speed of 1.5 m/s is a standard operating speed often specified in industrial AGV systems.
- **Inspection time and assembly time:** The manual inspection time of 5 minutes is considered typical for visual and manual inspections in manufacturing environments. The assembly time of 10 minutes is common in discrete manufacturing for assembly tasks.
- **Simulation period:** A 24-hour simulation period is standard to capture operational dynamics in continuous production systems and provide a complete representation of the manufacturing process over a full operating cycle.

Explanation of symbols:

This section provides an explanation of the key symbols used in the case study analysis. These symbols represent crucial parameters that impact the simulation and performance evaluation of the system. Understanding these variables is essential for interpreting the results and assessing the libraries' effectiveness. The symbols and their significance are outlined below.

- **λ (Lambda):** The rate of failure events per minute
- **μ (Mu):** The mean service rate or average number of products processed per hour.
- **T_{cycle}:** The AGV cycle time, calculated as the total time required for a single pickup and drop-off operation.
- **ρ (Rho):** The utilization of a station, indicating the fraction of time it is actively processing.
- **P (failure):** The probability of failure during processing.

Calculation:

This section calculated all the necessary data based on all the assumptions and provides theoretical results for the developed case. All equations used for calculation are based on Fishman (2001) and Law & Kelton (2000).

1. **Station failures:**

- MTBF: It represents the average operational time between consecutive failures:

$$\lambda = \frac{1}{MTBF} \tag{8}$$

- Given MTBF = 300 minutes:

$$\lambda = \frac{1}{300} = 0.00333\ failures\ per\ minute$$

- Failures in 24 Hours:

$$\text{Failures in 24 Hours} = \lambda \times \text{Total Time} \tag{9}$$

- Total time for 24 hours:

$$Total\ Time = 24 \times 60 = 1440\ minutes$$

$$Failures\ in\ 24\ Hours = 0.00333 \times 1440 = 4.8\ failures$$

- Total Downtime (Repair Time):

$$Total\ Downtime = Failures \times Repair\ Time$$

$$Total\ Downtime = 4.8 \times 30 = 144\ minutes$$

2. **AGV Operations:**

- Cycle Time (T_{cycle}):

$$T_{cycle} = Pickup\ Time + Drop - off\ Time \tag{10}$$

For Distance = 15 m and Speed = 1.5 m/s:

$$T_{cycle} = 20 + \frac{15}{1.5} + 20 = 50\ seconds$$

- Cycles in 24 Hours:

$$Total\ Time\ (in\ seconds) = 24 \times 60 \times 60 = 86400\ seconds$$

$$Total\ Cycles = \frac{Total\ Time}{T_{cycle}} \tag{11}$$

$$Total\ Cycles = \frac{86400}{50} = 1728\ cycles.$$

- Product can be transported by AGVs:

Assuming each cycle moves one product,

$$Products\ can\ be\ transported = 1728\ products$$

3. **Inspection load:**

- Failed products rate:

$$Failed\ products\ rate = Arrival\ rate \times P(failure\ during\ processing) \quad (12)$$

Given:

Arrival rate = 5 products/hour,
P(failure) = 0.181

$$Failed\ products\ rate = 5 \times 0.181 = 0.905\ products/hour$$

- Inspection load for failed products:

$$Inspection\ load\ in\ 24\ hour = Failed\ products\ rate \times Inspection\ time \times\ 24 \quad (13)$$

$$Inspection\ load = 0.905 \times 5 \times 24 = 108.6\ minutes$$

4. **Assembly station utilization:**

- Utilization (ρ):

$$\rho = \frac{\lambda}{\mu}, \quad (14)$$

Where,

λ = Input rate,
μ = Service rate

For:

Input rate (λ = 4 products/hour),
Service rate (μ = 6 products/hour)

$$\rho = \frac{4}{6} = 0.6666 = 66.7\ \%\ utilization$$

5. **Throughput:**

- Effective throughput:

$$Throughput = Arrival\ rate \times \big(1 - P(failure\ during\ processing)\big) \quad (15)$$

$$Throughput = 5 \times (1 - 0.181) = 5 \times 0.819 = 4.095\ products/hour$$

- Total throughput in 24 hours:

$$Total\ throughput = Throughput \times 24 \quad (16)$$

$$Totalthroughput = 4.095 \times 24 = 98.28\ products$$

Summary:

The 24-hour theoretical performance metrics are:

- Station failures: 4.8 failures, causing 144 minutes of downtime
- AGV throughput: 1728 products can be transported
- Inspection load: 152.28 minutes for failed products

- Assembly utilization: 66.7%
- System throughput: approximately 98 products

The theoretical calculations presented in this section provide a detailed baseline for evaluating the performance of the production system over a 24-hour period. By defining key metrics such as station failures, AGV cycle times, inspection loads, assembly utilization, and system throughput, a comprehensive understanding of the system's operational dynamics has been established. These results form the foundation for assessing the effectiveness and accuracy of the selected DES libraries. The next section will focus on implementing the case study using these libraries, translating the theoretical framework into practical simulations to analyze their performance and capabilities in replicating the outlined scenarios.

5.2 Case-Study implementation

The following section describes the implementation of the case study, which was carried out using the five above-mentioned selected Python libraries SimPy, Salabim, PyDSOL-core, Akatosh, and HADES-framework. The aim of this implementation was to develop a robust model that fulfils the simulation requirements described in the previous chapter. The implementation considers the modelled processes, includes arrival times, resource utilization, processing stations, AGV transport and buffer management. Each step was carefully planned and implemented with the specific function of each library.

5.2.1 Implementation strategy

The implementation of the case study follows three main phases: Pre-processing, main-processing, and post-processing. Each phase was tailored to the specific requirements of the case study and implemented using unique features of each library. Figure 8 shows the implementation steps followed in all libraries.

Implementation of Case

```
Initialize parameters (timing, AGV capacity, buffer sizes, etc.)
Initialize job queue, AGV status, buffers (Product_A, Product_B, NOK)
Start simulation loop (total simulation time: 24 hours)

While simulation time < total time:
    # Pre-Processing:
    For each raw material:
      Inspect and classify (Product_A or Product_B)
      Forward valid products to Station_1
      Send defective products to NOK buffer

    # Main Processing:
    For each product:
        If Station_1 is idle and job queue is not empty:
            Assign job to Station_1
            Process job
        If Station_1 fails:
```

```
            Mark product as defective
            Send to NOK buffer
            Repair Station_1
           Perform manual inspection on processed products:
              If passes, forward to Buffer_A or Buffer_B
              If fails, send to NOK buffer

        # Post-Processing:
         While AGV has tasks:
            If AGV is idle and products in buffer:
              Transport products to assembly station
            If buffer is full:
              Wait for buffer space or adjust process flow
            Assemble final products:
             If passes final inspection, prepare for shipping
             If fails, rework or discard products

         # Metrics Calculation:
            Track and calculate throughput, AGV utilization, product failure rates, and inspection load

        # Save result as JSOP, Excel or .CSV format

 End of simulation: Output performance metrics
```

Figure 8. Implementation of all steps for the case study.

Pre-processing:

In the first phase of implementation, the products (Product_A and Product_B) were created and assigned to product type specific production requirements. A fixed interval of 10 minutes was defined for the arrival times. In SimPy, this was realized using the env.timeout method, while Salabim used the component class to model products as independent entities. HADES-framework and PyDSOL-core enabled event scheduling for product generation, while Akatosh generated the products dynamically with flexible attributes.

Main processing:

The main processing phase comprised the modelling of Station_1 with three processes (Process_1, Process_2 and Process_3), manual inspection stations and the simulation of station failures. SimPy used resource objects to model the queues at the stations. The failures were integrated using user defined events. Salabim used a combination of resource objects and the lifecycle methods of component class to simulate processing and error handling. HADES-framework provided flexible control of station failures and repair processes through its event-driven architecture. PyDSOL managed parallel event queues to synchronize processing and error events. Akatosh supported adaptive fault modelling which allowed products to be dynamically rerouted.

Post-processing:

In the final phase of the implementation, the transport by AGVs, buffer management and final assembly were modelled. Salabim offered an integrated AGV class and animation engine that visualized models in real life. In SimPy and HADES framework, the movements logic and path planning of the AGVs had to be implemented manually. PyDSOL-core provides detailed statistical summary to assess AGV utilization, while Akatosh enabled dynamic routing based on the state of the buffers. Lastly, assembley was modelled in all libraries with a fixed processing time of 10 minutes.

All the input data was kept constant and at the beginning of the code for all five libraries to offer flexibility in terms of changes in times. To experiment with buffer priorities, like FIFO and LIFO, one variable was created which was used by buffers to decide priorities of the products.

This chapter detailed the design and implementation of the case study, including the theoretical groundwork for evaluating the performance of a production system over a 24-hour period. Key metrics such as failure rates, AGV cycles, inspection loads, and throughput were analyzed to establish a clear baseline for assessing the effectiveness of the selected DES libraries in modeling and simulating complex workflows. The implementation steps, along with the full source code for each simulation, are included in the appendix of this thesis. The next chapter focuses on evaluating the selected libraries against the predefined 16 features, comparing their performance based on the simulation results.

6. Evaluation of selected libraries

In this chapter the evaluation of the performance of the five selected libraries- SimPy, Salabim, PyDSOL-core, Akatosh and HADES-framework, based on the 16 features defined for comparison is described. The evaluation considers built-in functions, specific code implementations and special features of each library. In the following paragraphs, the evaluation of each library will be discussed.

6.1 Evaluation of SimPy

SimPy is known for its simple and flexible event control, making it particularly suitable for discrete event modelling. Nevertheless, the implementation of the case study with SimPy posed challenges, particularly in the area of advanced features such as the modelling of free moving objects and statistical analysis. Therefore, in the following section the performance of SimPy is evaluated against the defined criteria of this case study

1. **Customization of arrival processes:** With the env.timeout method, SimPy offers a basic modelling of arrival processes. However, different arrival frequencies and batch arrivals had to be supplemented by an user-defined code. The following function, as shown in Figure 9, generates products at regular intervals.

```
def product_arrival(env, system):
    while True:
        yield env.timeout(PROCESSING_TIME)  # Wait for next product arrival
        product = random.choice(['Product_A', 'Product_B'])  # Randomly select product type
        env.process(system.process_product(product))  # Start product processing
        env.process(system.agv_transport(product))  # Start AGV transportation
        # Start assembly if two products are available
        if len(system.successful_products) >= 2:
            env.process(system.assembly(system.successful_products.pop(0),
                                        system.successful_products.pop(0)))  # Record metrics at each time step
        time_metrics.append({
            'time': env.now,
            'station_failures': metrics['station_failures'],
            'agv_cycles': metrics['agv_cycles'],
            'throughput': metrics['throughput'],
            'failed_products': metrics['failed_products'],
            'total_downtime': metrics['total_downtime']
        })
```

Figure 9. Function of the arrival process in SimPy.

2. **Resource management:** SimPy provides classes such as Resource and PriorityResource to enable the management of resources with capacity restrictions and priorities. Figure 10 shows an example code. The utilization is not directly integrated and require additional coding.

```
def process_product(self, product):
    with self.station_1.request() as req:
        yield req  # Request access to Station_1
        yield self.env.timeout(PROCESS_1_TIME)  # Perform shared process
        # Check for failure during processing
        if random.uniform(0, 1) < self.failure_probability:
            metrics['failed_products'] += 1  # Increment failed product count
            self.failed_products.append(product)  # Track failed product
            return  # Stop further processing for failed product

        # Product-specific processing
        if product == 'Product_A':
            yield self.env.timeout(PROCESS_2_TIME)  # Exclusive process for Product_A
        elif product == 'Product_B':
            yield self.env.timeout(PROCESS_3_TIME)  # Exclusive process for Product_B

        self.successful_products.append(product)  # Track successful product
```

Figure 10. Code snippet for the resource management in SimPy.

3. **Simulation of mobile resources:** SimPy has no specific support for the simulation of AGVs. Movements had to be modelled manually. The movement logic for AGVs was implemented as shown in Figure 11.

```
def agv_transport(self, product):
    with self.agv.request() as req:
        yield req  # Request access to AGV
        # Simulate travel time and pickup/drop-off
        yield self.env.timeout(PICKUP_DROPOFF_TIME + (AGV_DISTANCE /
                                AGV_SPEED) + PICKUP_DROPOFF_TIME)
        metrics['agv_cycles'] += 1  # Increment AGV cycle count
        metrics['products_transported'] += 1  # Increment transported product count
```

Figure 11. Code that describes the simulation of mobile resources in SimPy.

4. **Queue management:** The management of FIFO is natively supported by resource objects. Separate data structures had to be defined for LIFO and other priority queues.
5. **Event control:** SimPy allows simple modelling of events using the env.process and env.timeout methods. Nonetheless, there are no extended mechanisms for synchronizing parallel events or for error handling.
6. **Statistical analysis:** SimPy does not offer any in-built statistical tools. All metrics such as throughput, downtime and AGV cycles were calculated manually and visualized using external tools such as Pandas and Matplotlib.
7. **Scenario tests:** Different scenarios could be modelled flexibly, but without special test frameworks. Scenario testing required manual adjustments to the code.

8. **Visualization of the output:** SimPy does not offer native visualization tools. The diagrams (see Figure 12), which show failed products over time and station failure over time, were created with matplotlib.

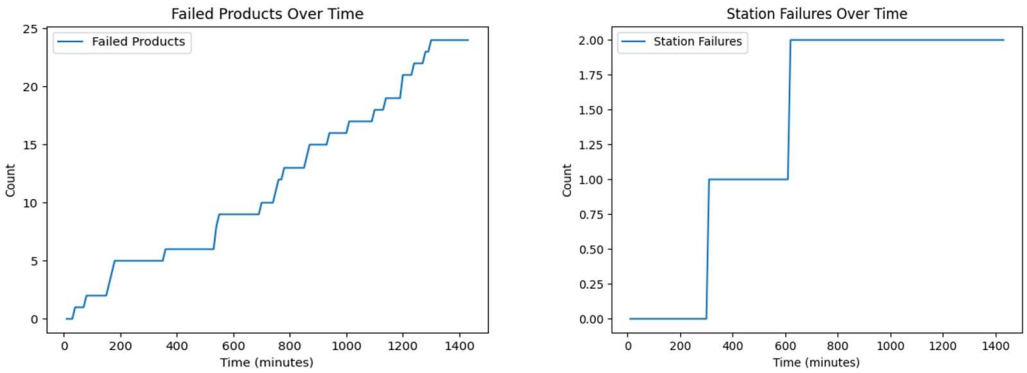


Figure 12. Visual diagrams for SimPy created with matplotlib.

9. **Model validation:** The model has been validated by comparing the simulation results with theoretical calculations and expected system behavior. However, there are no integrated validation functions.
10. **Real-time simulation:** SimPy allows real-time simulation by adjusting the time units. Nevertheless, the real-time interactions had to be implemented manually.
11. **Test framework:** SimPy does not contain any built-in test frameworks. Tests had to be written and executed manually.
12. **Flexibility in queue policies:** FIFO queues are supported, but LIFO and priority policies need to be implemented manually.
13. **Error handling and debugging:** SimPy does not provide any specific debugging tools. Debugging was performed using standard Python methods such as pdb or logging.
14. **Integration capability:** SimPy integrates seamlessly with external libraries such as Pandas, matplotlib and NumPy, which facilitate data processing and visualization.
15. **Event tracking:** Event logging was easily possible through custom logging, but no built-in functions were available.
16. **Data export formats:** SimPy supports the export of simulation data to formats such as CSV or JSON by using external Python libraires.

SimPy proved to be flexible and easy to use, but with limitations when it came to supporting complex functions. The manual implementation of many metrics and processes increased the development effort considerably. The integration of external modules such as pandas and matplotlib was necessary to meet advanced requirements such as statistics and visualization.

6.2 Evaluation of Salabim

Salabim offers a powerful and flexible platform for DES and is characterized by its integrated visualization and analysis tools. These features are particularly useful for modelling

complex systems such as the production process described in this case study. The following evaluation analyzes the performance of Salabim based on the 16 defined comparison criteria.

1. **Customization of arrival processes:** Salabim enables simple modelling of arrival processes with functions such as hold() and activate(). For example, the arrival process of products in this case study was implemented as in Figure 13. This flexibility made it possible to simulate different scenarios with random arrival intervals.

```
def process(self):
    # Generate products at regular intervals and assign them as Product A or Product B
    while True:
        # Randomly choose product type.
        product_type = random.choice(["A", "B"])
        if product_type == "A":
            ProductA(buffer=self.buffer_A, nok_buffer=self.nok_buffer,
                     agvs=self.agvs, workers=self.workers, sink=self.sink).activate()
        else:
            ProductB(buffer=self.buffer_B, nok_buffer=self.nok_buffer,
                     agvs=self.agvs, workers=self.workers, sink=self.sink).activate()
        yield self.hold(random.uniform(PRE_PROCESSING_TIME * 60,
                                        PRE_PROCESSING_TIME * 60))
```

Figure 13. Code snippet for the customization of the arrival processes in Salabim.

2. **Resource management:** Salabim provides the Resource class to manage resources such as workers and AGVs. Resources can be defined with capabilities and priorities. In this case study, this was used for AGVs and other components (see Figure 14). Salabim also supports the management of parallel resource requests.

```
class ProductionSimulation(sim.Component):
    def setup(self):
        # Define queues for Product A, Product B, and defective products (NOK buffer)
        self.buffer_A = sim.Queue(name='Buffer A', capacity=float('inf'))
        self.buffer_B = sim.Queue(name='Buffer B', capacity=float('inf'))
        self.nok_buffer = sim.Queue(name='NOK Buffer', capacity=float('inf'))

        # Define resources for AGVs and workers
        self.agvs = sim.Resource('AGV', capacity=3)
        self.workers = sim.Resource('Worker', capacity=21)

        # Define the sink component to track completed products
        self.sink = SinkComponent()  # Create sink for tracking products
```

Figure 14. Code snippet for the resource management with Salabim.

3. **Simulation of moving resources:** The AGVs simulation was implemented by combining hold() for the journey time and request() for resource allocation. Example of AGV modeling is shown in Figure 15.

```
yield self.request(self.agvs)  # Request an AGV for transport.
agv_travel_time = (AGV_DISTANCE / AGV_SPEED) +
AGV_PICKUP_DROPOFF_TIME
yield self.hold(agv_travel_time)  # Simulate AGV travel.

# Simulate AGV breakdowns and repairs.
if random.random() < (1 / MTBF):
    stats["agv_breakdowns"] += 1
    stats["event_log"].append(f"AGV breakdown occurred at time {self.env.now()}")
    yield self.hold(REPAIR_TIME * 60)  # AGV breakdown
yield self.hold(ASSEMBLY_PROCESSING_TIME * 60)  # Simulate assembly process.
if self.in_buffer:
    self.leave(self.buffer)  ## Leave the buffer if previously entered.
self.release(self.agvs)  # Release AGV after use.
```

Figure 15. Code snippet for the simulation of moving resources in Salabim.

4. **Queue management:** With the queue class, Salabim offers robust support for queues. The queue management policy (e.g. FIFO, LIFO) can be easily configured. Three queues were used in this case study: Buffer A, Buffer B and NOK Buffer.
5. **Event control:** Salabim offers an advanced event-driven architecture supported by functions such as activate() and cancel(). These mechanisms facilitate the management of complex event sequences.
6. **Statistical analysis:** Salabim contains integrated tools for collecting and analyzing statistics. For example, metrics such as throughput and AGV failures, as in Figure 15, have been calculated directly in the model and stored in JSON or CSV formats.
7. **Scenario tests:** This can be carried out by simply modifying the parameters and resources in the model. This enables efficient evaluation of different configuration options.

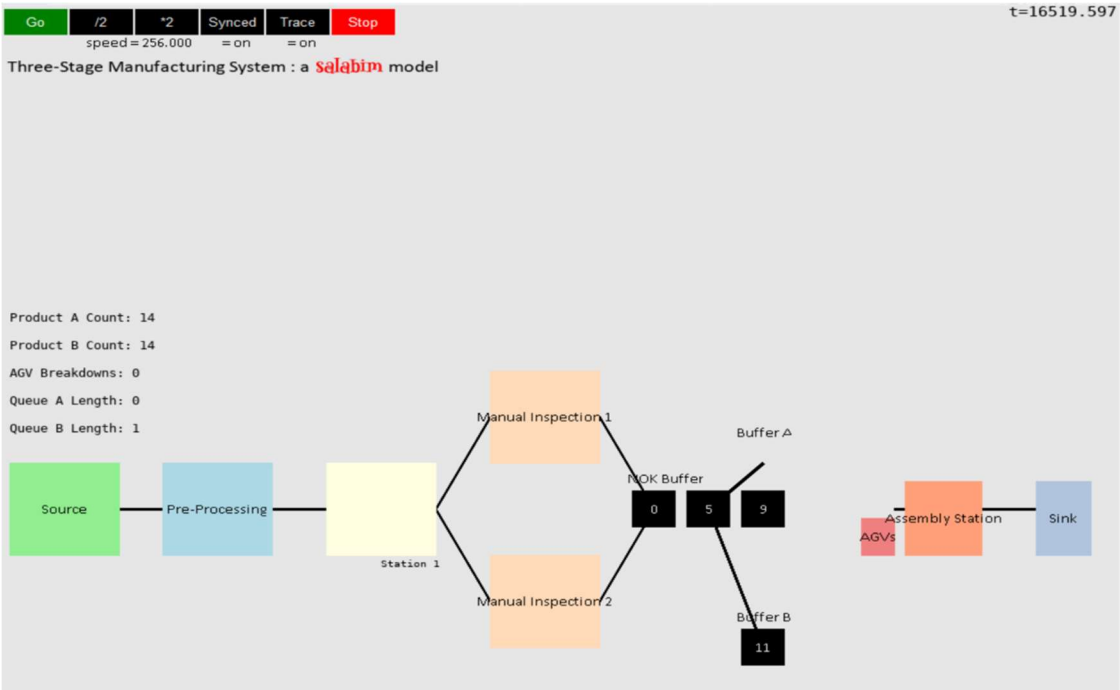


Figure 16. 2D animation of the case study with Salabim.

8. **Output visualization:** Salabim offers powerful visualization functions, including 2D and 3D animations. Figure 16 shows a visual representation of the material flow and production statistics. These integrated animations allow real-time monitoring of the simulation.
9. **Model validation:** The validation of the model was supported by comparing the simulation results with the theoretical calculations. The integrated tools facilitate data acquisition for validation.
10. **Real-time simulation:** Salabim allows real-time simulation and provides control tools such as trace and pause to analyze the simulation during runtime.
11. **Test framework:** Although there is no dedicated test framework, tests can be performed simply by adjusting the simulation parameters.
12. **Flexibility in queuing policy:** Salabim supports flexible queuing policies. FIFO is implemented by default, while other policies can be configured by customizing the code.
13. **Error handling and debugging:** Error handling has been implemented through customized logging. An example of logging AGV failures can be seen in Figure 17.

```
# Simulate AGV breakdowns and repairs.
if random.random() < (1 / MTBF):
    stats["agv_breakdowns"] += 1
    stats["event_log"].append(f"AGV breakdown occurred at time {self.env.now()}")
    yield self.hold(REPAIR_TIME * 60)  # AGV breakdown
```

Figure 17. Code snippet example for logging AGV failures in Salabim.

14. **Integration capability:** Salabim integrates seamlessly with external libraires such as pandas and matplotlib to analyze and visualize data.
15. **Event tracking:** Event logging is supported by functions such monitor and user-defined logs.
16. **Data export formats:** Salabim enables the export of simulation data in formats such as CSV, JSON and Excel, with the support of external libraries.

Salabim offers extensive functionality for the modelling and animation of discrete event systems. Particularly noteworthy are the integrated visualization tools and the support for moving resources such as AGVs. The ease of use and flexibility make Salabim an excellent choice for complex simulations.

6.3 Evaluation of PyDSOL-core

PyDSOL-core is a robust Python library for DES and offers numerous built-in functions for modelling, analysis and statistics. Its integration with powerful analytical and statistical tools makes it a good choice for developing simulations such as those described in this case study. Below, the library's performance is evaluated against defined comparison criteria.

1. **Customization of arrival processes:** PyDSOL-core allows easy customization of arrival processes using generator classes and the schedule_event_rel function. In this case study, product generation was implemented as in Figure 18.

```
def generate(self):
    t: float = self.simulator.simulator_time  # Current simulation time.
    entity: Entity = Entity(t, self.product_type)  # Create a new entity.
    if self.next:  # Pass the entity to the next station.
        self.next.arrive(entity)
    self.fire_timed(t, Generator.GENERATED_EVENT, 1)  # Fire a generation event.
    # Schedule the next generation.
    self.simulator.schedule_event_rel(self.iat, self, "generate", priority=0)
```

Figure 18. Code snippet of the customization of the arrival processes in PyDSOL-core.

2. **Resource management:** Resources are modelled by dedicated classes such as servers, which automatically log waiting times and utilization. Figure 19 visualize an example of resource management, which enables a detailed analysis of resource utilization during the simulation.

```
class Buffer(Station):
    """
    Represents a buffer in the system for storing entities temporarily.
    """
    def __init__(self, simulator: DEVSSimulatorFloat, buffer_name: str):
        """
        Initialize the buffer with a given name.
        :param simulator: The simulation environment.
        :param buffer_name: The name of the buffer for identification.
        """
        super().__init__(simulator)
        self.buffer_name = buffer_name  # Name of the buffer.
        self.queue = []  # Queue to hold the entities in the buffer.

    def arrive(self, entity: Entity):
        t: float = self.simulator.simulator_time  # Current simulation time.
        self.queue.append(entity)  # Add the entity to the buffer's queue.
        print(f"Entity {entity} stored in {self.buffer_name} at t={t}")  # Log the arrival.
```

Figure 19. Code snippet of the resource management with PyDSOL-core.

3. **Simulation of moving resources:** Although PyDSOL-core does not provide integrated support for the movement of resources such as AGVs, this has been realized manually by implementing travel times and queues. The use of the class facilitates routing of product buffers.
4. **Queue management:** The queues are implemented using a combination of queue data structure and event tracking. This allows waiting times and queue lengths to be logged efficiently.
5. **Event control:** The event-driven architecture of PyDSOL-core is based on mechanism that supports the planning and synchronization of events. This enables the simple modelling of processes with parallel events.

6. **Statistical analysis:** PyDSOL-core offers powerful tools such as Simcounter, SImTally, and SimPersistent, which automatically calculate detailed statistics. In this case study, metrics such as average wait time and resource utilization were collected using these functions.
7. **Scenario tests:** This can be carried out by simply adjusting the parameters in models. This makes PyDSOL-core a suitable choice for extensive tests.
8. **Output visualization:** PyDSOL-core does not offer any integrated visualization tools. Diagrams were created using matplotlib, such as the total throughput.
9. **Model validation:** The validation was supported by comparing results with the theoretical calculations. The statistical tools of PyDSOL-core facilitate the validation considerably.
10. **Real-time simulation**: It is possible with PyDSOL-core, but not integrated by default. The simulation is completely controlled by user-defined parameters.
11. **Test framework:** PyDSOL-core does not offer dedicated test frameworks, but tests can be realized by adapting the models and input data.
12. **Flexibility in queuing priority:** FIFO queues are supported by default, while other policies such as LIFO must be implemented manually.
13. **Error handling and debugging:** Error handling has been realized through user-defined logs. PyDSOL-core does not provide built-in debugging tools, but the structure facilitates logging and tracing of errors.
14. **Integration capability:** PyDSOL-core integrates seamlessly with external libraries such as pandas and matplotlib for analysis and visualization.
15. **Event tracking:** EventProducer class enables events to be logged and analyzed. This was used in this case study to track events such as generation, queue time and throughput.
16. **Data export formats:** Results can be exported in various formats such as CSV, JSON and Excel using external libraries.

PyDSOL impresses with its robust statistics and analysis tools but is limited in terms of visualization. The event-driven architecture facilitates the modelling of complex systems, while the integration with external tools offers a high degree of flexibility. It is an excellent choice for scenarios that require extensive statistical analyzes.

6.4 Evaluation of Akatosh

Akatosh offers extensive flexibility and supports the modelling of dynamic material flows, including real-time decisions and event processing. The focus is on the adaptability of event processing, resource management, model validation and integration of results for detailed analysis. This evaluation describes the features of the library based on predefined criteria.

1. **Customization of arrival process:** Akatosh library enables the customization of arrival processes by creating scheduled events for product generation. The Event class is used to control product arrivals at specific time intervals, as shown in Figure 20, in the

source class of the implementation. This feature provides flexibility in defining dy namic arrival processes, such as varying product generation rates.

```
# Define the Source class to generate products
class Source:
    def __init__(self):
        """
        Initialize the product source and start generating products.
        """
        self.product_count = 0  # Counter for the total number of products created.
        self.create_product_event()  # Schedule the first product generation event.

    def create_product_event(self):
        Event(universe.time, universe.time, action=self.generate_product,

                label="Product Generation", once=True)
        self.product_count += 1  # Increment the product counter.
        # Schedule the next product generation if within simulation duration.
        if universe.time +
            PRODUCT_ARRIVAL_INTERVAL < SIMULATION_ DURATION:
           Event(universe.time + PRODUCT_ARRIVAL_INTERVAL,
                  universe.time + PRODUCT_ARRIVAL_INTERVAL,
                  action=self.create_product_event,
                  label="Next Product Generation", once=True)

    def generate_product(self):

        # Alternate between Product_A and Product_B.
        product_type = "Product_A" if self.product_count % 2 == 0 else "Product_B"

        # Create a new entity.
        product = Entity(universe.time, SIMULATION_DURATION,
                         label=f"{product_type}_{self.product_count}")

        # Update pre-processing queue length.
         queue_lengths["pre-processing"].append((universe.time,
         queue_lengths["pre-processing"][-1][1] + 1 if queue_lengths["pre-processing"]
          else 1))
```

Figure 20. Code snippet for the customization of the arrival process with Akatosh.

2. **Resource management:** Akatosh has a robust resource management system. The Resource class allows the definition of resources, their capabilities, current fill level and utilization per entity. Entities can be acquired or released by resources, allowing detailed control over resource distribution and utilization.
3. **Simulation of mobile resources:** In Akatosh, mobile resources, such as AGVs, are modelled using events and entities, due to lack of in-built function to model and simulate moving resource. The interaction between events and resources enables the simulation of their utilization and availability during different process stages. The integrated event processing functionality makes it possible to efficiently move resources and track how they are used in the simulation.

4. **Queue management:** The library enables the management of queuing policies such as FIFO and LIFO via the flexible implementation of events and queues. Users can customize the queuing policy by defining priority and order of queuing events. Although specific queuing policies such as LIFO are not included as standard feature, they can be added by customizing the code to support different priority strategies.
5. **Event processing logic:** Akatosh is highly flexible in event processing. Events can be dynamically created, paused, resumed or cancelled and therefore allowing complete control over the simulation scenarios.
6. **Statistical analysis:** This is facilitated by recording data throughout the simulation. Metrics such as queue lengths, throughput times, and station failures are captured and stored.
7. **Scenario testing:** It can be easily performed in Akatosh by adjusting the event schedules and resource parameters. The 'check_for_failure()' function was used to test system robustness and response to random failures during the implementation.
8. **Visualization:** The Akatosh library supports result visualization using matplotlib to create diagrams. This enables system performance to be analyzed during and after simulation.
9. **Model validation:** It is supported by detailed recordings of key events and system states. The simulation captures the times of key events, allowing users to verify those processes are running as planned. This supports to verify the correctness of the model implementation.
10. **Real-time simulation:** Akatosh offers real-time simulation using 'asyncio' and the 'enable_realtime()' method, which makes it possible to synchronize the simulation time with real-time. Akatosh also offers in-built pause and resume of simulation functions.
11. **Test framework:** Akatosh does not provide dedicated test frameworks but tests can be carried out by adjusting the simulation parameters and model configurations. This provides a basic way to perform scenario testing and verify system performance.
12. **Queue policy flexibility:** The library allows for the management of queue policies such as FIFO through flexible implementation of events and queues. Although specific queue policies such as LIFO are not included as standard functionality, these can be added by adapting the code.
13. **Error handling and debugging:** Is implemented via custom logging and the use of functions such as 'watchdog()', which is activated when events fail to meet their defined deadlines. This method supports to identify and fix problems during the simulation.
14. **Integration capabilities:** Akatosh allows for integration of external libraries such as Pandas and Matplotlib to support data analysis and visualization
15. **Event tracing:** Akatosh offers event traceability by the built-in 'event_log' function, which ensures that all important events during the simulation are recorded. This enables a comprehensive analysis of simulation steps and their timing.

16. **Data output formats:** Akatosh library supports multiple data output formats for saving simulation results, including CSV, JSON and Excel with the help of external libraries.

The Akatosh library provides a powerful and flexible environment for implementing DES. Particularly noteworthy are the integrated functions for event management and resource control, which enable detailed modelling and analysis. The real-time simulation capability, as well as the flexibility for customizing resources queues, make Akatosh an excellent choice for modelling complex production processes. Some areas, such as the development of dedicated test framework, could still be improved, but overall Akatosh is a solid platform for analyzing and simulating system dynamics.

6.5 Evaluation of HADES-framework

The HADES-framework offers a wide range of integrated functions for modelling, analysis and statistics. In the following evaluation, the performance of the HADES-framework is discussed based on the 16 criteria identified for this workcase study.

1. **Customization of arrival process:** The HADES-framework makes it possible to flex ibly customize the arrival processes. This is done by using events and processes, as shown in Figure 21. Here we can flexibly define interval times and batch arrivals:

```
# Custom arrival process - Manages product arrival schedules
class CustomArrivalProcess(RandomProcess):
    def __init__(self, seed):
        super().__init__(seed)  # Initialize with a random seed
    async def notify(self, event: Event):
        if isinstance(event, SimulationStarted):
            t = event.t  # Get the event time
            # Schedule arrivals for Product_A and Product_B with random intervals
            self.add_event(ProductPreProcessed(t=t + self.random.randint(1, 5),
                                   product_type="Product_A"))
            self.add_event(ProductPreProcessed(t=t + self.random.randint(5, 10),
                                   product_type="Product_B"))
            # Continuously add new arrivals to extend the simulation
            self.add_event(SimulationStarted(t=t + 30))
            return NotificationResponse.ACK
        return NotificationResponse.NO_ACK
```

Figure 21. Code snippet for the customization of the arrival process with HADES Framework.

2. **Resource management:** In the HADES Framework, resources are modelled by pro cesses that control the availability and the use of resources such as AGVs, inspection stations and assembly workstations. For example, Figure 22 shows the AGV Transport class implemented for case study.

```
class AGVTransport(Process):
    async def notify(self, event: Event):
        # Check if the event is an inspection completion and the product status is OK
        if isinstance(event, InspectionCompleted) and event.status == "OK":
            t = event.t  # Get the event time
            product_type = event.product_type  # Get the product type
            print(f"AGV transporting {product_type} from buffer to assembly at time {t}")
            distance = AGV_DISTANCE  # Distance to be traveled (meters)
            speed = AGV_SPEED  # AGV speed (meters per second)
            transport_time = distance / speed  # Calculate transport time
            # Schedule an event for AGV transport start
            self.add_event(AGVTransportStarted(t=t + transport_time,
                                    product_type=product_type))
            # Log the transport operation
            results["events"].append({"time": t, "event": "AGV transport started",
                                      "product_type": product_type})
            results["statistics"]["agv_transports"] += 1  # Increment transport count
            return NotificationResponse.ACK
        return NotificationResponse.NO_ACK
```

Figure 22. Code snippet for the resource management with HADES Framework.

3. **Simulation of mobile resources:** The movements of resources, such as AGVs, are simulated by transport events, which have to be implemented manually. Figure 22 represents such an event where 'AGVTransport' class controls how products are transported between buffer and assembly station. This implementation enables a realistic simulation of the movement and availability of resources.
4. **Queue management:** HADES supports different queue policies, such as FIFO or LIFO, through flexible customization in process. Nevertheless, it does not offer any inbuilt queuing priorities.
5. **Event processing logic:** The HADES-framework provides a comprehensive event architecture that supports events. These events are processed by the 'notify()' method, which allows complex process flows to be modelled.

```
# Statistical analysis process - Collects and analyzes data
class StatisticalAnalysis(Process):
    async def notify(self, event: Event):
        if isinstance(event, FinalCheckCompleted):
            t = event.t  # Get the event time
            status = event.status  # Get the status of the final check
            print(f"Collecting statistical data at time {t} for status: {status}")
            # Log the statistical analysis operation
            results["events"].append({"time": t, "event": "Statistical data collected", "status":
status})
            return NotificationResponse.ACK
        return NotificationResponse.NO_ACK
```

Figure 23. Code snippet for the statistical analysis with HADES Framework.

6. **Statistical analysis:** The statistical analysis is supported by collecting and recording data during the simulation but have to be manually implemented. For example, the 'StatisticalAnalysis' class in Figure 23 logs inspection status and product completion.
7. **Scenario testing:** Scenario tests can be easily carried out by adjusting various parameters in the HADES framework. By varying the arrival intervals, station failures or repair times, the behavior of the system can be examined in different scenarios.
8. **Visualization:** For result visualization, HADES must be integrated with external libraries like matplotlib.
9. **Model validation:** HADES does not support model validation natively This has to be done by comparing model with theoretical calculation.
10. **Real-time simulation:** The HADES-framework supports a type of real-time simulation by synchronizing the simulation time with actual time. While pause and resume of simulation is not supported by default methods.
11. **Test framework:** Although there is no dedicated test framework in the HADES framework, tests can be easily carried out by adjusting the input data and model parameters. This provides a base for scenario testing and evaluating system performance.
12. **Queue policy flexibility:** Queue priorities, such as FIFO and LIFO, can be flexible adapted by implementing the events. This flexibility allows priorities to be changed to production requirements. Queue policies must be manually modified.
13. **Error handling and debugging:** The HADES-framework handles errors by logging events, the logging helps to identify errors and to do necessary adjustments.
14. **Integration capabilities:** This library also seamlessly integrates with external libraries such as pandas and matplotlib to perform analyzes and visualize results.
15. **Event tracing:** All important events logged during the simulation, enabling precise tracking of the process steps. This allows for a detailed analysis of the processes and their timing.
16. **Data output formats:** By using Python libraries like Pandas and OS, it is possible to export data into file formats like CSV and JSON.

The HADES-framework supports a wide range of event processing and analysis functions. In particular, its flexibility in adapting arrival processes and queue priorities and its integration with external tools such as Pandas and Matplotlib, makes it a good choice for complex simulation scenarios. However, HADES does not provide sufficient documentation, which makes it difficult to use. This is a challenge for users, especially when implementing complex scenarios. A dedicated testing framework and improved documentation would be desirable to further facilitate use. Overall, HADES remains a solid solution for the analysis and simulation of dynamic production processes.

Challenges faced during the implementation

The implementation of the simulation with the five selected Python libraries SimPy, Salabim, PyDSOL-core, Akatosh and HADES-framework posed specific challenges, which were caused by the functional limitations of the respective libraries and the complexity of this case study.

Statistical analysis: Both SimPy and Akatosh lacked built-in statistical analysis capabilities, making it difficult to calculate metric such as throughput, downtime and AGVs cycles. In contrast, PyDSOL core and Salabim offered comprehensive integrated analysis tools that made post-processing and interpretation of the results much easier. HADES-framework only had limited functions, which made it necessary to integrate external analysis tools.

Visualization: A major drawback of HADES framework, SimPy, Akatosh and PyDSOL was the lack of native visualization tools. While Salabim provided a powerful animation engine, in other libraries external tools such as matplotlib must be integrated to create graphical representations of the results. This led to additional development and testing efforts, especially when visualizing AGV movements and queue dynamics.

Event management and error handling: In HADES, the synchronization of parallel events was a complex challenge. The event-driven architecture required careful planning and implementation of dependencies between processes to avoid deadlocks and wrong sequences. SimPy and Akatosh present less difficulties in this topic, as their architecture allows for easy handling of events. However, all libraries lack comprehensive debugging tools that could automatically detect and fix errors and anomalies. While HADES-framework and Salabim offer advanced event logging and traceability, users in SimPy, Akatosh and PyDSOL-core, relay heavily on manual error handling and the use of external debugging mechanisms such as Python's logging or pdb.

AGV routing: The implementation of dynamic routing logic for AGVs varies greatly between the five libraries, SimPy does not provide native functions for AGV movements path allocation, such features must be implemented entirely manually. Similarly, HADES-framework supports basic event scheduling but does not provide specialized support for free moving resources. Akatosh supports dynamic routing through real-time monitoring of buffer states but requires significant customization for complex real-time applications. On the other hand, Salabim, provides robust tools for AGV simulation and animation, including the ability to visually represent AGV routes. However, this comes with a steep learning curve. PyDSOL-core offers limited support for the AGV routing. Motion logic and routing must be largely defined manually, although good statistical analysis tools facilitate evaluation.

Integration of external modules: The integration of external modules was required in some libraries, especially for functions such as data analysis and visualization. Salabim and PyDSOL provided native functions for many use cases, minimizing the need for external modules. Salabim included integrated visualization tools, while PyDSOL provided robust statistical analysis tools. SimPy, HADES and Akatosh, on the other hand, required extensive external modules to achieve similar functionality. For example, SimPy and Akatosh required matplotlib for visualization and pandas for data analysis. HADES-framework was particularly challenging as both external libraries and additional customization were required to efficiently analyze and display data. The limited documentation in HADES-framework made the integration and use of external tools considerably more difficult.

General complexity: The implementation of this case study was challenging due to its complex processes such as station failures modelling, manual inspection and free moving objects like AGVs and humans. Especially limited community support and documentation for libraries like HADES-framework, Akatosh and PyDSOL-core, made it difficult to model and simulate, while SimPy and Salabim offered extensive support from community as well as documentation as matured libraries.

The implementation challenges varied depending on library and application area. While Salabim and PyDSOL were convincing due to their comprehensive integrated functions, SimPy, HADES-framework and Akatosh required considerable additional development to fulfil the requirements of the case study.

The evaluation highlights the versatility of open-source DES libraries, but also shows that no single library fully covers all requirements. These findings form the base for the recommendations and perspectives presented in the next chapter and provide valuable insights into future developments and research in this area.

7. Conclusion

This master work analyzes and compares five Python libraries for DES – SimPy, Salabim, PyDSOL-core, Akatosh and HADES-framework – using a case study involving the simulation of the behavior of automated vehicles and humans in complex processes. The results revealed that no library is universally superior, but each offers specific strengths for different requirements, such as ease of use in SimPy, powerful animation features in Salabim or advanced resource management in PyDSOL-core. Nevertheless, limitations such as limited documentation and lack of hybrid modelling capabilities were identified. This chapter concludes for this work the strengths and limitations of this work, comparing the selected libraries, suggest practical applications and recommendations for future research direction.

7.1 Strengths of this work

This work is characterized by several notable strengths. One of the key strengths is the broad evaluation spectrum, which involved the consideration of numerous criteria ranging from resource management to visualization and statistical analysis. This comprehensive approach allowed for an in-depth evaluation of the libraries. Additionally, the work is marked by its practical orientation, as the analysis was grounded in the real requirements of industrial applications. This focus ensures that the findings and recommendations are highly relevant to practical scenarios. Furthermore, the diversity of the tools examined adds significant value. The selected libraries encompass a wide range of use cases, catering to both simple projects and complex simulations with real-time requirements, thereby offering versatility and adaptability for varied simulation needs.

7.2 Limitations of this work

Despite the comprehensive analysis conducted in this study, there are several limitations that should be acknowledged. First, the evaluation was based on general industrial scenarios and did not delve deeply into specific domains such as logistics or healthcare, which may have unique simulation requirements. Second, the study lacked quantitative benchmarks, as aspects like simulation runtime and memory consumption were not analyzed in detail. Third, the evaluation of certain features, such as visualization and debugging capabilities, relied heavily on user feedback and existing documentation rather than direct empirical testing. Additionally, it is important to note that not all Python libraries for DES were analyzed in this work, meaning that some potentially relevant tools may have been excluded from consideration. These limitations highlight areas for potential future exploration and refinement.

7.3 Comparative analysis of selected libraries

The Comparative analysis of selected libraries- SimPy, Salabim, PyDSOL-core, Akatosh and HADES-framework showed that each library has its own strengths and weaknesses depending on the specific simulation requirements. All essential features and functions of these libraries were compared in detail in a summary table (Table 5) to provide a clear base for deciding how to use them in different scenarios.

Table 5. Comparative analysis of the selected libraries for this case study.

Features	SimPy	Salabim	PyDSOL-core	Akatosh	HADES Framework
Customiza-tion of Arrival processes	Manual code needed for customiza-tion	Easy customization via built-in functions	Custom generators for arrivals	Flexible event scheduling	Flexible customization
Resource management	Basic resource management classes	Robust resource management with priorities	Detailed classes for resources	Robust resource handling	Resource modelling with processes
Simulation of mobile resources	No native feature	Supports AGVs with animation	No native support	Simulates AGVs via events	Manual AGV simulation
Queue management	Supports FIFO; others manual	Buit-in support for FIFO/LIFO	Queues supported with events	FIFO/LIFO supported via customization	FIFO/LIFO via processes
Event control	Basic event handling	Advanced event-driven control	Advanced event-driven model	Dynamic event control	Comprehensive event architecture
Statistical analysis	No built-in tools	Integrated tools	Powerful statistical tools	Tracks metrics directly	Data recording but manual
Scenario tests	Manual adjustment of code	Simple modifications	Parameter based adjustments	Adjustable events/re-sources	Adjustable parameters
Visualization of the output	Uses external libraries for graphs	Integrated basic 2D/3D layout visualization	No native visualization	Graphical visualization via matplotlib	Graphical visualization via matplotlib

Model Validation	Manual validation required	Validated via integrated stats	Supported with stats tools	Detailed event logs	Manual comparison required
Real-time simulation	Manual implementation needed	Supports real-time control	User-defined control	Real-time simulation supported	Partial real-time sync
Test framework	Manual	Manual	Manual	Manual	Manual
Flexibility in queue policies	Only FIFO by default	Flexible queue policies	FIFO; other manual	Flexible queue logic	Queue policies customizable
Error handling and debugging	Uses standard Python tools	Error logs provided, but requires external libraries	Custom logs used	Watchdog/logging	Event logging used
Integration capability	Integrates with pandas/matplotlib	Seamless integration	Integrates with pandas/matplotlib	Integrates with pandas/matplotlib	Integrates with pandas/matplotlib
Event tracking	No built-in tracking	Built-in monitoring/logging	Supports event tracking	Buit-in traceability	Logs key events
Data export formats	With external libraries	With external libraries	With external libraries	Multiple formats supported	With external libraries

Salabim stands out due to its advanced features, built-in tools, and ease of use, making it an excellent choice for users seeking comprehensive and user-friendly simulation capabilities. PyDSOL-core and Akatosh cater to users who require detailed resource management and customization options, though they may involve a more complex setup process. SimPy, being lightweight and flexible, is better suited for smaller and simpler projects as it lacks the advanced features offered by the other frameworks. HADES excels in robust process modeling and event-

driven architecture but often requires manual implementation for several features, which can increase the development effort.

7.4 Practical applications of DES libraries in industry

The libraries evaluated in this work demonstrate a diverse range of applications across various domains. For supply chain optimization, Salabim stands out with its advanced resource management and visualization capabilities, making it particularly well-suited for modeling complex supply chains. SimPy, on the other hand, excels in handling smaller or less complex processes, providing a straightforward and efficient solution. Akatosh proves valuable for real-time control, offering robust features for operational control and decision-making in dynamic environments. Additionally, Salabim's support for AGV makes it an excellent choice for modeling automated transportation systems, highlighting its versatility in automation scenarios.

7.5 Recommendations for future research

Based on the findings of this work, several recommendations for future research can be identified. First, conducting a quantitative performance evaluation would provide valuable insights, with detailed benchmarks on simulation speed, memory consumption, and scalability enabling an objective comparison of the libraries' performance. Second, domain-specific investigations focusing on industries such as transportation, manufacturing, or healthcare could deepen understanding of each library's applicability to specialized scenarios. Third, enhancing usability by developing intuitive user interfaces and interactive tools could make more complex libraries accessible to a broader audience. Lastly, integrating advanced visualization features, such as interactive tools, could significantly improve the usability and appeal of these libraries, particularly for industrial applications.

The results of this work show that the selection of the appropriate simulation library depends strongly on the specific requirements. Each of the investigated libraries offers unique advantages that make them suitable for certain scenarios. Further development in areas such as usability, performance optimization, and visualization will be needed to further expand the use of DES in industry. Future work should focus on quantitative evaluations, domain-specific applications and integration of modern technologies to further maximize the utility of these tools.

Bibliography

Agalianos, K.; Ponis, S. T.; Aretoulaki, E.; Plakas, G.; Efthymiou, O. (2020): Discrete Event Simulation and Digital Twins: Review and Challenges for Logistics. In Procedia Manufacturing 51, pp. 1636–1641. DOI: 10.1016/j.promfg.2020.10.228.

Akatosh (2024). Available online at https://ulfaric.github.io/Akatosh/, updated on 5/31/2024, checked on 7/14/2024.

Atalan, Abdulkadir; Şahin, Hasan; Atalan, Yasemin Ayaz (2022): Integration of Machine Learning Algorithms and Discrete-Event Simulation for the Cost of Healthcare Resources. In Healthcare (Basel, Switzerland) 10 (10). DOI: 10.3390/healthcare10101920.

averbraeck/PyDSOL-core. Python Distributed Simulation Object Library (2024). Available online at https://github.com/averbraeck/PyDSOL-core, updated on 7/14/2024, checked on 7/14/2024.

Bagdatli, B.; Mavris, D. (2012): Use of high-level architecture discrete event simulation in a system of systems design. In : 2012 IEEE Aerospace Conference. 2012 IEEE Aerospace Conference. Big Sky, MT, 03.03.2012 - 10.03.2012: IEEE, pp. 1–13.

Bangsow, Steffen (Ed.) (2012): Use cases of discrete event simulation. Appliance and research. Berlin, Heidelberg: Springer.

Banks, Jerry; Carson, John; Nelson, Barry L.; MacNicol, David (2014): Discrete-event system simulation. Fifth edition. Harlow, United Kingdom: Pearson. Available online at https://ebookcentral.proquest.com/lib/kxp/detail.action?docID=5832676.

Doudareva, Evgueniia; Carter, Michael (2022): Discrete event simulation for emergency department modelling: A systematic review of validation methods. In Operations Research for Health Care 33, p. 100340. DOI: 10.1016/j.orhc.2022.100340.

Fishman, George S. (2001): Discrete-Event Simulation. Modeling, Programming, and Analysis. New York, NY: Springer (Springer eBook Collection Mathematics and Statistics).

Karanjkar, Neha; Joshi, Subodh M. (2023): A Python-Based Mixed Discrete-Continuous Simulation Framework for Digital Twins. In Gerd Wagner, Frank Werner, Tuncer Oren, Floriano de Rango (Eds.): Simulation and Modeling Methodologies, Technologies and Applications, vol. 601. Cham: Springer International Publishing (Lecture Notes in Networks and Systems), pp. 204–223.

Kelton, W. David; Sadowski, Randall P.; Sturrock, David T. (2004): Simulation with Arena. 3. ed., international ed. Boston, Mass.: McGraw-Hill Higher Education (McGraw-Hill series in industrial engineering and management science). Available online at http://www.loc.gov/catdir/enhancements/fy0701/2003052652-d.html.

ki-oss/HADES: HADES. (HADES Asynchronous Discrete-Event Simulation) is a small, user friendly framework for creating simulations in python! (2024). Available online at https://github.com/ki-oss/HADES, updated on 7/14/2024, checked on 7/14/2024.

Lang, Sebastian; Reggelin, Tobias; Müller, Marcel; Nahhas, Abdulrahman (2021): Open-source discrete-event simulation software for applications in production and logistics: An

alternative to commercial tools? In Procedia Computer Science 180, pp. 978–987. DOI: 10.1016/j.procs.2021.01.349.

Law, Averill M.; Kelton, W. David (2000): Simulation modeling and analysis. 3rd ed. Boston: McGraw-Hill (McGraw-Hill series in industrial engineering and management science). Available online at https://scholar.google.com/citations?user=ihn2nqcaaaaj&hl=en&oi=sra.

Macal, Charles; North, Michael (2014): Introductory tutorial: Agent-based modeling and simulation. Savannah, GA, USA: IEEE.

Marszałek, Karol; Domański, Adam; Cupek, Rafał; Lin, Jerry Chun-Wei; Kwiecień, Andrzej (2024): A Queueing System to Jobs Scheduling Problem in AGVs. In: 2024 32nd Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP). 2024 32nd Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP). Dublin, Ireland, 20.03.2024 - 22.03.2024: IEEE, pp. 199–206.

Metropolis, N.; Ulam, S. (1949): The Monte Carlo method. In Journal of the American Statistical Association 44 (247), pp. 335–341. DOI: 10.1080/01621459.1949.10483310.

Mitsuyuki, Taiga; Okubo, Yui (2024): pDESy: A Python package for discrete time-event simulation to engineering project. In Software Impacts 19, p. 100621. DOI: 10.1016/j.simpa.2024.100621.

Matloff (2008): Introduction to Discrete-Event Simulation and the SimPy Languag. Available online at https://heather.cs.ucdavis.edu/matloff/public_html/156/PLN/DESimIntro.pdf, checked on 7/14/2024.

OpenAI. (2025a). ChatGPT (version 4.0), chatbot output from January 9, 2025, regarding the comparative analysis of DES libraries (most relevant prompt: "Generate a German-language conclusion for a work on DES libraries, including strengths, weaknesses, and future research directions"). INTERACTIVE.

OpenAI. (2025b). ChatGPT (version 4.0), chatbot output from January 9, 2025, regarding the summary and comparison of SimPy, Salabim, PyDSOL-core, Akatosh, and HADES libraries (most relevant prompt: "Write a comparative analysis of DES libraries, focusing on strengths and weaknesses"). CREATED.

OpenAI. (2025c). ChatGPT (version 4.0), chatbot revision of a user-provided draft dated January 9, 2025, regarding a work conclusion on DES libraries (most relevant prompt: "Rewrite the conclusion for a work in German, integrating a comparative analysis"). REVISED.

Oujezsky, Vaclav; Horvath, Tomas (2016): Case study and comparison of SimPy 3 and OMNeT++ Simulation. In: 2016 39th International Conference on Telecommunications and Signal Processing (TSP). 2016 39th International Conference on Telecommunications and Signal Processing (TSP). Vienna, Austria, 27.06.2016 - 29.06.2016: IEEE, pp. 15–19.

Palmer, Geraint I.; Knight, Vincent A.; Harper, Paul R.; Hawa, Asyl L. (2019): Ciw: An open-source discrete event simulation library. In Journal of Simulation 13 (1), pp. 68–82. DOI: 10.1080/17477778.2018.1473909.

Pidd, Michael (2004): Computer simulation in management science. 5th ed. Chichester England, Hoboken NJ: Wiley.

Py-DES (2024). Available online at https://pydes.readthedocs.io/en/latest/, updated on 2/10/2024, checked on 7/14/2024.

PyDSOL-core documentation (2024). Available online at https://PyDSOL-core.readthedocs.io/en/latest/index.html, updated on 5/17/2024, checked on 7/14/2024.

PyDSOL-model documentation. PyDSOL-model 0.1 documentation (2023). Available online at https://PyDSOL-model.readthedocs.io/en/latest/index.html, updated on 7/5/2023, checked on 11/27/2024.

PyPI (2024): Python Package Index. Available online at https://pypi.org/, updated between January 2024 and July 2024, checked in this period.

Robinson, S. (2005): Discrete-event simulation: from the pioneers to the present, what next? In Journal of the Operational Research Society 56 (6), pp. 619–629. DOI: 10.1057/palgrave.jors.2601864.

Ruud van der Ham (2024): salabim. Available online at https://www.salabim.org/manual/salabim.pdf, checked on 3/7/2024.

Sargent, R. G. (2013): Verification and validation of simulation models. In Journal of Simulation 7 (1), pp. 12–24. DOI: 10.1057/jos.2012.20.

SimPN (2024). Available online at https://github.com/bpogroup/simpn, updated on 7/14/2024, checked on 7/14/2024.

SimPy documentation (2024). Available online at https://SimPy.readthedocs.io/en/latest/index.html, updated on 10/8/2024, checked on 10/26/2024.

Sun, Zhuanlan; Bai, Ruhai; Bai, Zhenggang (2023): The application of simulation methods during the COVID-19 pandemic: A scoping review. In Journal of Biomedical Informatics 148, p. 104543. DOI: 10.1016/j.jbi.2023.104543.

Taccini, Marco; Iori, Manuel; Dotti, Giulia; Subramanian, Anand (2023): Improving Buffer Storage Performance in Ceramic Tile Industry Via Simulation. In: 2023 Winter Simulation Conference (WSC). 2023 Winter Simulation Conference (WSC). San Antonio, TX, USA, 10.12.2023 - 13.12.2023: IEEE, pp. 1676–1687.

ulfaric/Akatosh (2024). Available online at https://github.com/ulfaric/Akatosh, updated on 7/14/2024, checked on 7/14/2024.

Ullrich, Oliver; Lückerath, Daniel (2017): An Introduction to Discrete-Event Modeling and Simulation. In Simul. Notes Eur. 27 (1), pp. 9–16.

Zeigler, Bernard P.; Prähofer, Herbert; Kim, Tag Gon (2000): Theory of modeling and simulation. 2. ed. USA: Academic Press, Inc.

Appendix

This appendix includes all the code used in the study, showcasing the implementation of a discrete-event simulation for a multi-stage manufacturing process.

SimPy Implementation code:

```
# Import necessary libraries
# SimPy: Used for discrete-event simulation.
# pandas: For data handling and exporting simulation results.
# matplotlib: For visualizing simulation metrics.
# random: For random number generation, e.g., product types and failures.
# json: For saving metrics in JSON format.
# os: For creating directories and handling file paths.
# openpyxl: For exporting metrics to Excel files.
import SimPy
import pandas as pd
import matplotlib.pyplot as plt
import random
import json
import os
from openpyxl import Workbook

# Directory for saving results
# Ensure output directory exists; create it if it does not.
output_dir = “output path”
os.makedirs(output_dir, exist_ok=True)

# Case Study: Three-Stage Manufacturing System
# This simulation models a production system with the following stages:
# 1. Pre-processing: Initial product preparation.
# 2. Main processing: Shared and exclusive workflows for Product A and Product B.
# 3. Post-processing: Manual inspection, transport via AGVs, assembly, and final output.
# The simulation tracks the movement and state of products, AGV breakdowns, queue lengths, and other performance met-
rics.

# Define constants and assumptions for the simulation.
# These constants represent fixed times and probabilities in the system.
PROCESSING_TIME = 10            # Time for pre-processing (minutes)
PROCESS_1_TIME = 8              # Time for shared process at Station_1 (minutes)
PROCESS_2_TIME = 7              # Time for Product_A's exclusive process (minutes)
PROCESS_3_TIME = 7              # Time for Product_B's exclusive process (minutes)
MTBF = 300                      # Mean time between failures (minutes)
REPAIR_TIME = 30                # Time to repair Station_1 after failure (minutes)
INSPECTION_TIME = 5             # Time for manual inspection (minutes)
AGV_SPEED = 1.5                 # Speed of AGVs (meters per second)
AGV_DISTANCE = 15               # Distance AGVs travel (meters)
PICKUP_DROPOFF_TIME = 20        # Time for AGVs to pick up/drop off (seconds)
ASSEMBLY_TIME = 10              # Time for assembly process (minutes)
SIMULATION_DURATION = 24 * 60   # Total simulation duration (minutes)
BUFFER_POLICY = 'fifo'          # Buffer management policy ('fifo' or 'lifo')

# Metrics to store results for visualization and analysis.
# time_metrics stores dynamic metrics over time for plotting.
# metrics aggregates overall performance statistics.
time_metrics = []
metrics = {
    'station_failures': 0,
    'total_downtime': 0,
    'agv_cycles': 0,
    'products_transported': 0,
    'failed_products': 0,
    'throughput': 0,
    'assembly_utilization': 0
}
```

```
# Define the ManufacturingSystem class.
# This class encapsulates the logic for simulating the manufacturing workflow.
class ManufacturingSystem:
    def __init__(self, env):
        self.env = env  # Simulation environment
        # Define resources representing limited-capacity system components.
        self.station_1 = SimPy.Resource(env, capacity=1)  # Processing station
        self.agv = SimPy.Resource(env, capacity=1)        # Automated guided vehicles
        self.assembly_station = SimPy.Resource(env, capacity=1)  # Assembly station
        self.failed_products = []  # List to track failed products
        self.successful_products = []  # List to track successful products
        # Buffer (FIFO or priority-based) for managing products.
        self.buffer = SimPy.Store(env, capacity=float('inf')) if BUFFER_POLICY == 'fifo' else SimPy.PriorityStore(env,
                                  capacity=float('inf'))
        self.failure_probability = 0.181  # Probability of station failure during processing
        self.mtbf = MTBF                  # Mean time between failures
        self.repair_time = REPAIR_TIME    # Time required for station repair
        # Start a background process for simulating station failures.
        env.process(self.process_station_failures())

    # Define the main processing logic for products.
    def process_product(self, product):
        with self.station_1.request() as req:
            yield req  # Request access to Station_1
            yield self.env.timeout(PROCESS_1_TIME)  # Perform shared process
            # Check for failure during processing
            if random.uniform(0, 1) < self.failure_probability:
                metrics['failed_products'] += 1  # Increment failed product count
                self.failed_products.append(product)  # Track failed product
                return  # Stop further processing for failed product

            # Product-specific processing
            if product == 'Product_A':
                yield self.env.timeout(PROCESS_2_TIME)  # Exclusive process for Product_A
            elif product == 'Product_B':
                yield self.env.timeout(PROCESS_3_TIME)  # Exclusive process for Product_B

            self.successful_products.append(product)  # Track successful product

    # Simulate AGV transportation of products
    def agv_transport(self, product):
        with self.agv.request() as req:
            yield req  # Request access to AGV
            # Simulate travel time and pickup/drop-off
            yield self.env.timeout(PICKUP_DROPOFF_TIME + (AGV_DISTANCE / AGV_SPEED) +
                                   PICKUP_DROPOFF_TIME)
            metrics['agv_cycles'] += 1  # Increment AGV cycle count
            metrics['products_transported'] += 1  # Increment transported product count

    # Simulate assembly of two products
    def assembly(self, product_a, product_b):
        with self.assembly_station.request() as req:
            yield req  # Request access to the assembly station
            yield self.env.timeout(ASSEMBLY_TIME)  # Perform assembly process
            metrics['throughput'] += 1  # Increment throughput

    # Background process to simulate station failures
    def process_station_failures(self):
        while True:
            yield self.env.timeout(random.expovariate(1.0 / self.mtbf))  # Wait for next failure
            metrics['station_failures'] += 1  # Increment failure count
            yield self.env.timeout(self.repair_time)  # Simulate repair time
            metrics['total_downtime'] += self.repair_time  # Increment downtime metric

# Set up the simulation environment
random.seed(42)  # Ensure reproducibility
env = SimPy.Environment()  # Initialize simulation environment
manufacturing_system = ManufacturingSystem(env)  # Create the manufacturing system instance
```

```
# Generate product arrivals
# This process continuously creates new products and initiates processing, transportation, and assembly.
def product_arrival(env, system):
    while True:
        yield env.timeout(PROCESSING_TIME)  # Wait for next product arrival
        product = random.choice(['Product_A', 'Product_B'])  # Randomly select product type
        env.process(system.process_product(product))  # Start product processing
        env.process(system.agv_transport(product))  # Start AGV transportation
        # Start assembly if two products are available
        if len(system.successful_products) >= 2:
            env.process(system.assembly(system.successful_products.pop(0), system.successful_products.pop(0)))
        # Record metrics at each time step
        time_metrics.append({
            'time': env.now,
            'station_failures': metrics['station_failures'],
            'agv_cycles': metrics['agv_cycles'],
            'throughput': metrics['throughput'],
            'failed_products': metrics['failed_products'],
            'total_downtime': metrics['total_downtime']
        })

# Start the product arrival process
env.process(product_arrival(env, manufacturing_system))

# Run the simulation for the defined duration
env.run(until=SIMULATION_DURATION)

# This section creates time-based graphs, bar charts, and exports metrics to various formats.
# Results Visualization - Detailed Graphs
# Time-Based Graphs

# Station Failures Over Time
plt.figure()
plt.plot([m['time'] for m in time_metrics], [m['station_failures'] for m in time_metrics], label='Station Failures')
plt.xlabel('Time (minutes)')
plt.ylabel('Count')
plt.title('Station Failures Over Time')
plt.legend()
plt.savefig(os.path.join(output_dir, 'station_failures_over_time.png'))
plt.show()

# AGV Cycles Over Time
plt.figure()
plt.plot([m['time'] for m in time_metrics], [m['agv_cycles'] for m in time_metrics], label='AGV Cycles')
plt.xlabel('Time (minutes)')
plt.ylabel('Count')
plt.title('AGV Cycles Over Time')
plt.legend()
plt.savefig(os.path.join(output_dir, 'agv_cycles_over_time.png'))
plt.show()

# Throughput Over Time
plt.figure()
plt.plot([m['time'] for m in time_metrics], [m['throughput'] for m in time_metrics], label='Throughput')
plt.xlabel('Time (minutes)')
plt.ylabel('Count')
plt.title('Throughput Over Time')
plt.legend()
plt.savefig(os.path.join(output_dir, 'throughput_over_time.png'))
plt.show()

# Failed Products Over Time
plt.figure()
plt.plot([m['time'] for m in time_metrics], [m['failed_products'] for m in time_metrics], label='Failed Products')
plt.xlabel('Time (minutes)')
plt.ylabel('Count')
plt.title('Failed Products Over Time')
```

```
plt.legend()
plt.savefig(os.path.join(output_dir, 'failed_products_over_time.png'))
plt.show()

# Total Downtime Over Time
plt.figure()
plt.plot([m['time'] for m in time_metrics], [m['total_downtime'] for m in time_metrics], label='Total Downtime')
plt.xlabel('Time (minutes)')
plt.ylabel('Minutes')
plt.title('Total Downtime Over Time')
plt.legend()
plt.savefig(os.path.join(output_dir, 'total_downtime_over_time.png'))
plt.show()

# Summary Bar Charts
# Station Failures
plt.figure()
plt.bar(['Station Failures'], [metrics['station_failures']])
plt.xlabel('Failure Type')
plt.ylabel('Count')
plt.title('Total Station Failures Over 24 Hours')
plt.savefig(os.path.join(output_dir, 'station_failures.png'))
plt.show()

# AGV Cycles
plt.figure()
plt.bar(['AGV Cycles'], [metrics['agv_cycles']])
plt.xlabel('AGV Operations')
plt.ylabel('Count')
plt.title('Total AGV Cycles Over 24 Hours')
plt.savefig(os.path.join(output_dir, 'agv_cycles.png'))
plt.show()

# Throughput
plt.figure()
plt.bar(['Throughput'], [metrics['throughput']])
plt.xlabel('Metric')
plt.ylabel('Count')
plt.title('Total Throughput Over 24 Hours')
plt.savefig(os.path.join(output_dir, 'throughput.png'))
plt.show()

# Failed Products
plt.figure()
plt.bar(['Failed Products'], [metrics['failed_products']])
plt.xlabel('Product Quality')
plt.ylabel('Count')
plt.title('Total Failed Products Over 24 Hours')
plt.savefig(os.path.join(output_dir, 'failed_products.png'))
plt.show()

# Total Downtime
plt.figure()
plt.bar(['Total Downtime'], [metrics['total_downtime']])
plt.xlabel('Downtime')
plt.ylabel('Minutes')
plt.title('Total Downtime Due to Failures Over 24 Hours')
plt.savefig(os.path.join(output_dir, 'total_downtime.png'))
plt.show()

# Save Metrics to CSV
metrics_df = pd.DataFrame([metrics])
metrics_df.to_csv(os.path.join(output_dir, 'simulation_metrics.csv'), index=False)

# Save Time-Based Metrics to CSV
time_metrics_df = pd.DataFrame(time_metrics)
time_metrics_df.to_csv(os.path.join(output_dir, 'time_based_metrics.csv'), index=False)
```

```
# Save Metrics to JSON
with open(os.path.join(output_dir, 'simulation_metrics.json'), 'w') as json_file:
    json.dump(metrics, json_file, indent=4)

# Save to Excel
wb = Workbook()
ws = wb.active
ws.title = "Metrics"

for idx, (key, value) in enumerate(metrics.items(), start=1):
    ws.cell(row=idx, column=1, value=key)
    ws.cell(row=idx, column=2, value=value)

wb.save(os.path.join(output_dir, 'simulation_metrics.xlsx'))

# Summary Output
print("Simulation Complete. Results saved in the output directory.")
```

Salabim implementation code

```
# Import necessary libraries for the simulation
import salabim as sim  # For discrete-event simulation and animation.
import random  # For generating random events and probabilities.
import pandas as pd  # For organizing and saving simulation results.
import matplotlib.pyplot as plt  # For creating visualizations of simulation outcomes.
import json  # For saving results in JSON format.
import os  # For handling file and directory operations.

# Case Study: Three-Stage Manufacturing System
# This simulation models a production system with the following stages:
# 1. Pre-processing: Initial product preparation.
# 2. Main processing: Shared and exclusive workflows for Product A and Product B.
# 3. Post-processing: Manual inspection, transport via AGVs, assembly, and final output.
# The simulation tracks the movement and state of products, AGV breakdowns, queue lengths, and other performance met-
rics.

# Disable yieldless mode for better compatibility with Python
sim.yieldless(False)

# Define simulation constants
PRE_PROCESSING_TIME = 10  # Pre-processing time in minutes.
MAIN_PROCESS_PROCESS_1_TIME = 8  # Time for shared processing in Station 1.
MAIN_PROCESS_PROCESS_2_TIME = 7  # Exclusive process time for Product A.
MAIN_PROCESS_PROCESS_3_TIME = 7  # Exclusive process time for Product B.
MTBF = 300  # Mean Time Between Failures for AGVs in minutes.
REPAIR_TIME = 30  # Time required to repair AGV breakdowns in minutes.
INSPECTION_TIME = 5  # Time for manual inspection in minutes.
AGV_DISTANCE = 15  # Distance AGVs travel in meters.
AGV_SPEED = 1.5  # AGV speed in meters per second.
AGV_PICKUP_DROPOFF_TIME = 20  # Time for AGV pickup/dropoff operations in seconds.
ASSEMBLY_PROCESSING_TIME = 10  # Assembly processing time in minutes.
SIMULATION_DURATION = 1440  # Simulation duration in minutes (24 hours).
AGV_DISPATCH_PRIORITY = "FIFO"  # Dispatch priority for AGVs (e.g., FIFO, LIFO, Priority).
output_path = outputpath  # Path to save results.

# Dictionary to track simulation statistics
stats = {
    "product_A_count": 0,  # Total count of Product A processed.
    "product_B_count": 0,  # Total count of Product B processed.
    "queue_A_length": [],  # Queue length for Buffer A over time.
    "queue_B_length": [],  # Queue length for Buffer B over time.
    "agv_utilization": [],  # Utilization percentage of AGVs.
    "agv_breakdowns": 0,  # Total number of AGV breakdowns.
    "sink_product_count": 0,  # Total products reaching the final sink.
    "event_log": []  # Log of significant events (e.g., AGV breakdowns).
}
```

```
# Main simulation environment
class ProductionSimulation(sim.Component):
    def setup(self):
        # Define queues for Product A, Product B, and defective products (NOK buffer)
        self.buffer_A = sim.Queue(name='Buffer A', capacity=float('inf'))
        self.buffer_B = sim.Queue(name='Buffer B', capacity=float('inf'))
        self.nok_buffer = sim.Queue(name='NOK Buffer', capacity=float('inf'))

        # Define resources for AGVs and workers
        self.agvs = sim.Resource('AGV', capacity=3)
        self.workers = sim.Resource('Worker', capacity=21)

        # Define the sink component to track completed products
        self.sink = SinkComponent()  # Create sink for tracking products

    def process(self):
        # Generate products at regular intervals and assign them as Product A or Product B
        while True:
            product_type = random.choice(["A", "B"])  # Randomly choose product type.
            if product_type == "A":
                ProductA(buffer=self.buffer_A, nok_buffer=self.nok_buffer, agvs=self.agvs, workers=self.workers,
                         sink=self.sink).activate()
            else:
                ProductB(buffer=self.buffer_B, nok_buffer=self.nok_buffer, agvs=self.agvs, workers=self.workers,
                         sink=self.sink).activate()
            yield self.hold(random.uniform(PRE_PROCESSING_TIME * 60, PRE_PROCESSING_TIME * 60))

# Define Product A lifecycle
class ProductA(sim.Component):
    def setup(self, buffer, nok_buffer, agvs, workers, sink):
        # Initialize resources and buffers for Product A
        self.buffer = buffer
        self.nok_buffer = nok_buffer
        self.agvs = agvs
        self.workers = workers
        self.sink = sink
        self.in_buffer = False  # Track whether the product has entered a buffer.

    def process(self):
        stats["product_A_count"] += 1  # Increment Product A count.
        yield self.hold(PRE_PROCESSING_TIME * 60)  # Simulate pre-processing.
        yield self.hold(MAIN_PROCESS_PROCESS_1_TIME * 60)  # Simulate shared processing in Station 1.
        yield self.request(self.workers)  # Request a worker for inspection.
        if random.random() < 0.1:  # 10% chance of failure during inspection.
            self.enter(self.nok_buffer)  # Move to NOK buffer if inspection fails.
            yield self.hold(INSPECTION_TIME * 60)
        else:
            yield self.hold(INSPECTION_TIME * 60)  # Successful inspection.
            self.enter(self.buffer)  # Enter Buffer A for further processing.
            self.in_buffer = True
        yield self.request(self.agvs)  # Request an AGV for transport.
        agv_travel_time = (AGV_DISTANCE / AGV_SPEED) + AGV_PICKUP_DROPOFF_TIME
        yield self.hold(agv_travel_time)  # Simulate AGV travel.

        # Simulate AGV breakdowns and repairs.
        if random.random() < (1 / MTBF):
            stats["agv_breakdowns"] += 1
            stats["event_log"].append(f"AGV breakdown occurred at time {self.env.now()}")
            yield self.hold(REPAIR_TIME * 60)  # AGV breakdown
        yield self.hold(ASSEMBLY_PROCESSING_TIME * 60)  # Simulate assembly process.
        if self.in_buffer:
            self.leave(self.buffer)  ## Leave the buffer if previously entered.
        self.release(self.agvs)  # Release AGV after use.
        self.release(self.workers)  # Release worker after use.
        self.sink.enter_product()  # Move product to the sink.

# Define Product B lifecycle (inherits from Product A)
```

```
class ProductB(sim.Component):
    def setup(self, buffer, nok_buffer, agvs, workers, sink):
        self.buffer = buffer
        self.nok_buffer = nok_buffer
        self.agvs = agvs
        self.workers = workers
        self.sink = sink
        self.in_buffer = False  # Track if the product has entered the buffer

    def process(self):
        stats["product_B_count"] += 1
        yield self.hold(PRE_PROCESSING_TIME * 60)  # Pre-processing
        yield self.hold(MAIN_PROCESS_PROCESS_1_TIME * 60)  # Station 1 processing
        yield self.request(self.workers)  # Request 1 worker for manual inspection
        if random.random() < 0.1:  # 10% chance of failure
            self.enter(self.nok_buffer)  # NOK products go to NOK buffer
            yield self.hold(INSPECTION_TIME * 60)  # Manual inspection for NOK
        else:
            yield self.hold(INSPECTION_TIME * 60)  # Manual inspection for OK products
            self.enter(self.buffer)  # OK products go to respective buffer
            self.in_buffer = True  # Mark that the product has entered the buffer
        yield self.request(self.agvs)
        agv_travel_time = (AGV_DISTANCE / AGV_SPEED) + AGV_PICKUP_DROPOFF_TIME
        yield self.hold(agv_travel_time)
        if random.random() < (1 / MTBF):
            stats["agv_breakdowns"] += 1
            stats["event_log"].append(f"AGV breakdown occurred at time {self.env.now()}")
            yield self.hold(REPAIR_TIME * 60)  # AGV breakdown
        yield self.hold(ASSEMBLY_PROCESSING_TIME * 60)  # Assembly
        if self.in_buffer:
            self.leave(self.buffer)  # Leave the buffer after assembly if it was entered
        self.release(self.agvs)
        self.release(self.workers)
        self.sink.enter_product()  # Enter sink after assembly

# Define the Sink component to track completed products
class SinkComponent(sim.Component):
    def enter_product(self):
        stats["sink_product_count"] += 1  # Increment the sink product count.

# Function to run the simulation and visualize its environment
def run_simulation(run_time):
    env = sim.Environment(trace=True)  # Initialize the simulation environment.
    sim.yieldless(False)  # Disable yieldless mode
    production_simulation = ProductionSimulation()
    production_simulation.activate()  # Start the production simulation.

    # Animation parameters for visualization
    # Set up animation viewport
    env.animation_parameters(modelname='Three-Stage Manufacturing System', speed=2, background_color='90%gray')

    # Pre-processing stage
    sim.AnimateRectangle(spec=(10, 170, 110, 270), fillcolor='lightgreen', text='Source', textcolor='black', layer=1)
    sim.AnimateRectangle(spec=(150, 170, 250, 270), fillcolor='lightblue', text='Pre-Processing', textcolor='black', layer=1)

    # Station 1 with 3 working positions inside the frame
    sim.AnimateRectangle(spec=(300, 170, 400, 270), fillcolor='lightyellow', layer=1)
    sim.AnimateText(text='Station 1', x=350, y=155, textcolor='black', fontsize=10, layer=2)
    for i in range(3):
        sim.AnimateRectangle(spec=(320 + (i * 20), 190, 340 + (i * 20), 250), fillcolor='lightgray', text=f'WP{i + 1}',
                             textcolor='black', layer=2)

    # Manual inspection stations and NOK buffer (aligned on the same x-axis with different y-axis to represent parallel
paths)
    sim.AnimateRectangle(spec=(450, 270, 550, 370), fillcolor='peachpuff', text='Manual Inspection 1', textcolor='black',
                         layer=1)
    sim.AnimateRectangle(spec=(450, 70, 550, 170), fillcolor='peachpuff', text='Manual Inspection 2', textcolor='black',
```

```
                                layer=1)
    sim.AnimateQueue(queue=production_simulation.nok_buffer, x=600, y=220, title='NOK Buffer', direction='e',
                     max_length=3, id='red')

    # Buffers A and B (aligned on the same x-axis but different y-axis to represent parallel paths)
    sim.AnimateQueue(queue=production_simulation.buffer_A, x=700, y=270, title='Buffer A', direction='e',
                     max_length=3, id='lightblue')
    sim.AnimateQueue(queue=production_simulation.buffer_B, x=700, y=70, title='Buffer B', direction='e', max_length=3,
                     id='lightgreen')

    # AGVs positioned just before assembly station, moved left by 10 units
    sim.AnimateRectangle(spec=(790, 170, 820, 210), fillcolor='lightcoral', text='AGVs', textcolor='black', layer=1)

    # Post-processing stage (Assembly Station)
    sim.AnimateRectangle(spec=(830, 170, 900, 250), fillcolor='lightsalmon', text='Assembly Station', textcolor='black',
                         layer=1)

    # Sink (directly after Assembly Station with a distance of 50)
    sim.AnimateRectangle(spec=(950, 170, 1000, 250), fillcolor='lightsteelblue', text='Sink', textcolor='black', layer=1)

    # Adding arrows for material flow
    sim.AnimateLine(spec=(110, 220, 150, 220), linewidth=3, linecolor='black', layer=2)  # Arrow from Source to Pre-Pro-
cessing
    sim.AnimateLine(spec=(250, 220, 300, 220), linewidth=3, linecolor='black', layer=2)  # Arrow from Pre-Processing to
Station 1
    sim.AnimateLine(spec=(400, 220, 450, 320), linewidth=3, linecolor='black', layer=2)  # Arrow from Station 1 to Man-
ual Inspection 1
    sim.AnimateLine(spec=(400, 220, 450, 120), linewidth=3, linecolor='black', layer=2)  # Arrow from Station 1 to Man-
ual Inspection 2
    sim.AnimateLine(spec=(550, 320, 600, 220), linewidth=3, linecolor='black', layer=2)  # Arrow from Manual Inspection
1 to NOK Buffer
    sim.AnimateLine(spec=(550, 120, 600, 220), linewidth=3, linecolor='black', layer=2)  # Arrow from Manual Inspection
2 to NOK Buffer
    sim.AnimateLine(spec=(650, 220, 700, 270), linewidth=3, linecolor='black', layer=2)  # Arrow from NOK Buffer to
Buffers A and B
    sim.AnimateLine(spec=(650, 220, 700, 70), linewidth=3, linecolor='black', layer=2)  # Arrow from NOK Buffer to Buff-
ers A and B
    sim.AnimateLine(spec=(820, 220, 900, 220), linewidth=3, linecolor='black', layer=2)  # Arrow from Buffers to Assem-
bly Station
    sim.AnimateLine(spec=(900, 220, 950, 220), linewidth=3, linecolor='black', layer=2)  # Arrow from Assembly Station
to Sink

    # Detailed material flow and numbers
    sim.AnimateText(text=lambda: f'Product A Count: {stats["product_A_count"]}', x=10, y=420, textcolor='black',
                    fontsize=12, layer=2)
    sim.AnimateText(text=lambda: f'Product B Count: {stats["product_B_count"]}', x=10, y=390, textcolor='black',
                   fontsize=12, layer=2)
    sim.AnimateText(text=lambda: f'AGV Breakdowns: {stats["agv_breakdowns"]}', x=10, y=360, textcolor='black',
                   fontsize=12, layer=2)
    sim.AnimateText(text=lambda: f'Queue A Length: {len(production_simulation.buffer_A)}', x=10, y=330,
                    textcolor='black', fontsize=12, layer=2)
    sim.AnimateText(text=lambda: f'Queue B Length: {len(production_simulation.buffer_B)}', x=10, y=300,
                    textcolor='black', fontsize=12, layer=2)

    env.animate(True)  # Enable animation.
    env.run(till=run_time)  # Run the simulation.

    # Calculate AGV utilization statistics.
    agv_utilization = (stats["agv_breakdowns"] / max(1, stats["sink_product_count"])) * 100
    stats["agv_utilization"] = agv_utilization

    # Record the queue lengths at the end of the simulation
    stats["queue_A_length"] = len(production_simulation.buffer_A)
    stats["queue_B_length"] = len(production_simulation.buffer_B)

    # Output the results
    print("Simulation Results:")
    print(f"Total Product A Produced: {stats['product_A_count']}")
```

```
    print(f"Total Product B Produced: {stats['product_B_count']}")
    print(f"Total Throughput at Sink: {stats['sink_product_count']}")
    print(f"AGV Utilization: {agv_utilization:.2f}%")
    print(f"Event Log: {stats['event_log']}")

# Run simulation for a 24-hour production period (1440 minutes)
run_simulation(SIMULATION_DURATION * 60)

# Save results in CSV, Excel, and JSON formats
stats_df = pd.DataFrame.from_dict({k: [v] if not isinstance(v, list) else v for k, v in stats.items()})
os.makedirs(output_path, exist_ok=True)

# Save to CSV
csv_path = os.path.join(output_path, "production_simulation_results_salabim.csv")
stats_df.to_csv(csv_path, index=False)

# Save to Excel
excel_path = os.path.join(output_path, "production_simulation_results_salabim.xlsx")
stats_df.to_excel(excel_path, index=False)

# Save to JSON
json_path = os.path.join(output_path, "production_simulation_results_salabim.json")
with open(json_path, 'w') as json_file:
    json.dump(stats, json_file)

# Visualization of results
plt.figure(figsize=(12, 8))

# Plot Product A and B production counts
plt.subplot(2, 2, 1)
plt.bar(['Product A', 'Product B'], [stats['product_A_count'], stats['product_B_count']], color=['lightblue', 'lightgreen'])
plt.title('Total Products Produced')
plt.ylabel('Count')

# Plot average queue lengths over time
plt.subplot(2, 2, 2)
plt.plot(stats['queue_A_length'], label='Queue A Length', color='lightblue')
plt.plot(stats['queue_B_length'], label='Queue B Length', color='lightgreen')
plt.title('Queue Lengths Over Time')
plt.xlabel('Time Step')
plt.ylabel('Queue Length')
plt.legend()

# Plot AGV utilization over time
plt.subplot(2, 2, 3)
plt.plot(stats['agv_utilization'], label='AGV Utilization (%)', color='lightcoral')
plt.title('AGV Utilization Over Time')
plt.xlabel('Time Step')
plt.ylabel('Utilization (%)')
plt.legend()

plt.tight_layout()
plt.show()
# Save plots as PNG
plot_path = os.path.join(output_path, "production_simulation_results_salabim_plots.png")
plt.savefig(plot_path)
```

PyDSOL-core implementation code

```
# Import necessary libraries
import random  # For random number generation and probabilistic events.
from abc import ABC  # For defining abstract base classes.
import pandas as pd  # For saving and processing simulation results.
import matplotlib.pyplot as plt  # For generating visualizations of results.
import json  # For exporting results in JSON format.
import os  # For creating and managing file paths and directories.

# Import PyDSOL-core libraries for discrete-event simulation
from PyDSOL.core.experiment import SingleReplication  # For handling a single replication of the simulation.
from PyDSOL.core.model import DSOLModel  # Base model class for creating the simulation model.
from PyDSOL.core.simulator import DEVSSimulatorFloat  # Simulator for Discrete Event Simulation with Floating-point
precision.
from PyDSOL.core.statistics import SimCounter, SimTally, SimPersistent  # For tracking and summarizing simulation
statistics.
from PyDSOL.core.pubsub import EventProducer, EventType  # For event-driven components and event definitions.

# Case Study: Three-Stage Manufacturing System
# This simulation models a production system with the following stages:
# 1. Pre-processing: Initial product preparation.
# 2. Main processing: Shared and exclusive workflows for Product A and Product B.
# 3. Post-processing: Manual inspection, transport via AGVs, assembly, and final output.
# The simulation tracks the movement and state of products, AGV breakdowns, queue lengths, and other performance met-
rics.

# Simulation Constants
PRE_PROCESSING_TIME = 10  # Pre-processing time in minutes.
MAIN_PROCESS_PROCESS_1_TIME = 8  # Time for shared processing at Station 1.
MAIN_PROCESS_PROCESS_2_TIME = 7  # Time for exclusive processing of Product A.
MAIN_PROCESS_PROCESS_3_TIME = 7  # Time for exclusive processing of Product B.
MTBF = 300  # Mean Time Between Failures for AGVs in minutes.
REPAIR_TIME = 30  # Repair time for AGV breakdowns in minutes.
INSPECTION_TIME = 5  # Time for manual inspection in minutes.
AGV_DISTANCE = 15  # Distance covered by AGVs in meters.
AGV_SPEED = 1.5  # AGV speed in meters per second.
AGV_PICKUP_DROPOFF_TIME = 20  # Time for AGV pickup and dropoff in seconds.
ASSEMBLY_PROCESSING_TIME = 10  # Time for the assembly process in minutes.
SIMULATION_DURATION = 1440  # Total simulation duration in minutes (24 hours).
AGV_DISPATCH_PRIORITY = "FIFO"  # AGV dispatch priority; can be FIFO, LIFO, or Priority-based.

# Define a class to represent entities (products) in the system
class Entity():
    """
    Represents a product entity in the simulation, including its unique ID and type.
    """
    __glob_nr = 0  # Global counter for unique entity IDs.

    @classmethod
    def uniq_nr(cls) -> int:
        cls.__glob_nr += 1
        return cls.__glob_nr

    def __init__(self, create_time: float, product_type: str):
        self.nr = self.uniq_nr()  # Assign a unique number to the entity.
        self.create_time = create_time  # Record the time when the entity was created.
        self.product_type = product_type  # Store the product type (e.g., Product A or B).
        print(f"Entity {self} created at t={create_time}")  # Log entity creation.

    def __str__(self):
        return f"{self.product_type}-E{self.nr:02}"  # String representation of the entity.

    def __repr__(self):
        return str(self)

# Define a base class for stations in the production process
class Station(EventProducer, ABC):
```

```
    """
    Represents a generic station in the production process that produces events.
    """
    def __init__(self, simulator: DEVSSimulatorFloat):
        EventProducer.__init__(self)  # Initialize as an event producer.
        self.simulator = simulator  # Reference to the simulation environment.
        self.next: Station = None  # Pointer to the next station in the process flow.

# Define a generator station for creating entities
class Generator(Station):
    """
    Generates entities at a defined interarrival time (IAT).
    """
    GENERATED_EVENT: EventType = EventType("GENERATED_EVENT")  # Define an event type for entity genera-
tion.

    def __init__(self, simulator: DEVSSimulatorFloat, iat: float, product_type: str):
        super().__init__(simulator)
        self.iat = iat  # Interarrival time for entity generation.
        self.product_type = product_type  # Type of product generated.
        self.simulator.schedule_event_rel(self.iat, self, "generate", priority=0)  # Schedule the first generation event.

    def generate(self):
        t: float = self.simulator.simulator_time  # Current simulation time.
        entity: Entity = Entity(t, self.product_type)  # Create a new entity.
        if self.next:  # Pass the entity to the next station.
            self.next.arrive(entity)
        self.fire_timed(t, Generator.GENERATED_EVENT, 1)  # Fire a generation event.
        self.simulator.schedule_event_rel(self.iat, self, "generate", priority=0)  # Schedule the next generation.

# Define a server station for processing entities
class Server(Station):
    """
    Processes entities, maintaining a queue and tracking utilization and queue length.
    """
    BUSY_EVENT: EventType = EventType("BUSY_EVENT")
    IN_QUEUE_TIME_EVENT: EventType = EventType("IN_QUEUE_TIME_EVENT")
    QUEUE_LEN_EVENT: EventType = EventType("QUEUE_LEN_EVENT")
    PROCESS_TIME_EVENT: EventType = EventType("PROCESS_TIME_EVENT")

    def __init__(self, simulator: DEVSSimulatorFloat, pt: float):
        super().__init__(simulator)
        self.pt = pt  # Processing time for each entity.
        self.queue = []  # Queue for incoming entities.
        self.fire_queue()  # Trigger an initial queue length event.
        self.process_entity = None  # Entity currently being processed.
        self.set_busy(0)  # Server starts idle.
        self.queue_log = []  # Log of queue length over time.
        self.utilization_log = []  # Log of server utilization over time.

    def arrive(self, entity: Entity):
        t: float = self.simulator.simulator_time  # Current simulation time.
        if self.process_entity is None:  # If server is idle.
            self.process_entity = entity  # Assign entity to server.
            self.start_process_time = t  # Record the start time of processing.
            self.set_busy(1)  # Mark server as busy.
            self.simulator.schedule_event_rel(self.pt, self, "depart", priority=0)  # Schedule departure event.
            print(f"Entity {entity} processing started at {t}, queue={self.queue}")
        else:  # If server is busy.
            self.queue.append((t, entity))  # Add entity to the queue.
            self.fire_queue()  # Update queue length event.
            print(f"Entity {entity} in queue at {t}, queue={self.queue}")
        self.queue_log.append((t, len(self.queue)))  # Log queue length.

    def depart(self):
        t: float = self.simulator.simulator_time  # Current simulation time.
        ready_entity: Entity = self.process_entity  # Get processed entity.
        busytime: float = t - self.start_process_time  # Calculate processing time.
```

```
        self.fire_timed(t, Server.PROCESS_TIME_EVENT, busytime)  # Fire processing time event.
        print(f"Entity {ready_entity} ready processing at {t}, busytime={busytime}, queue={self.queue}")
        if self.next:  # Pass entity to the next station.
            self.next.arrive(ready_entity)
        if len(self.queue) == 0:  # If queue is empty.
            self.set_busy(0)  # Mark server as idle.
            self.process_entity = None
        else:  # If queue is not empty.
            entry = self.queue.pop(0)  # Remove next entity from queue.
            self.fire_queue()  # Update queue length event.
            inqueuetime: float = t - entry[0]  # Calculate queue time.
            self.fire_timed(t, Server.IN_QUEUE_TIME_EVENT, inqueuetime)  # Fire queue time event.
            entity: Entity = entry[1]
            print(f"Entity {entity} left the queue at {t}, queue time={inqueuetime}")
            self.process_entity = entity  # Assign next entity to server.
            self.start_process_time = t
            self.set_busy(1)  # Mark server as busy.
            self.simulator.schedule_event_rel(self.pt, self, "depart", priority=0)  # Schedule next departure.

    def fire_queue(self):
        t: float = self.simulator.simulator_time  # Current simulation time.
        self.fire_timed(t, Server.QUEUE_LEN_EVENT, len(self.queue))  # Trigger queue length event.

    def set_busy(self, busy):
        self.busy = busy  # Update server status (busy or idle).
        self.fire_timed(self.simulator.simulator_time, Server.BUSY_EVENT, busy)  # Trigger busy status event.

class Buffer(Station):
    """
    Represents a buffer in the system for storing entities temporarily.
    """
    def __init__(self, simulator: DEVSSimulatorFloat, buffer_name: str):
        """
        Initialize the buffer with a given name.
        :param simulator: The simulation environment.
        :param buffer_name: The name of the buffer for identification.
        """
        super().__init__(simulator)
        self.buffer_name = buffer_name  # Name of the buffer.
        self.queue = []  # Queue to hold the entities in the buffer.

    def arrive(self, entity: Entity):
        """
        Handles an entity's arrival at the buffer.
        :param entity: The entity arriving at the buffer.
        """
        t: float = self.simulator.simulator_time  # Current simulation time.
        self.queue.append(entity)  # Add the entity to the buffer's queue.
        print(f"Entity {entity} stored in {self.buffer_name} at t={t}")  # Log the arrival.

class AssemblyStation(Station):
    """
    Represents an assembly station where products are assembled from components in buffers.
    """

    def __init__(self, simulator: DEVSSimulatorFloat, pt: float, buffer_a: Buffer, buffer_b: Buffer):
        """
        Initialize the assembly station with processing time and buffers.
        :param simulator: The simulation environment.
        :param pt: Processing time for assembly.
        :param buffer_a: Buffer for Product A components.
        :param buffer_b: Buffer for Product B components.
        """
        super().__init__(simulator)
        self.pt = pt  # Processing time for assembly.
        self.buffer_a = buffer_a  # Buffer for Product A components.
        self.buffer_b = buffer_b  # Buffer for Product B components.
```

```
    def try_assemble_order(self):
        """
        Attempt to assemble an order if enough components are available in the buffers.
        """
        t: float = self.simulator.simulator_time  # Current simulation time.
        if len(self.buffer_a.queue) >= 2 and len(self.buffer_b.queue) >= 1:
            # If sufficient components are available, assemble the order.
            products = [self.buffer_a.queue.pop(0), self.buffer_a.queue.pop(0),  self.buffer_b.queue.pop(0)]
            print(f"Order assembled at t={t}: {products}")  # Log the assembly.
            self.simulator.schedule_event_rel(self.pt, self, "finish_assembly", products=products)  # Schedule assembly com-
pletion.

    def finish_assembly(self, products):
        """
        Complete the assembly process.
        :param products: The products used in the assembly.
        """
        t: float = self.simulator.simulator_time  # Current simulation time.
        print(f"Order completed at t={t}: {products}")  # Log the completion of assembly.
class Sink(Station):
    """
    Represents the sink station where completed entities leave the system.
    """
    SYS_TIME_EVENT: EventType = EventType("SYS_TIME_EVENT")  # Event for tracking system time of entities.
    THROUGHPUT_EVENT: EventType = EventType("THROUGHPUT_EVENT")  # Event for tracking throughput.

    def __init__(self, simulator: DEVSSimulatorFloat):
        """
        Initialize the sink station.
        :param simulator: The simulation environment.
        """
        super().__init__(simulator)
        self.throughput_log = []  # Log for throughput over time.

    def arrive(self, entity: Entity):
        """
        Handles an entity's arrival at the sink.
        :param entity: The entity arriving at the sink.
        """
        t: float = self.simulator.simulator_time  # Current simulation time.
        time_in_system: float = t - entity.create_time  # Calculate time spent in the system.
        self.fire_timed(t, Sink.SYS_TIME_EVENT, time_in_system)  # Log the system time event.
        print(f"Entity {entity} leaves at t={t}, systime={time_in_system}")  # Log the departure.

        # Log throughput and trigger the throughput event.
        self.fire_timed(t, Sink.THROUGHPUT_EVENT, 1)
        self.throughput_log.append((t, 1))  # Append throughput data to the log.

class QueuingModel(DSOLModel):
    def __init__(self, simulator: DEVSSimulatorFloat):
        super().__init__(simulator)

    def construct_model(self):
        generator_a = Generator(self.simulator, 10, "Product_A")
        generator_b = Generator(self.simulator, 12, "Product_B")
        self.server = Server(self.simulator, 8)
        self.buffer_a = Buffer(self.simulator, "Buffer_A")
        self.buffer_b = Buffer(self.simulator, "Buffer_B")
        self.assembly_station = AssemblyStation(self.simulator, ASSEMBLY_PROCESSING_TIME, self.buffer_a,
self.buffer_b)
        self.sink = Sink(self.simulator)

        generator_a.next = self.server
        generator_b.next = self.server
        self.server.next = self.sink

        # Set counters and statistics
        self.gen_counter_a = SimCounter("generator_a.nr", "number of generated entities - Product A", self.simulator)
```

```
        self.gen_counter_a.listen_to(generator_a, Generator.GENERATED_EVENT)
        self.gen_counter_b = SimCounter("generator_b.nr", "number of generated entities - Product B", self.simulator)
        self.gen_counter_b.listen_to(generator_b, Generator.GENERATED_EVENT)
        self.tiq_tally = SimTally("server.time_in_queue", "server time in queue of entities", self.simulator)
        self.tiq_tally.listen_to(self.server, Server.IN_QUEUE_TIME_EVENT)
        self.tis_tally = SimTally("entity.time_in_sys", "entity time in system", self.simulator)
        self.tis_tally.listen_to(self.sink, Sink.SYS_TIME_EVENT)
        self.sql_persistent = SimPersistent("server.queue_len", "server queue length", self.simulator)
        self.sql_persistent.listen_to(self.server, Server.QUEUE_LEN_EVENT)
        self.sut_persistent = SimPersistent("server.util", "server time-weighted utilization", self.simulator)
        self.sut_persistent.listen_to(self.server, Server.BUSY_EVENT)


class ManufacturingSimulation():
    def __init__(self):
        self.simulator = DEVSSimulatorFloat("manufacturing_simulation")
        self.model = QueuingModel(self.simulator)
        self.model.construct_model()
        self.replication = SingleReplication("rep", 0.0, 0.0, SIMULATION_DURATION)
        self.simulator.initialize(self.model, self.replication)
        self.simulator.start()
        self.save_results()
        self.generate_graphs()

    def save_results(self):
        # Save results to Excel, CSV, and JSON files
        results = {
            "Total Generated Product A": self.model.gen_counter_a.count,
            "Total Generated Product B": self.model.gen_counter_b.count,
            "Average Time in Queue": self.model.tiq_tally.mean,
            "Average Time in System": self.model.tis_tally.mean,
            "Server Queue Length": self.model.sql_persistent.weighted_mean(),
            "Server Utilization": self.model.sut_persistent.weighted_mean(),
        }

        # Convert results to DataFrame for easy saving
        df = pd.DataFrame([results])

        # Save as Excel, CSV, and JSON
        output_dir = outputpath # change it with outputpath
        if not os.path.exists(output_dir):
            os.makedirs(output_dir)

        df.to_excel(os.path.join(output_dir, "manufacturing_results.xlsx"), index=False)
        df.to_csv(os.path.join(output_dir, "manufacturing_results.csv"), index=False)
        with open(os.path.join(output_dir, "manufacturing_results.json"), "w") as json_file:
            json.dump(results, json_file, indent=4, default=str)

    def generate_graphs(self):
        # Generate graphs for queue length, server utilization, and throughput
        server = self.model.server
        sink = self.model.sink

        if len(server.queue_log) > 0:
            times_queue, queue_lengths = zip(*server.queue_log)
            # Plot queue length over time
            plt.figure()
            plt.plot(times_queue, queue_lengths, label="Queue Length")
            plt.xlabel("Time (minutes)")
            plt.ylabel("Queue Length")
            plt.title("Queue Length Over Time")
            plt.legend()
            plt.grid()
            plt.savefig(os.path.join(outputpath)) # change it with outputpath
        else:
            print("No queue data available for plotting.")

        if len(server.utilization_log) > 0:
```

```
        times_util, utilizations = zip(*server.utilization_log)
        # Plot server utilization over time
        plt.figure()
        plt.plot(times_util, utilizations, label="Server Utilization")
        plt.xlabel("Time (minutes)")
        plt.ylabel("Utilization")
        plt.title("Server Utilization Over Time")
        plt.legend()
        plt.grid()
        plt.savefig(os.path.join(outputpath)) # change it with outputpath
    else:
        print("No utilization data available for plotting.")

    if len(sink.throughput_log) > 0:
        times_throughput, throughput_values = zip(*sink.throughput_log)
        cumulative_throughput = [sum(throughput_values[:i + 1]) for i in range(len(throughput_values))]
        # Plot throughput over time
        plt.figure()
        plt.plot(times_throughput, cumulative_throughput, label="Throughput")
        plt.xlabel("Time (minutes)")
        plt.ylabel("Cumulative Throughput")
        plt.title("Throughput Over Time")
        plt.legend()
        plt.grid()
        plt.savefig(outputpath)) # change it with outputpath
    else:
        print("No throughput data available for plotting.")


if __name__ == "__main__":
    ManufacturingSimulation()
```

Akatosh implementation code

```
# Import necessary libraries
import asyncio  # For running asynchronous tasks.
import csv  # For saving simulation results in CSV format.
import json  # For saving simulation results in JSON format.
import logging  # For logging simulation events.
import random  # For random number generation.
import pandas as pd  # For handling data and saving results to Excel.
from Akatosh.universe import Universe  # Simulation framework for creating and managing the universe.
from Akatosh.event import Event  # Handles simulation events.
from Akatosh.entity import Entity  # Represents entities (e.g., products) in the simulation.
from Akatosh.resource import Resource  # Manages resources like buffers, humans, AGVs.
import matplotlib.pyplot as plt  # For generating plots.

# Case Study: Three-Stage Manufacturing System
# This simulation models a production system with the following stages:
# 1. Pre-processing: Initial product preparation.
# 2. Main processing: Shared and exclusive workflows for Product A and Product B.
# 3. Post-processing: Manual inspection, transport via AGVs, assembly, and final output.
# The simulation tracks the movement and state of products, AGV breakdowns, queue lengths, and other performance met-
rics.

# Constants and parameters for the simulation (all time in minutes unless specified otherwise)
SIMULATION_DURATION = 24 * 60  # Simulation duration of 24 hours.
PRODUCT_ARRIVAL_INTERVAL = 12  # Interval between product arrivals.
PRE_PROCESSING_TIME = 10  # Time required for pre-processing stage.
PROCESS_1_TIME = 8  # Common processing time for all products.
PROCESS_2_TIME = 7  # Additional processing time for Product_A.
PROCESS_3_TIME = 7  # Additional processing time for Product_B.
MTBF = 300  # Mean Time Between Failures for Station_1.
REPAIR_TIME = 30  # Time required to repair Station_1 after a failure.
INSPECTION_TIME = 5  # Time required for manual inspection of products.
PICKUP_DROPOFF_TIME = 20 / 60  # AGV pickup/drop-off time in minutes.
```

```
ASSEMBLY_TIME = 10  # Time required for assembling products.
BUFFER_CAPACITY = 50  # Maximum capacity of buffers.

# Initialize the simulation universe
universe = Universe()
universe.disable_realtime()  # Disable real-time mode for faster simulation.
universe._time_resolution = 3  # Set simulation time resolution.
universe._time_step = 1  # Set simulation time step.

# Define resources
agv = Resource(capacity=1)  # Automatic Guided Vehicle (AGV) with a single capacity.
human = Resource(capacity=1)  # Single human resource for manual inspection.

# Define buffers with specified capacity
buffer_a = Resource(capacity=BUFFER_CAPACITY)  # Buffer for Product_A.
buffer_b = Resource(capacity=BUFFER_CAPACITY)  # Buffer for Product_B.
nok_buffer = Resource(capacity=BUFFER_CAPACITY)  # Buffer for defective (NOK) products.

# Initialize data structures for tracking metrics and logs
event_log = []  # List to record simulation events.
total_assembled_orders = 0  # Counter for total assembled orders.
station_failures_count = 0  # Counter for total station failures.
queue_lengths = {  # Dictionary to store queue lengths at various stages over time.
    "pre-processing": [],
    "station_1": [],
    "manual_inspection": [],
    "buffer_a": [],
    "buffer_b": []
}
station_failures_over_time = []  # List to record times of station failures.
assembly_throughput_times = []  # List to record times of completed assemblies.

# Function to log events and update queue lengths
def log_event(event_type, time, details="", important=True):
    """
    Logs a simulation event and records queue lengths.
    :param event_type: The type of event (e.g., "Station Failure").
    :param time: The time of the event in the simulation.
    :param details: Additional details about the event.
    :param important: Whether the event should be logged as significant.
    """
     if important:
        event_log.append({"event": event_type, "time": time, "details": details})  # Add event to log.
        logging.info(f"{event_type} at time {time}: {details}")  # Log the event.
    # Update queue length metrics.
    record_queue_lengths(time)

def record_queue_lengths(current_time):
    """
    Records the lengths of all queues at the current simulation time.
    """
    queue_lengths["pre-processing"].append((current_time, max(queue_lengths["pre-processing"][-1][1], 0)
if queue_lengths["pre-processing"] else 0))
    queue_lengths["station_1"].append((current_time, max(queue_lengths["station_1"][-1][1], 0)
if queue_lengths["station_1"] else 0))
    queue_lengths["manual_inspection"].append((current_time, max(queue_lengths["manual_inspection"][-1][1], 0)
if queue_lengths["manual_inspection"] else 0))
    queue_lengths["buffer_a"].append((current_time, buffer_a.level))
    queue_lengths["buffer_b"].append((current_time, buffer_b.level))

# Define the Source class to generate products
class Source:
    def __init__(self):
        """
        Initialize the product source and start generating products.
        """
        self.product_count = 0  # Counter for the total number of products created.
        self.create_product_event()  # Schedule the first product generation event.
```

```
    def create_product_event(self):
        """
        Schedule the next product generation event.
        """
        Event(universe.time, universe.time, action=self.generate_product, label="Product Generation", once=True)
        self.product_count += 1  # Increment the product counter.
        # Schedule the next product generation if within simulation duration.
        if universe.time + PRODUCT_ARRIVAL_INTERVAL < SIMULATION_DURATION:
            Event(universe.time + PRODUCT_ARRIVAL_INTERVAL, universe.time
                    + PRODUCT_ARRIVAL_INTERVAL, action=self.create_product_event, label="Next Product Genera-
                     tion", once=True)

    def generate_product(self):
        """
        Generate a new product and place it in the pre-processing queue.
        """
        product_type = "Product_A" if self.product_count % 2 == 0 else "Product_B"  # Alternate between Product_A and
Product_B.
        product = Entity(universe.time, SIMULATION_DURATION, label=f"{product_type}_{self.product_count}")  #
Create a new entity.
        queue_lengths["pre-processing"].append((universe.time, queue_lengths["pre-processing"][-1][1] + 1
if queue_lengths["pre-processing"] else 1))  # Update pre-processing queue length.
        log_event(f"{product_type} Created", universe.time, f"{product.label} added to pre-processing queue")  # Log the
creation.
        PreProcessing(product)  # Start pre-processing for the product.
# Pre-processing stage
class PreProcessing:
    """
    Handles the pre-processing stage for a product.
    """
    def __init__(self, product):
        # Schedule the pre-processing event for the product.
        Event(universe.time, universe.time + PRE_PROCESSING_TIME, action=self.process_product,
            label=f"Pre-Processing_{product.label}")
        self.product = product

    def process_product(self):
        # Update the queue length for pre-processing and log the event.
        queue_lengths["pre-processing"][-1] = (universe.time, max(queue_lengths["pre-processing"][-1][1] - 1, 0))
        log_event(f"Pre-Processing Complete", universe.time, f"{self.product.label} transferred to Station_1")
        # Transfer the product to Station_1 for further processing.
        Station1(self.product)

# Main processing - Station 1
class Station1:
    """
    Handles processing of products at Station 1, including potential failures.
    """
    def __init__(self, product):
        self.product = product
        # Update the queue length for Station_1 and determine the processing time.
        queue_lengths["station_1"].append((universe.time, queue_lengths["station_1"][-1][1] + 1
if queue_lengths["station_1"] else 1))
        process_time = PROCESS_1_TIME + (PROCESS_2_TIME if "Product_A" in product.label else PRO-
CESS_3_TIME)

        if self.check_for_failure():

            # If a failure occurs, schedule repair and resume processing.
            Event(universe.time + REPAIR_TIME, universe.time + REPAIR_TIME, action=self.complete_processing,
                label=f"Repair and Resume Processing_{product.label}")
        else:
            # If no failure, proceed with regular processing.
            Event(universe.time, universe.time + process_time, action=self.complete_processing,
                label=f"Station_1 Processing_{product.label}")

    def check_for_failure(self):
```

```
        """
        Checks if a failure occurs during processing and handles it.
        """
        global station_failures_count
        failure_probability = 1 / MTBF
        if random.random() < failure_probability:
            # Log the failure event and update NOK buffer.
            log_event("Station Failure", universe.time, f"{self.product.label} failed and sent to NOK Buffer", important=True)
            nok_buffer.distribute(self.product, 1)
            station_failures_count += 1
            station_failures_over_time.append(universe.time)
            return True
        return False

    def complete_processing(self):
        """
        Completes the processing of the product and moves it to the next stage.
        """
        queue_lengths["station_1"][-1] = (universe.time, max(queue_lengths["station_1"][-1][1] - 1, 0))
        if "NOK Buffer" not in self.product.label:
            log_event("Station_1 Processing Complete", universe.time, f"{self.product.label} moved to Manual Inspection")
            ManualInspection(self.product)

# Manual inspection stage
class ManualInspection:
    """
    Handles the manual inspection stage for a product.
    """
    def __init__(self, product):
        self.product = product
        # Update the queue length for manual inspection and check resource availability.
        queue_lengths["manual_inspection"].append((universe.time, queue_lengths["manual_inspection"][-1][1] + 1
if queue_lengths["manual_inspection"] else 1))
        if human.level > 0:
            human.distribute(self.product, 1)
            # Schedule the inspection event.
            Event(universe.time, universe.time + INSPECTION_TIME, action=self.inspect_product,
                label=f"Manual Inspection {product.label}")

    def inspect_product(self):
        """
        Completes the inspection and moves the product to the appropriate buffer.
        """
        human.collect(self.product)
        queue_lengths["manual_inspection"][-1] = (universe.time, max(queue_lengths["manual_inspection"][-1][1] - 1, 0))
        if "Product_A" in self.product.label:
            # Move Product_A to Buffer_A.
            buffer_a.distribute(self.product, 1)
            log_event("Manual Inspection Complete", universe.time, f"{self.product.label} moved to Buffer_A")
        else:
            # Move Product_B to Buffer_B.
            buffer_b.distribute(self.product, 1)
            log_event("Manual Inspection Complete", universe.time, f"{self.product.label} moved to Buffer_B")
        Assembly()

# Assembly stage
class Assembly:
    """
    Handles the assembly process where products from buffers are combined.
    """
    def __init__(self):
        global total_assembled_orders
        # Check if required products are available in buffers.
        if buffer_a.level > 0 and buffer_b.level > 0:
            buffer_a.collect(None, 1)
            buffer_b.collect(None, 1)
            total_assembled_orders += 1
            # Schedule the assembly completion event.
```

```
        Event(universe.time, universe.time + ASSEMBLY_TIME, action=self.complete_assembly, label="Assembly Pro-
cess")

    def complete_assembly(self):
        """
        Completes the assembly process and logs the event.
        """
        log_event("Assembly Complete", universe.time, f"Order assembled and ready for dispatch")
        assembly_throughput_times.append(universe.time)

# Instantiate Source
source = Source()

# Run the simulation
logging.basicConfig(level=logging.INFO)
asyncio.run(universe.simulate(SIMULATION_DURATION))

# Log total throughput
log_event("Total Throughput", SIMULATION_DURATION, f"Total products assembled and sent to sink: {total_assem-
bled_orders}")

# Generate graphs for metrics
def generate_graphs():
    # Queue lengths over time
    plt.figure(figsize=(12, 6))
    for key, values in queue_lengths.items():
        if values:
            times, lengths = zip(*values)
            plt.plot(times, lengths, label=key)
    plt.xlabel('Time (minutes)')
    plt.ylabel('Queue Length')
    plt.title('Queue Lengths Over Time')
    plt.legend()
    plt.grid(True)
    plt.savefig(r'C:\Users\aksha\OneDrive\Desktop\Master Work\Codes\Akatosh\queue_lengths.png')
    plt.close()

    # Assembly throughput over time
    plt.figure(figsize=(12, 6))
    plt.hist(assembly_throughput_times, bins=20)
    plt.xlabel('Time (minutes)')
    plt.ylabel('Orders Assembled')
    plt.title('Assembly Throughput Over Time')
    plt.grid(True)
    plt.savefig(r'C:\Users\aksha\OneDrive\Desktop\Master Work\Codes\Akatosh\assembly_throughput.png')
    plt.close()

generate_graphs()

# Save results to CSV, JSON, and Excel formats
results = {
    "total_assembled_orders": total_assembled_orders,
    "station_failures_count": station_failures_count,
    "queue_lengths": queue_lengths,
    "station_failures_over_time": station_failures_over_time,
    "assembly_throughput_times": assembly_throughput_times
}

# Save results to CSV
csv_filename = r'C:\Users\aksha\OneDrive\Desktop\Master Work\Codes\Akatosh\simulation_results.csv'
with open(csv_filename, 'w', newline='') as csvfile:
    fieldnames = ['Metric', 'Value']
    writer = csv.DictWriter(csvfile, fieldnames=fieldnames)

    writer.writeheader()
    writer.writerow({'Metric': 'Total Assembled Orders', 'Value': total_assembled_orders})
    writer.writerow({'Metric': 'Station Failures Count', 'Value': station_failures_count})
    writer.writerow(
```

```
        {'Metric': 'Queue Lengths', 'Value': json.dumps(queue_lengths)})  # Serializing queue lengths as JSON
    writer.writerow({'Metric': 'Station Failures Over Time', 'Value': station_failures_over_time})
    writer.writerow({'Metric': 'Assembly Throughput Times', 'Value': assembly_throughput_times})

# Save results to JSON
json_filename = r'C:\Users\aksha\OneDrive\Desktop\Master Work\Codes\Akatosh\simulation_results.json'
with open(json_filename, 'w') as jsonfile:
    json.dump(results, jsonfile, indent=4)

# Save results to Excel
excel_filename = r'C:\Users\aksha\OneDrive\Desktop\Master Work\Codes\Akatosh\simulation_results.xlsx'
df_results = pd.DataFrame({
    'Metric': ['Total Assembled Orders', 'Station Failures Count', 'Queue Lengths', 'Station Failures Over Time',
               'Assembly Throughput Times'],
    'Value': [
        total_assembled_orders,
        station_failures_count,
        json.dumps(queue_lengths),  # Serializing queue lengths as JSON-like string
        str(station_failures_over_time),
        str(assembly_throughput_times)
    ]
})
df_results.to_excel(excel_filename, index=False)

print("Simulation results saved in CSV, JSON, and Excel formats.")

# Inform about the total throughput
print(f"Total Throughput: {total_assembled_orders} orders assembled")
```

HADES-framework implementation code

```
# Import necessary libraries
import asyncio  # For asynchronous programming
import pandas as pd  # For saving results in structured formats
import json  # For saving results in JSON format
import matplotlib.pyplot as plt  # For generating graphs
import os  # For handling file directories
from datetime import datetime  # For timestamping results
from enum import Enum  # For defining event categories
from HADES import Event, HADES, NotificationResponse, Process, RandomProcess, SimulationStarted, Simula-
tionEnded

# Case Study: Three-Stage Manufacturing System
# This simulation models a production system with the following stages:
# 1. Pre-processing: Initial product preparation.
# 2. Main processing: Shared and exclusive workflows for Product A and Product B.
# 3. Post-processing: Manual inspection, transport via AGVs, assembly, and final output.
# The simulation tracks the movement and state of products, AGV breakdowns, queue lengths, and other performance met-
rics.

# Input data for simulation configuration
PRE_PROCESSING_TIME = 10  # Pre-processing time in minutes
MAIN_PROCESS_PROCESS_1_TIME = 8  # Time for main process 1 in minutes
MAIN_PROCESS_PROCESS_2_TIME = 7  # Time for main process 2 in minutes
MAIN_PROCESS_PROCESS_3_TIME = 7  # Time for main process 3 in minutes
MTBF = 300  # Mean Time Between Failures (in minutes)
REPAIR_TIME = 30  # Time to repair a failure (in minutes)
INSPECTION_TIME = 5  # Inspection time for manual checks (in minutes)
AGV_DISTANCE = 15  # Distance covered by AGV in meters
AGV_SPEED = 1.5  # AGV speed in meters per second
AGV_PICKUP_DROPOFF_TIME = 20  # Pickup and drop-off time for AGV (in seconds)
ASSEMBLY_PROCESSING_TIME = 10  # Time for assembly processing (in minutes)
SIMULATION_DURATION = 1440  # Total simulation time in minutes (24 hours)
AGV_DISPATCH_PRIORITY = "FIFO"  # Dispatch priority for AGV (FIFO, LIFO, or Priority-based)
```

```python
# Define simulation events for tracking and processing
class PreProcessingStarted(Event):
    """Event to indicate the start of pre-processing."""
    pass

class ProductPreProcessed(Event):
    """Event to indicate a product has completed pre-processing."""
    product_type: str  # Specifies the type of product (e.g., Product_A, Product_B)

class StationFailure(Event):
    """Event to indicate a station has failed."""
    pass

class RepairCompleted(Event):
    """Event to indicate a repair has been completed."""
    pass

class InspectionCompleted(Event):
    """Event to indicate manual inspection has been completed."""
    product_type: str  # Type of product inspected
    status: str  # Inspection result: OK or NOK

class AGVTransportStarted(Event):
    """Event to indicate AGV has started transporting a product."""
    product_type: str  # Type of product being transported

class ProductAssembled(Event):
    """Event to indicate a product has been assembled."""
    pass

class FinalCheckCompleted(Event):
    """Event to indicate the final quality check has been completed."""
    status: str  # Final check result: OK or NOK

# Directory for saving results
SAVE_DIR = (outputpath)) # change it with outputpath
os.makedirs(SAVE_DIR, exist_ok=True)

# Data structure for storing simulation results
results = {
    "events": [],  # Log of all events that occur during the simulation
    "statistics": {
        "failures": 0,  # Count of station failures
        "repairs": 0,  # Count of completed repairs
        "inspections_passed": 0,  # Count of inspections that passed
        "inspections_failed": 0,  # Count of inspections that failed
        "products_assembled": 0,  # Total number of products assembled
        "agv_transports": 0,  # Total AGV transport operations
        "final_checks_passed": 0,  # Products that passed the final check
        "final_checks_failed": 0  # Products that failed the final check
    }
}

# Define simulation processes
class PreProcessing(Process):
    """Handles pre-processing of products."""
    async def notify(self, event: Event):
        if isinstance(event, SimulationStarted):
            # Start the pre-processing cycle
            t = event.t
            self.add_event(PreProcessingStarted(t=t + 1))
            return NotificationResponse.ACK
        elif isinstance(event, PreProcessingStarted):
            # Generate Product_A and Product_B periodically
            t = event.t
            print(f"Pre-processing started at time {t}")
            self.add_event(ProductPreProcessed(t=t + PRE_PROCESSING_TIME, product_type="Product_A"))
            self.add_event(ProductPreProcessed(t=t + PRE_PROCESSING_TIME + 5, product_type="Product_B"))
```

```
            results["events"].append({"time": t, "event": "Pre-processing started"})
            self.add_event(PreProcessingStarted(t=t + 30))  # Schedule next cycle
            return NotificationResponse.ACK
        return NotificationResponse.NO_ACK

class MainProcessing(RandomProcess):
    """Handles the main processing stage, including station failures."""
    def __init__(self, seed):
        super().__init__(seed)
        self.failed = False  # Tracks if the station is currently failed

    async def notify(self, event: Event):
        if isinstance(event, ProductPreProcessed):
            # Handle processing for pre-processed products
            t = event.t
            product_type = event.product_type
            if self.failed:
                # Station failure prevents processing
                print(f"Station failure at time {t}, cannot process {product_type}")
                results["events"].append({"time": t, "event": "Station failure", "product_type": product_type})
                return NotificationResponse.ACK_BUT_IGNORED
            process_time = MAIN_PROCESS_PROCESS_1_TIME if product_type == "Product_A" else MAIN_PRO-
CESS_PROCESS_2_TIME
            if self.random.random() < 0.02:  # 2% chance of failure
                self.failed = True
                self.add_event(StationFailure(t=t + 1))
            else:
                self.add_event(InspectionCompleted(t=t + process_time, product_type=product_type, status="OK"))
            return NotificationResponse.ACK
        elif isinstance(event, StationFailure):
            # Handle station failure and schedule repair
            t = event.t
            print(f"Station failed at time {t}, starting repairs")
            results["events"].append({"time": t, "event": "Station failure"})
            results["statistics"]["failures"] += 1
            self.add_event(RepairCompleted(t=t + REPAIR_TIME))
            return NotificationResponse.ACK
        elif isinstance(event, RepairCompleted):
            # Mark the station as operational after repair
            t = event.t
            print(f"Repairs completed at time {t}, resuming operations")
            results["events"].append({"time": t, "event": "Repairs completed"})
            results["statistics"]["repairs"] += 1
            self.failed = False
            return NotificationResponse.ACK
        return NotificationResponse.NO_ACK

class ManualInspection(Process):
    """Handles the manual inspection of products."""
    async def notify(self, event: Event):
        if isinstance(event, InspectionCompleted):
            t = event.t
            product_type = event.product_type
            status = event.status
            if status == "NOK":
                print(f"{product_type} failed inspection at time {t}")
                results["events"].append({"time": t, "event": "Inspection failed", "product_type": product_type})
                results["statistics"]["inspections_failed"] += 1
            else:
                print(f"{product_type} passed inspection at time {t}")
                results["events"].append({"time": t, "event": "Inspection passed", "product_type": product_type})
                results["statistics"]["inspections_passed"] += 1
            return NotificationResponse.ACK
        return NotificationResponse.NO_ACK
# AGV Transport process - Handles transporting products using AGV
class AGVTransport(Process):
    async def notify(self, event: Event):
        # Check if the event is an inspection completion and the product status is OK
```

```
        if isinstance(event, InspectionCompleted) and event.status == "OK":
            t = event.t  # Get the event time
            product_type = event.product_type  # Get the product type
            print(f"AGV transporting {product_type} from buffer to assembly at time {t}")
            distance = AGV_DISTANCE  # Distance to be traveled (meters)
            speed = AGV_SPEED  # AGV speed (meters per second)
            transport_time = distance / speed  # Calculate transport time
            # Schedule an event for AGV transport start
            self.add_event(AGVTransportStarted(t=t + transport_time, product_type=product_type))
            # Log the transport operation
            results["events"].append({"time": t, "event": "AGV transport started", "product_type": product_type})
            results["statistics"]["agv_transports"] += 1  # Increment transport count
            return NotificationResponse.ACK
        return NotificationResponse.NO_ACK

# Assembly station process - Handles assembly of products
class AssemblyStation(Process):
    async def notify(self, event: Event):
        # Check if the event is an AGV transport start
        if isinstance(event, AGVTransportStarted):
            t = event.t  # Get the event time
            product_type = event.product_type  # Get the product type
            print(f"Assembly started for {product_type} at time {t}")
            # Schedule an event for product assembly completion
            self.add_event(ProductAssembled(t=t + ASSEMBLY_PROCESSING_TIME))
            return NotificationResponse.ACK
        elif isinstance(event, ProductAssembled):
            t = event.t  # Get the event time
            print(f"Product assembled at time {t}, performing final checks")
            # Schedule an event for final check completion
            self.add_event(FinalCheckCompleted(t=t + INSPECTION_TIME, status="OK"))
            # Log the assembly operation
            results["events"].append({"time": t, "event": "Product assembled"})
            results["statistics"]["products_assembled"] += 1  # Increment assembly count
            return NotificationResponse.ACK
        return NotificationResponse.NO_ACK

# Final check process - Handles quality checks for assembled products
class FinalCheck(Process):
    async def notify(self, event: Event):
        # Check if the event is a final check completion
        if isinstance(event, FinalCheckCompleted):
            t = event.t  # Get the event time
            status = event.status  # Get the final check status (OK/NOK)
            if status == "OK":
                print(f"Product passed final check at time {t}, ready for dispatch")
                # Log the successful final check
                results["events"].append({"time": t, "event": "Final check passed"})
                results["statistics"]["final_checks_passed"] += 1  # Increment pass count
            else:
                print(f"Product failed final check at time {t}, discarded")
                # Log the failed final check
                results["events"].append({"time": t, "event": "Final check failed"})
                results["statistics"]["final_checks_failed"] += 1  # Increment fail count
            return NotificationResponse.ACK
        return NotificationResponse.NO_ACK

# Custom arrival process - Manages product arrival schedules
class CustomArrivalProcess(RandomProcess):
    def __init__(self, seed):
        super().__init__(seed)  # Initialize with a random seed

    async def notify(self, event: Event):
        if isinstance(event, SimulationStarted):
            t = event.t  # Get the event time
            # Schedule arrivals for Product_A and Product_B with random intervals
            self.add_event(ProductPreProcessed(t=t + self.random.randint(1, 5), product_type="Product_A"))
            self.add_event(ProductPreProcessed(t=t + self.random.randint(5, 10), product_type="Product_B"))
```

```
            # Continuously add new arrivals to extend the simulation
            self.add_event(SimulationStarted(t=t + 30))
            return NotificationResponse.ACK
        return NotificationResponse.NO_ACK

# Queue management process - Handles product queues (FIFO, LIFO)
class QueueManagement(Process):
    async def notify(self, event: Event):
        if isinstance(event, ProductPreProcessed):
            t = event.t  # Get the event time
            product_type = event.product_type  # Get the product type
            print(f"Managing queue for {product_type} at time {t}")
            # Log the queue management operation
            results["events"].append({"time": t, "event": "Queue management", "product_type": product_type})
            return NotificationResponse.ACK
        return NotificationResponse.NO_ACK

# Real-time control process - Handles dynamic adjustments during simulation
class RealTimeControl(Process):
    async def notify(self, event: Event):
        if isinstance(event, SimulationStarted):
            t = event.t  # Get the event time
            print(f"Real-time control initiated at time {t}")
            # Log the initiation of real-time control
            results["events"].append({"time": t, "event": "Real-time control initiated"})
            return NotificationResponse.ACK
        return NotificationResponse.NO_ACK

# Statistical analysis process - Collects and analyzes data
class StatisticalAnalysis(Process):
    async def notify(self, event: Event):
        if isinstance(event, FinalCheckCompleted):
            t = event.t  # Get the event time
            status = event.status  # Get the status of the final check
            print(f"Collecting statistical data at time {t} for status: {status}")
            # Log the statistical analysis operation
            results["events"].append({"time": t, "event": "Statistical data collected", "status": status})
            return NotificationResponse.ACK
        return NotificationResponse.NO_ACK

# Function to save simulation results in multiple formats
def save_results():
    # Save results to CSV
    df = pd.DataFrame(results["events"])  # Convert events to a DataFrame
    df.to_csv(os.path.join(SAVE_DIR, "results.csv"), index=False)

    # Save results to Excel
    df.to_excel(os.path.join(SAVE_DIR, "results.xlsx"), index=False)

    # Save results to JSON
    with open(os.path.join(SAVE_DIR, "results.json"), "w") as json_file:
        json.dump(results, json_file, indent=4)

    # Generate event timeline graph
    times = [event["time"] for event in results["events"]]
    event_types = [event["event"] for event in results["events"]]

    plt.figure(figsize=(10, 6))
    plt.plot(times, event_types, marker='o', linestyle='-', color='b')
    plt.xlabel('Time')
    plt.ylabel('Event Type')
    plt.title('Event Timeline')
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.savefig(os.path.join(SAVE_DIR, "event_timeline.png"))
    plt.show()

    # Generate summary statistics graph
```

```
    stats = results["statistics"]
    plt.figure(figsize=(10, 6))
    plt.bar(stats.keys(), stats.values(), color='g')
    plt.xlabel('Statistic Type')
    plt.ylabel('Count')
    plt.title('Summary Statistics')
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.savefig(os.path.join(SAVE_DIR, "summary_statistics.png"))
    plt.show()

# Function to run the case study simulation
async def run_case_study():
    world = HADES()  # Initialize the HADES simulation world

    # Register all the processes in the simulation
    world.register_process(PreProcessing())
    world.register_process(MainProcessing(seed="master_work"))
    world.register_process(ManualInspection())
    world.register_process(AGVTransport())
    world.register_process(AssemblyStation())
    world.register_process(FinalCheck())
    world.register_process(CustomArrivalProcess(seed="arrival_seed"))
    world.register_process(QueueManagement())
    world.register_process(RealTimeControl())
    world.register_process(StatisticalAnalysis())

    # Run the simulation for the specified duration
    await world.run(until=SIMULATION_DURATION)

    # Save the results after the simulation ends
    save_results()

# Run the simulation
if __name__ == "__main__":
    asyncio.run(run_case_study())  # Start the asynchronous simulation
```